

Deep Learning

"Data is the sword of the 21st century, those who wield it well, the Samurai." - **Gareth James**, Professor of Data Sciences and Operations, Marshall School of Business, University of Southern California.

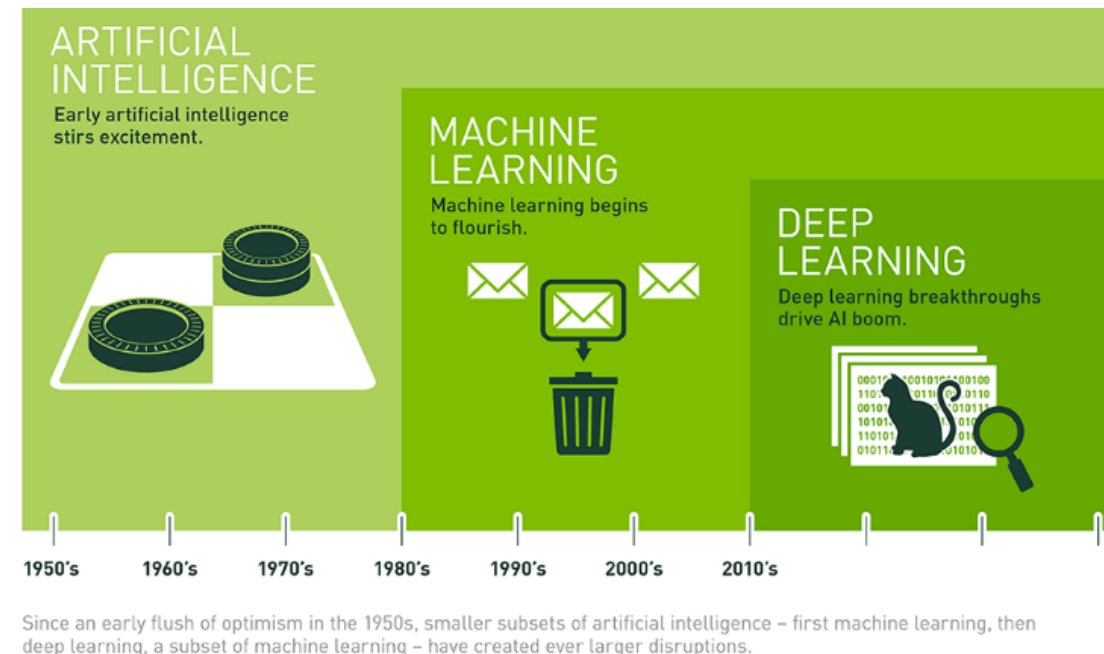
Dr. Poonam Seth Tiwari
Sc/Egr SG
Indian Institute of Remote Sensing
poonam@iirs.gov.in

Artificial Intelligence, Machine Learning and Deep Learning

AI: Machine mimic actions like humans

Machine Learning: Computer is trained to automate task. Uses data to feed an algorithm that can understand relationship between input and output. When machine finished learning, it can predict class of new data point

Deep Learning: Makes use of Deep neural network. Machine uses different layers to learn from the data. Depth of model is represented by number of layers of model



Computer Vision Tasks

- Computer vision has been the primary area of interest for ML
- The tasks include: classification, localization, object detection, instance segmentation

Classification



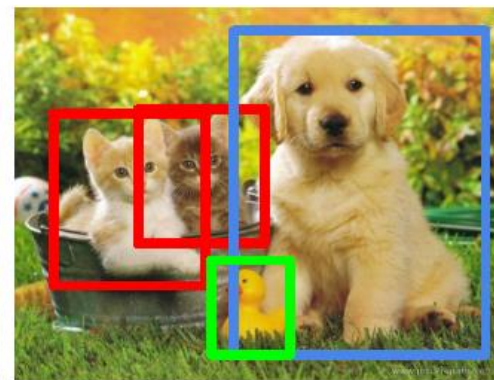
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



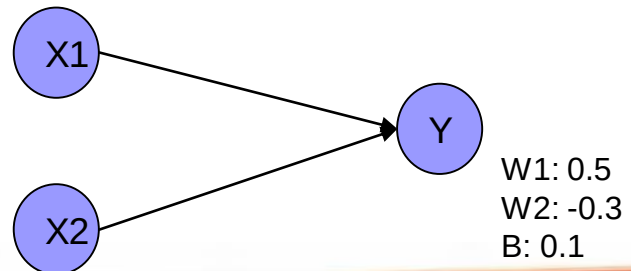
CAT, DOG, DUCK

Single object

Multiple objects

Artificial Neural Network

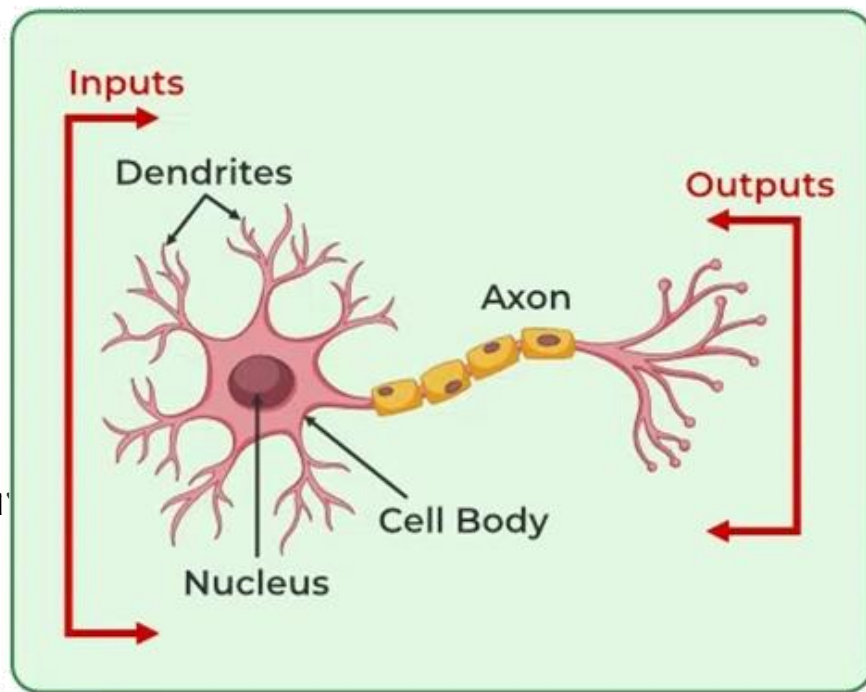
- An artificial neural network consists of a pool of simple processing units which communicate by sending signals to each other over a large number of weighted connections.
 - a set of processing units (cells).
 - an external input for each unit.
 - connections between the units. Generally each connection is defined by a weight.
 - a state of activation for every unit and an activation function
 - a propagation rule, which determines the effective input of a unit from its external inputs.
 - a method for information gathering (the learning rule).
 - an environment within which the system must operate, providing input signals and _ if necessary _ error signals.



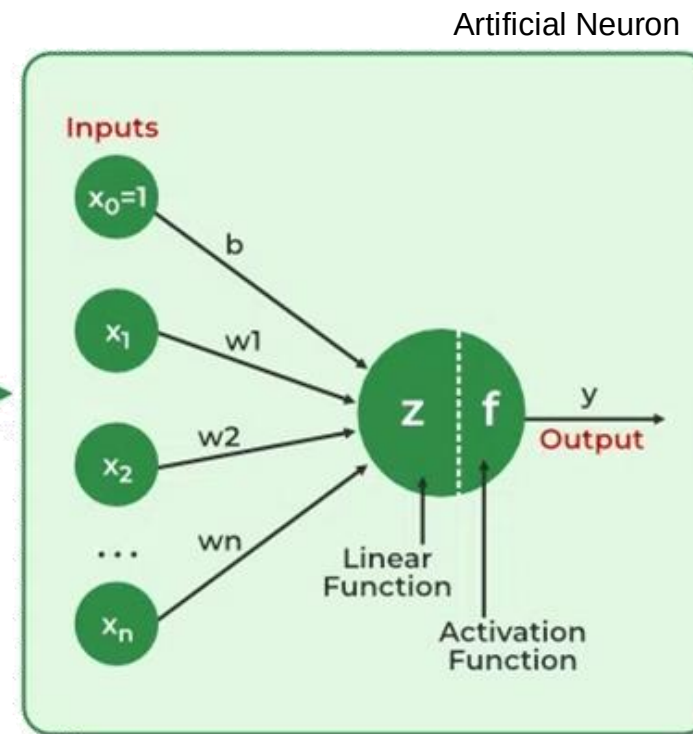
X1: Total Hours of Study
X2: Total Hours of Sleep/Rest
Y: Probability to Pass

Neural Networks

- A neural network is structured like the human brain and consists of artificial neurons, also known as nodes.

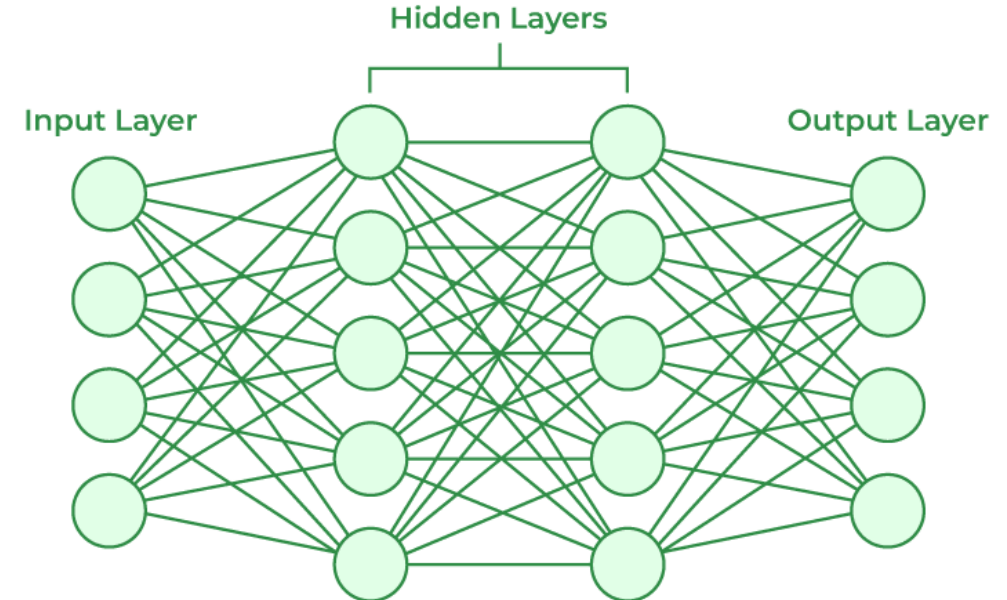
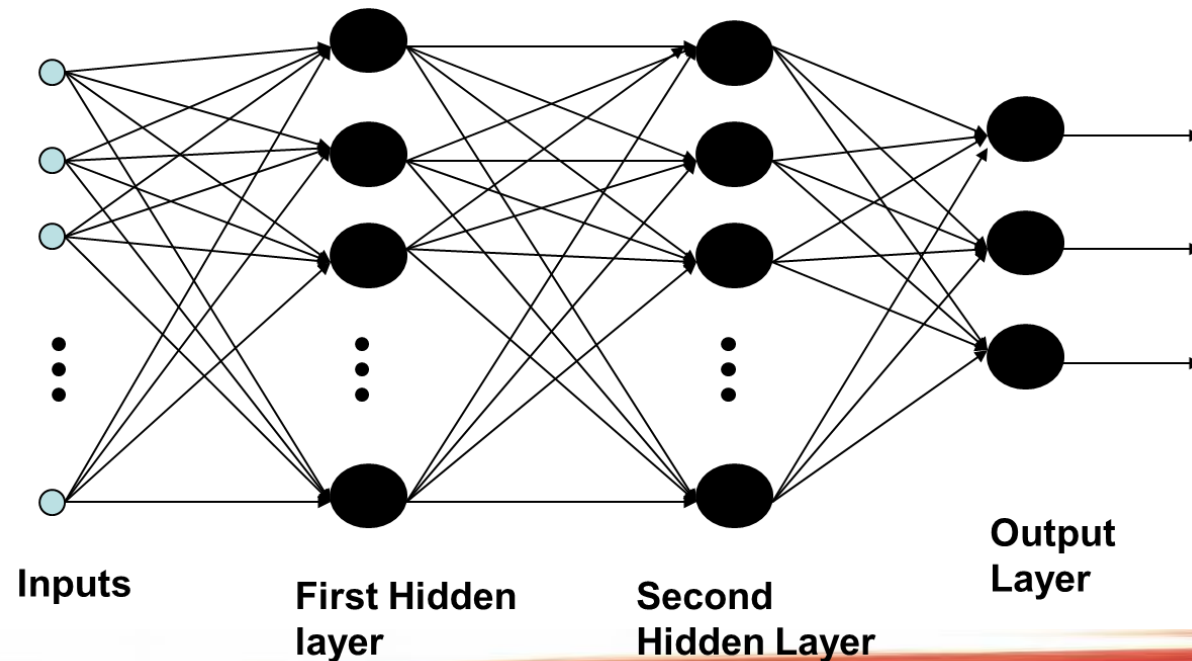


Dendrites: Input
Cell body: Processor
Synaptic: Link
Axon: Output



- A neuron is connected to other neurons through about 10,000 synapses
- A neuron receives input from other neurons. Inputs are combined
- Once input exceeds a critical level, the neuron discharges a spike - an electrical pulse that travels from the body, down the axon, to the next neuron(s)
- Data provides each node with information in the form of inputs. The node multiplies the inputs with random weights, calculates them, and adds a bias.

- Nodes are stacked next to each other in three layers:
 - The input layer:** This is the very first layer of the ANN in which there are placed number of neurons equals the number of input variables. This is the layer through which the input data is fed into the ANN.
 - The hidden layer(s):** These are the layers between the input and output layers in the ANN. Their role is to perform intermediate computations and feature transformations as well. There is no fixed number for them there can be one or more hidden layers in an ANN.
 - The output layer:** The final estimate of the output. This layer produces the output of the network as a whole. The number of neurons in the output layer must be equal to the number of classes (desired outputs).

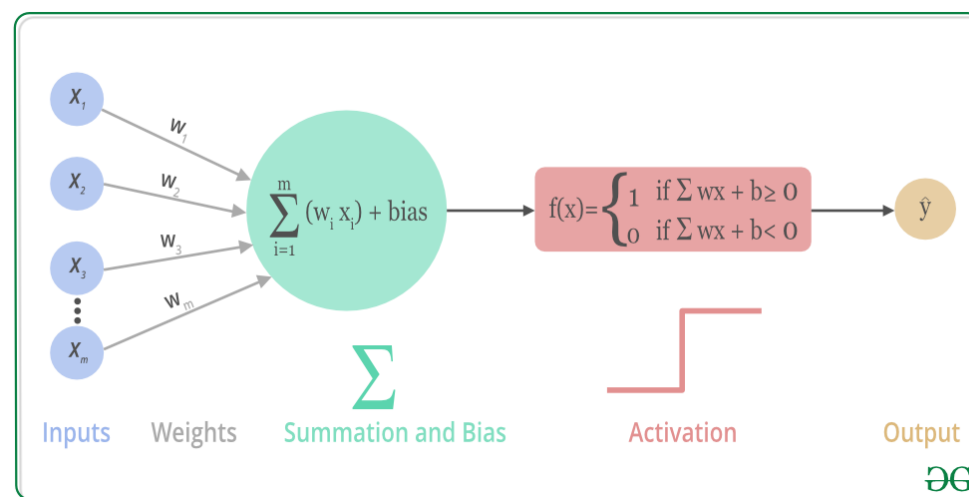


How Information Flows Through the Network

- 1. Input:** First the input data is fed into the neurons in the input layer.
- 2. Weighted Sum:** Each neuron in the layer simply calculates the weighted sum of all its inputs. If $x_1, x_2, \dots, x_n$ are inputs and $w_1, w_2, \dots, w_n$ are the corresponding weights then the weighted sum will be calculated as

$$Z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$
("b" is referred to as bias)
- 3. Activation Function:** The next step is the passing of the above-weighted sum (z) through the activation function. Because of this non-linearity is being produced in the neuron's output (a)

$$a = \text{activation}(z)$$
- 4. Propagation:** This is the step in which the output (a) is then passed to the neurons in the next layer. The process repeats for each of the layers until the output layer is reached.



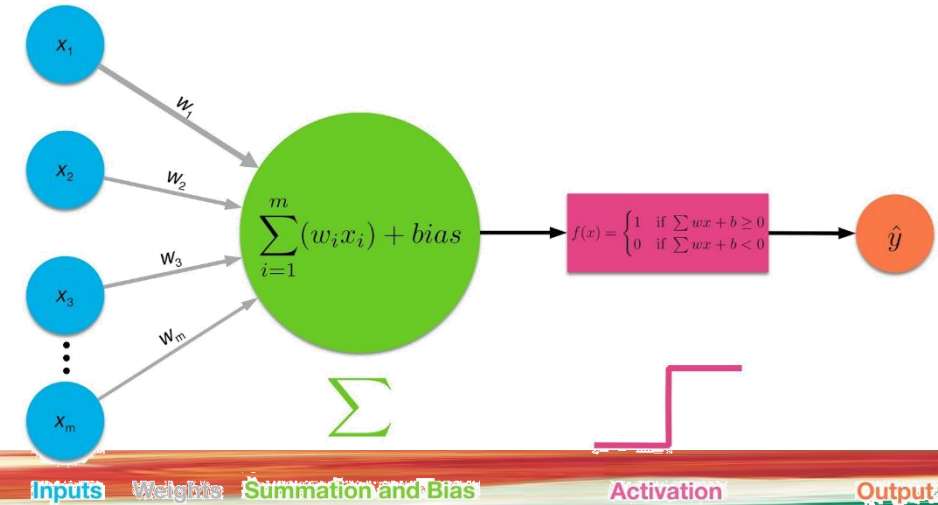
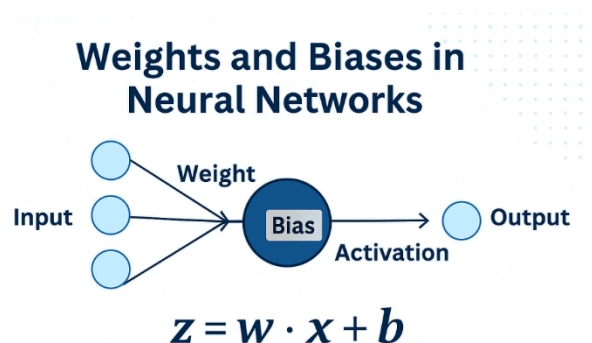
Concept of Weights and Biases

Weights:

- Weights are the parameters that determine the importance of every input to a neuron.
- During training, we can adjust the weights to minimize the error and efficient training of the network.
- The weight (w_i) that is associated with the input (x_i) simply is controlling the strength and direction of the influence of the output of the neuron.

Biases:

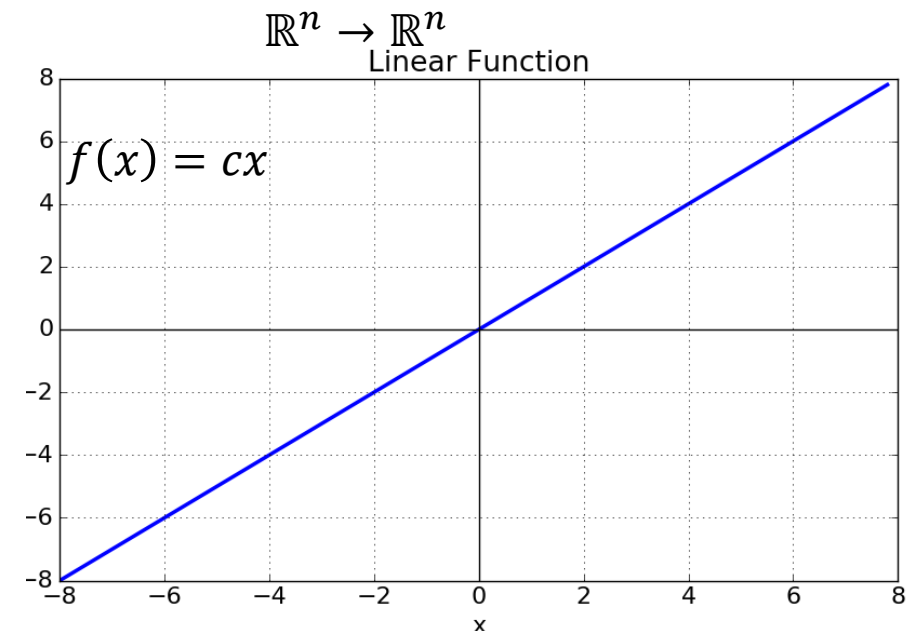
- It is one additional parameter that is added to the weighted sum before applying the activation function. Bias allows the activation function to shift to left or right. This shift helps the network to fit the data more efficiently.
- If all the outputs are zero for suppose then bias (b) here ensures that the neuron can also have some non-zero output as well.



Activation Function

- Activation functions are the backbone of ANN that simply enables it to handle and solve complex tasks.
- The simple logic of activation functions is it produces nonlinearity in the network that makes it capable enough to even learn complex features and relationships.
- If we remove the activation functions from the ANN then it would be the same as a simple linear model and that is the next step in how the limitations of linear models can be overcome
- *Linear function* means that the output signal is proportional to the input signal to the neuron

- If the value of the constant c is 1, it is also called **identity activation function**
- This activation type is used in regression problems
 - E.g., the last layer can have linear activation function, in order to output a real number (and not a class membership)

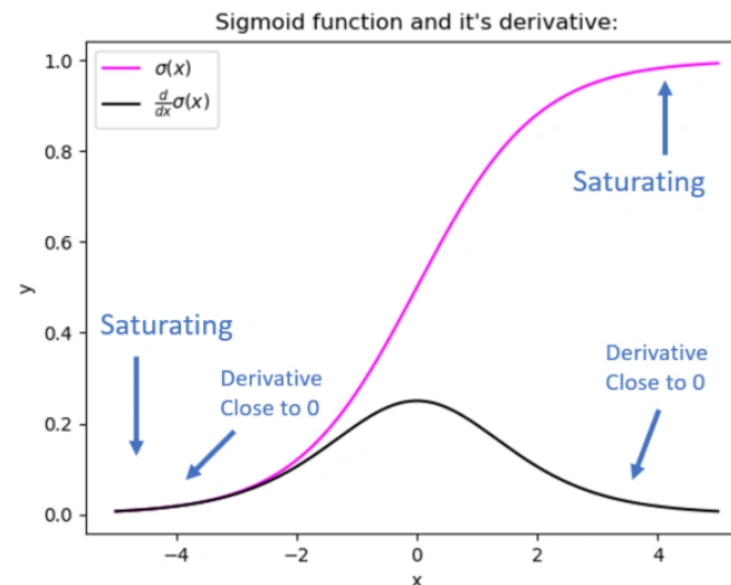
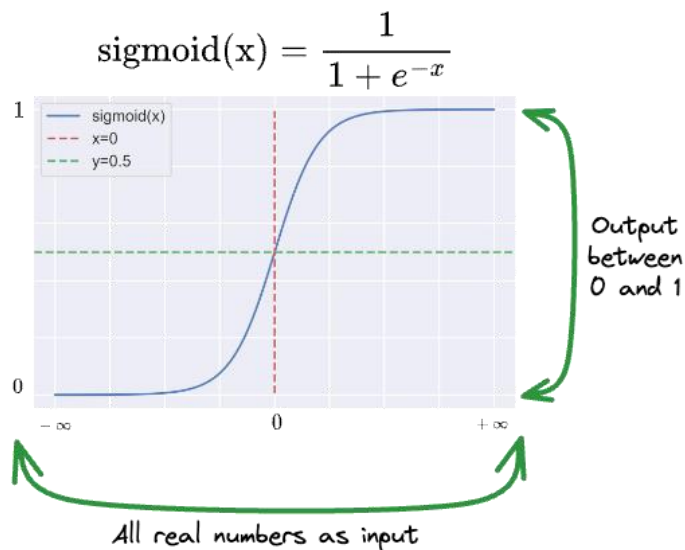


Non-linear activations are needed to learn complex (non-linear) data representations

Sigmoid Activation Function:

It is an S-shaped curve that does not pass through the origin. It produces an output that lies between 0 and 1. The output values are often treated as a probability. It is often used for **binary classification**.

Sigmoid activation function has a problem of "Vanishing Gradient".



$$\sigma'(x) = \frac{1}{(1+e^{-x})} \left(1 - \frac{1}{(1+e^{-x})}\right)$$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

Advantage :-

a) As output of sigmoid function ranges between 0 to 1 hence this helps to normalize (or scale data) and to achieve non linearity of the data set.

Disadvantage :-

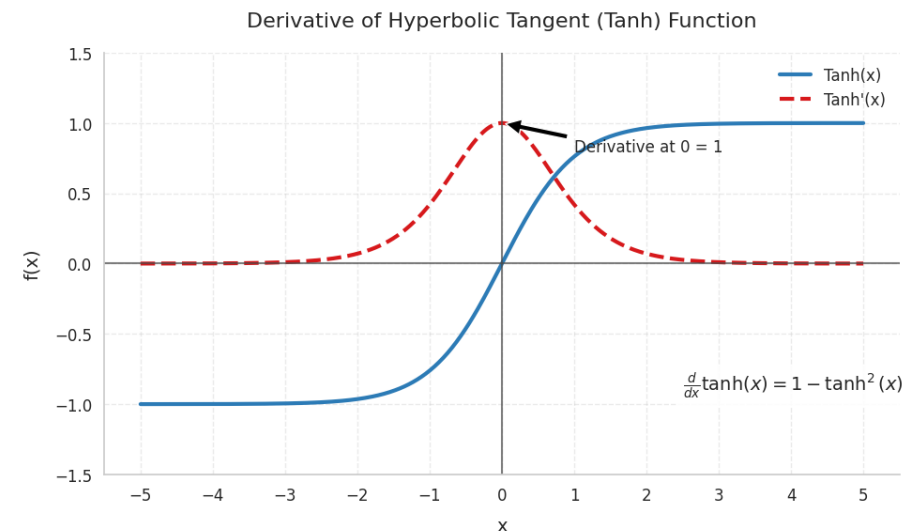
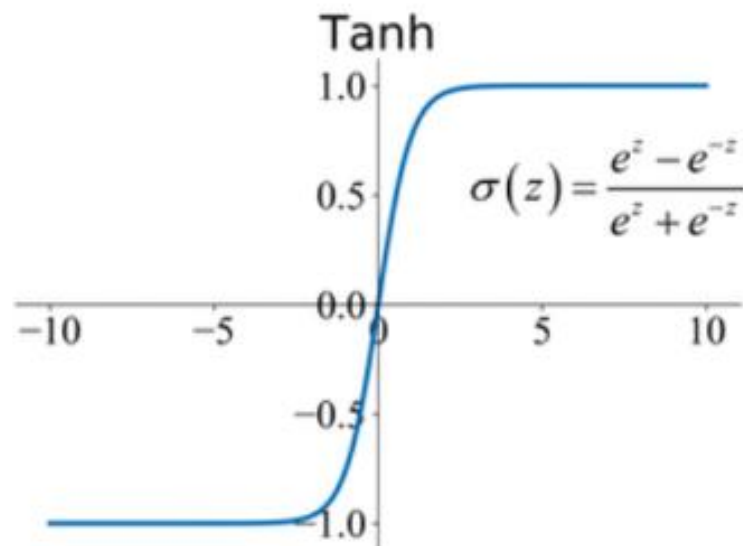
a) As Sigmoid function is not a zero centric function (does not cross origin) so it always gives gradient into +ve direction, which is not good and make optimiser hard to learn/adjust weight of NN.

b) **Vanishing gradient issue**: As the differential value of Sigmoid function ranges between 0 to 1, hence effect of gradient decreases layer by layer which cause issue during learning of neural network or very less impact on first layer of the Neuron.

c) As Sigmoid function is combination of exponential function (exp(x)), hence it required lots of computational power which makes this more expensive.

Tanh Activation Function:

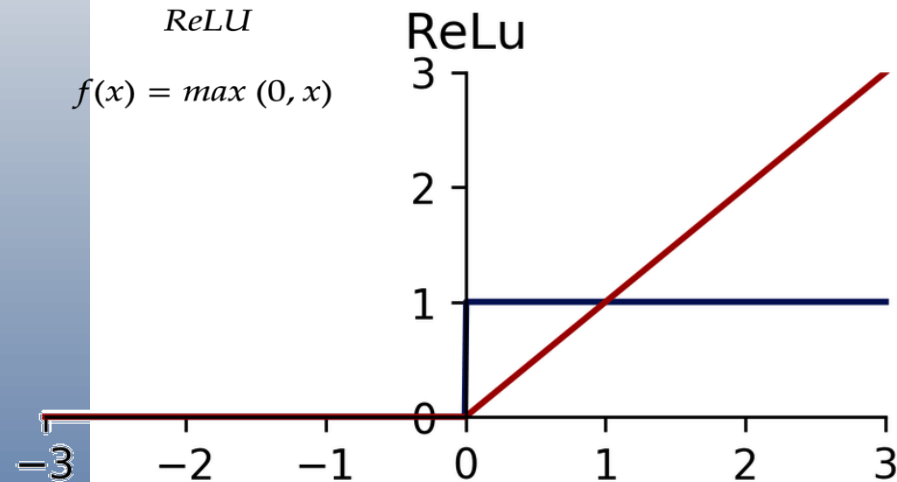
Tanh function is a non-linear and differentiable function similar to the sigmoid function but output values range from -1 to +1. It is an S-shaped curve that passes through the origin



- Symmetry Around Zero:** Unlike the Sigmoid function, which only outputs between 0 and 1, Tanh's range of -1 to 1 enables **zero-centered outputs**, which can speed up **gradient descent** by reducing bias shifts.
- Improved Gradient Flow:** Tanh maintains a *steeper gradient* around its center, making it easier for the neural network to learn during **backpropagation** compared to Sigmoid. This characteristic helps avoid some of the issues caused by the **vanishing gradient problem**.
- Balanced Output:** Since Tanh maps both negative and positive values, it's often more effective than ReLU in networks that require **balanced data**.
- Computationally Intensive:** Tanh requires computing exponential functions, which can be slower than simpler functions like ReLU, especially in very large networks.

ReLU Activation Function:

ReLU stands for Rectified Linear Activation function. It is a non-linear function and, graphically ReLU has the following transformative behavior:



$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

The derivative is simple but undefined at $x=0$. By convention, it is often set to 0 at this point.

Advantages

- Simplicity:** ReLU is computationally efficient as it involves only a thresholding operation.
- Non-Linearity:** Although it seems like a piecewise linear function, ReLU is still a non-linear function. This allows the model to learn more complex data patterns and model intricate relationships between features.
- Sparse Activation:** ReLU's ability to output zero for negative inputs introduces sparsity in the network, meaning that only a fraction of neurons activate at any given time. This can lead to more efficient and faster computation.
- Gradient Computation:** ReLU offers computational advantages in terms of backpropagation, as its derivative is simple—either 0 (when the input is negative) or 1 (when the input is positive). This helps to avoid the vanishing gradient problem, which is a common issue with sigmoid or tanh activation functions.

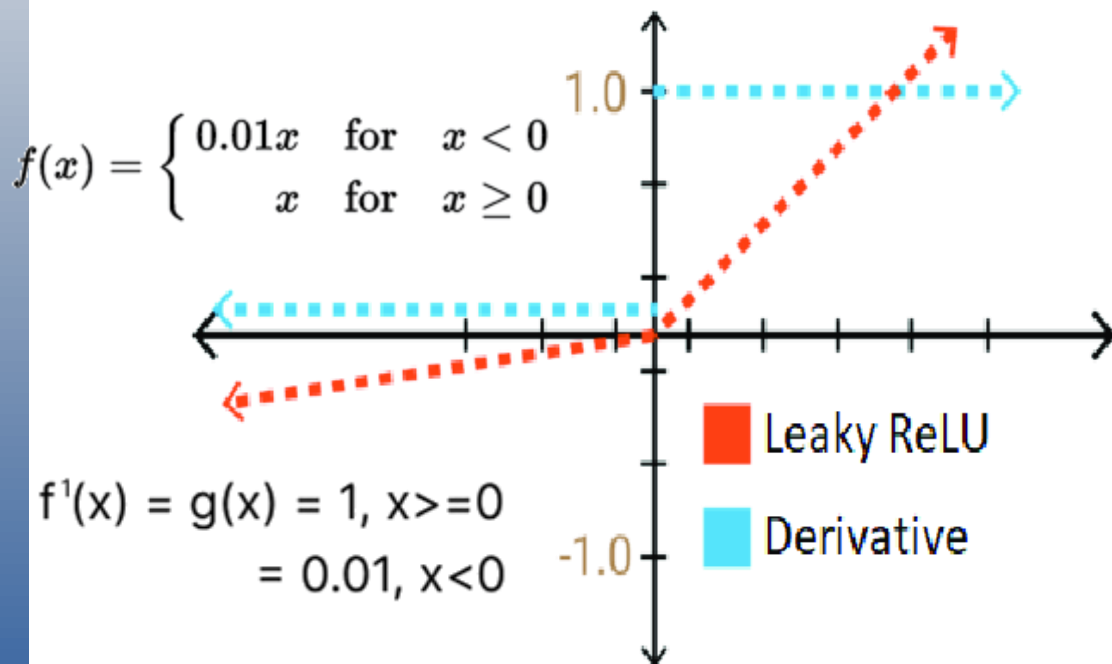
Drawbacks

- Dying ReLU Problem:** One of the most significant drawbacks of ReLU is the "dying ReLU" problem, where neurons can sometimes become inactive and only output 0. This happens when large negative inputs result in zero gradient, leading to neurons that never activate and cannot learn further.
- Unbounded Output:** Unlike other activation functions like sigmoid or tanh, the ReLU activation is unbounded on the positive side, which can sometimes result in exploding gradients when training deep networks.
- Noisy Gradients:** The gradient of ReLU can be unstable during training, especially when weights are not properly initialized. In some cases, this can slow down learning or lead to poor performance.

Leaky ReLU Activation Function:

Leaky ReLU introduces a small slope for negative values instead of outputting zero, which helps keep neurons from "dying".

This function is very useful as when the input is negative the differentiation of the function is not zero. Hence the learning of neurons doesn't stop.



Instead of the function being 0 when $x < 0$, a leaky ReLU has a small negative slope (e.g., $\alpha = 0.01$, or similar)

Advantages :

- Mitigates the "Dying ReLU" problem:** Unlike standard ReLU, which outputs zero for all negative inputs, Leaky ReLU allows a small, non-zero gradient for negative inputs. This prevents neurons from becoming inactive (or "dead") and unable to learn during training if their input consistently falls into the negative region.

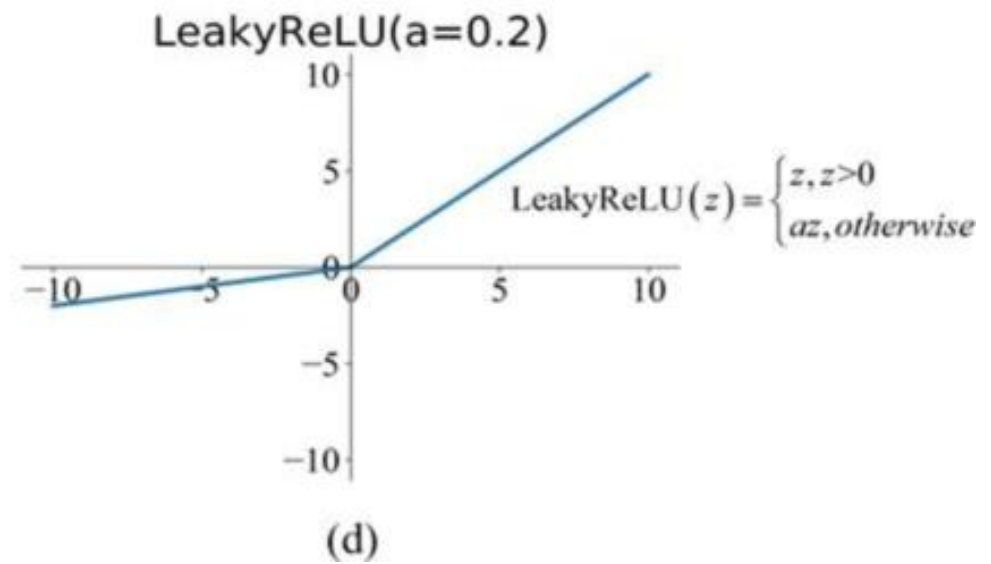
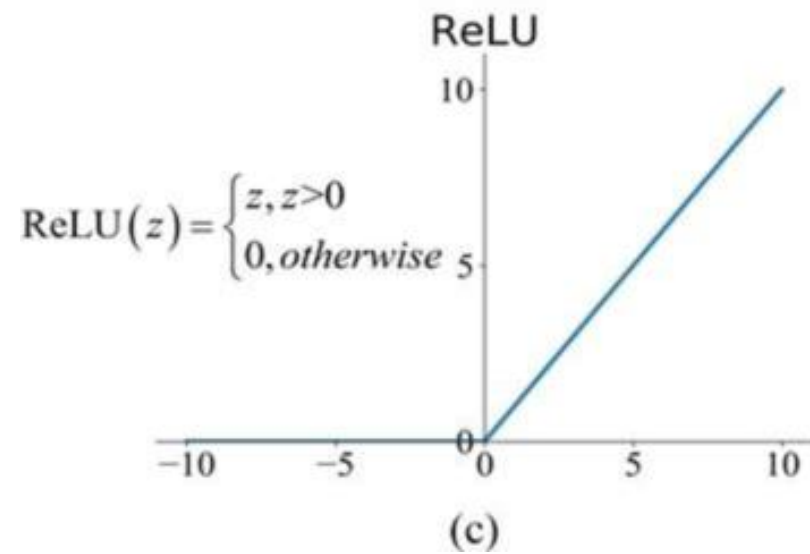
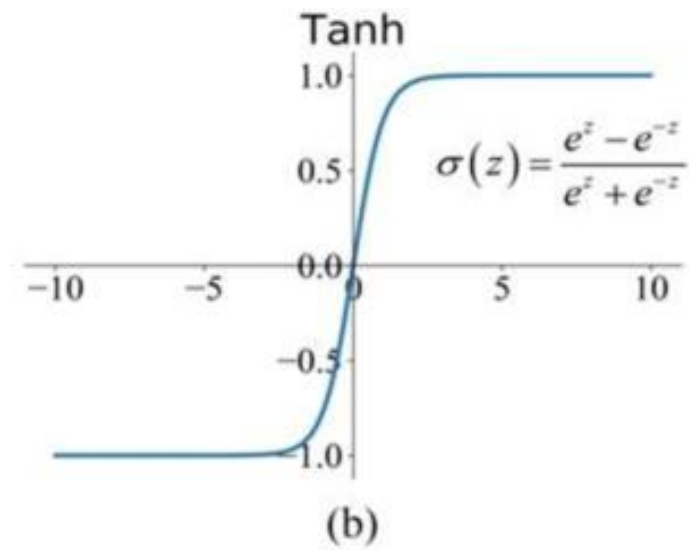
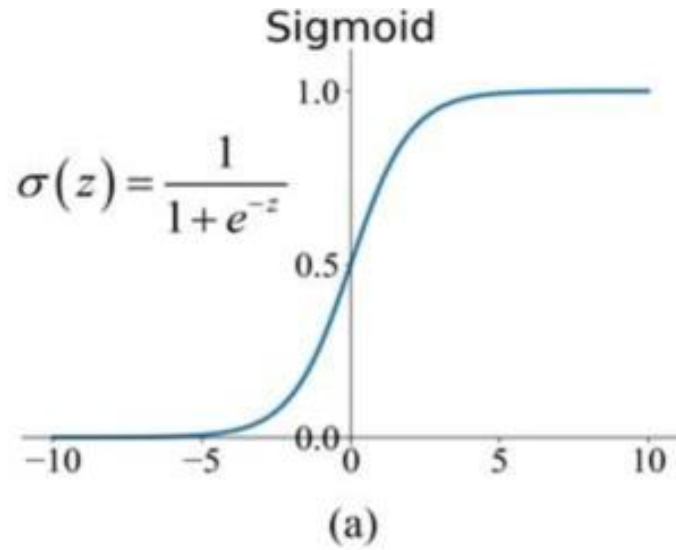
- Maintains gradient flow:** By allowing a small gradient for negative inputs, Leaky ReLU ensures that gradients can still flow during backpropagation, even when inputs are negative, which can help in training deep networks.

- Computationally efficient:** Similar to ReLU, Leaky ReLU is computationally efficient due to its simple linear form for both positive and negative inputs.

Disadvantages :

- Introduction of a hyperparameter:** The "leakiness" parameter (the small constant multiplied by negative inputs) needs to be chosen, typically through hyperparameter tuning.

- Not zero-centered:** Like ReLU, Leaky ReLU is not a zero-centered function, meaning its output is primarily positive. This can sometimes lead to issues during optimization.



Softmax Activation Function:

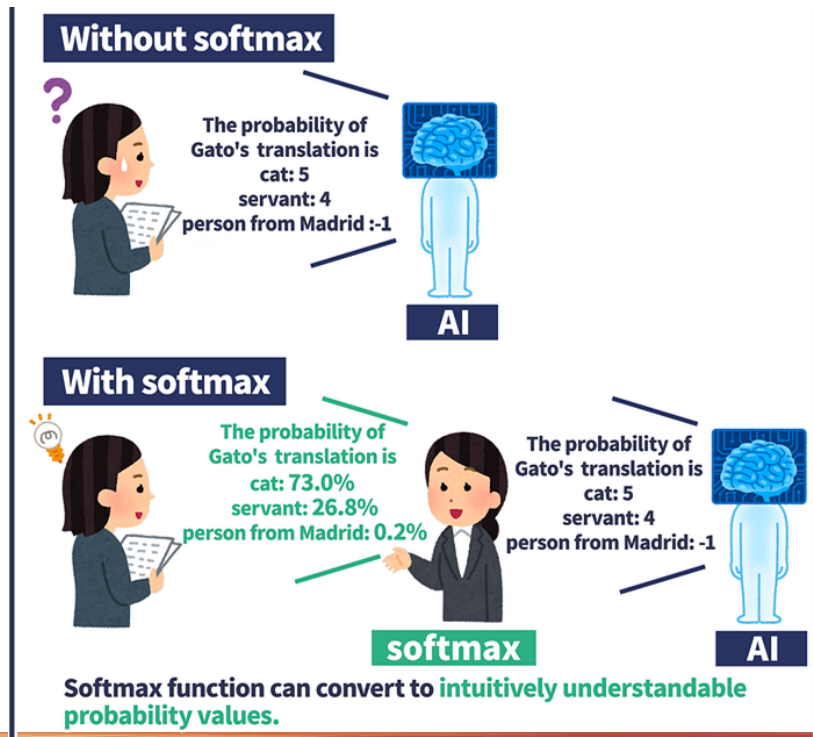
- The softmax function is different from other activation functions as it is placed at the last to normalize the output.
- Softmax function is often described as a combination of multiple sigmoid.
- We can use other activation functions in combination with Softmax to produce the output in probabilistic form.
- It is used in **multiclass classification** and generates an output of probabilities whose sum is 1.
- The range of output lies between 0 and 1

$$\text{Softmax} = e^{x_a} / \left(\sum_{a=1}^n e^{x_a} \right) = e^{x_1} / (e^{x_1} + e^{x_2} + e^{x_3})$$

$$\frac{\partial(\text{Softmax})}{\partial x_1} = (e^{x_1} \times (e^{x_2} + e^{x_3})) / (e^{x_1} + e^{x_2} + e^{x_3})^2$$

Input	softmax	Output
5	148.4	0.730 (73.0%)
4	54.6	0.268 (26.8%)
-1	0.4	0.002 (0.2%)

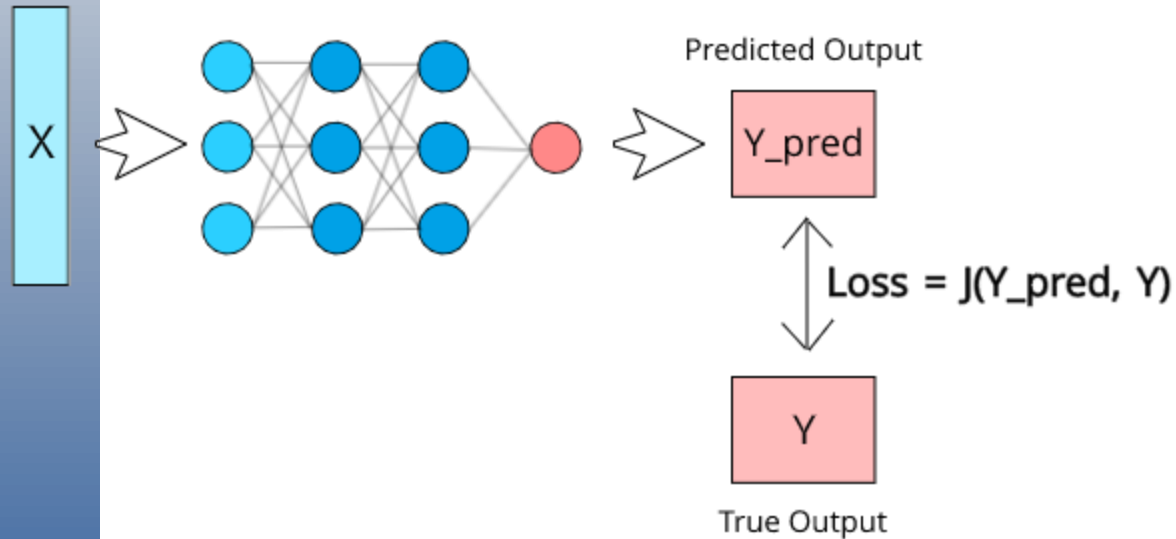
In softmax, (1) exponentiation (2) dividing by the total.



Loss / Cost /Error Function:

- A loss function in neural networks is a mathematical function that quantifies the difference between a model's predicted output and the true, actual value.
- Loss function guides the optimization process to improve model accuracy..
- Different types of loss functions, such as Mean Squared Error (MSE) for regression and Cross-Entropy Loss for classification, are chosen based on the specific task.

Input Data



$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

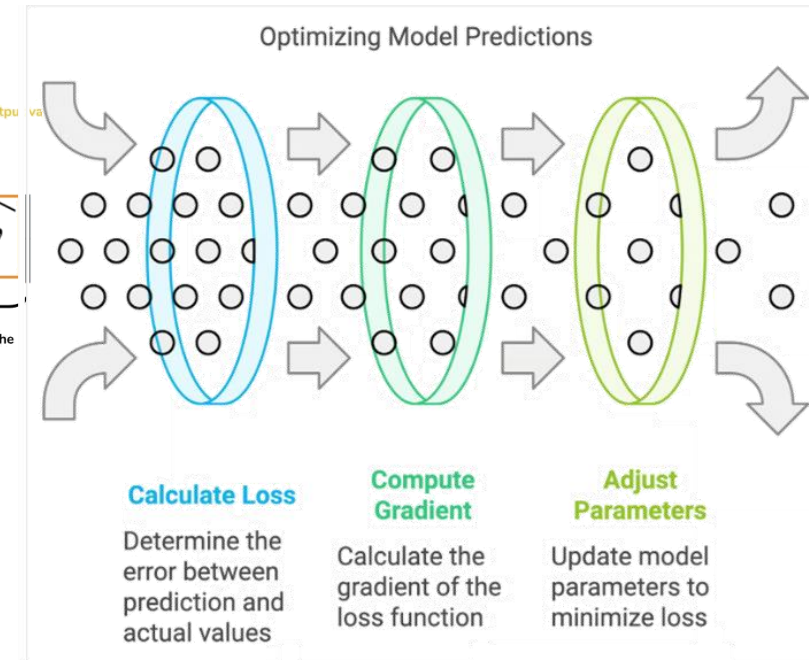
Divide by the total number of data points

Actual output value

Predicted output value

Sum of

The absolute value of the residual



Loss Function:

A loss function/error function is for **single training** example/input.

Cost Function:

A cost function, on the other hand, is the average loss **over the entire training dataset**.

Common Loss function in Deep Learning

- 1.MSE(Mean Squared Error)
- 2.MAE(Mean Absolute Error)
- 3.Huber loss
- 4.Binary cross-entropy
- 5.Categorical cross-entropy
- 6.KL Divergence

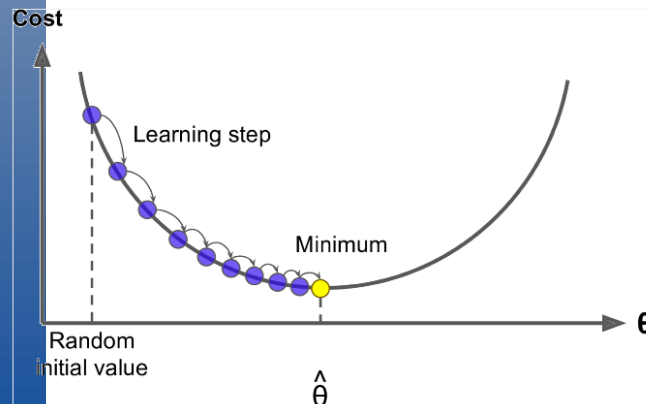
Optimiser

Optimizers are responsible for updating the weights of the neural network during the training process. Algorithms that help in finding the optimal set of weights that minimize the loss function.

Gradient Descent

Uses calculus to consistently modify the values and achieve the local minimum.

1. Initialize Coefficients
2. Evaluate Cost
3. Search for Lower Cost
4. Update Coefficient
5. Repeat Process
6. Reach Local Minimum



Expensive to calculate the gradients if the size of the data is huge.

Stochastic Gradient Descent

Instead of processing the entire dataset during each iteration, we randomly select batches of data



SGD uses a higher number of iterations to reach the local minima. Due to an increase in the number of iterations, the overall computation time increases. But even after increasing the number of iterations, the computation cost is still less than that of the gradient descent optimizer.

Adam (Adaptive Moment Estimation)

Adam is ideal when you have a complex problem with a large dataset, especially when the data is noisy or has a lot of variability.

Adaptive Learning Rate: Adam automatically adjusts the learning rate during training, making it more flexible and often leading to faster convergence.

Handles Noise Well: It's robust to noise in the data, which can be particularly useful in real-world scenarios where data is rarely perfect.

Widely used in deep learning models like CNNs and RNNs for tasks such as image classification or language processing

Training an ANN:

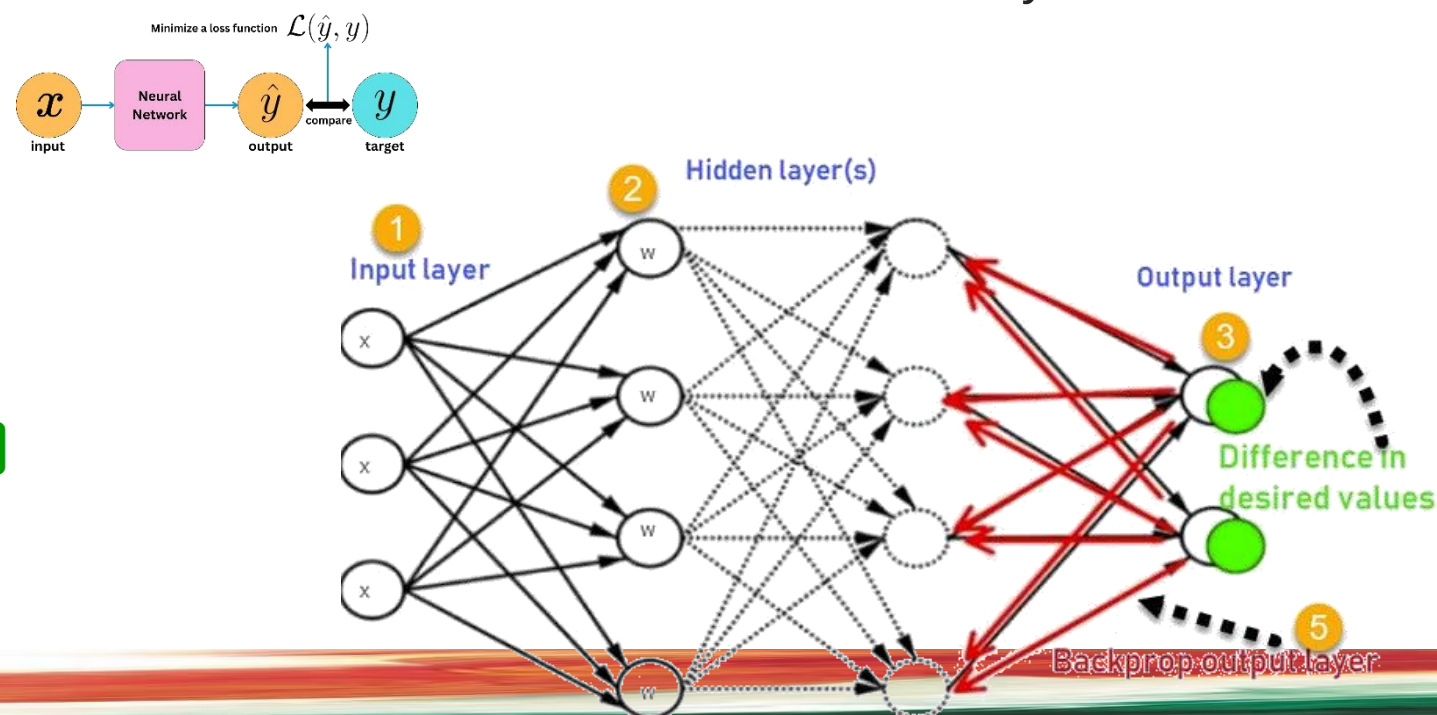
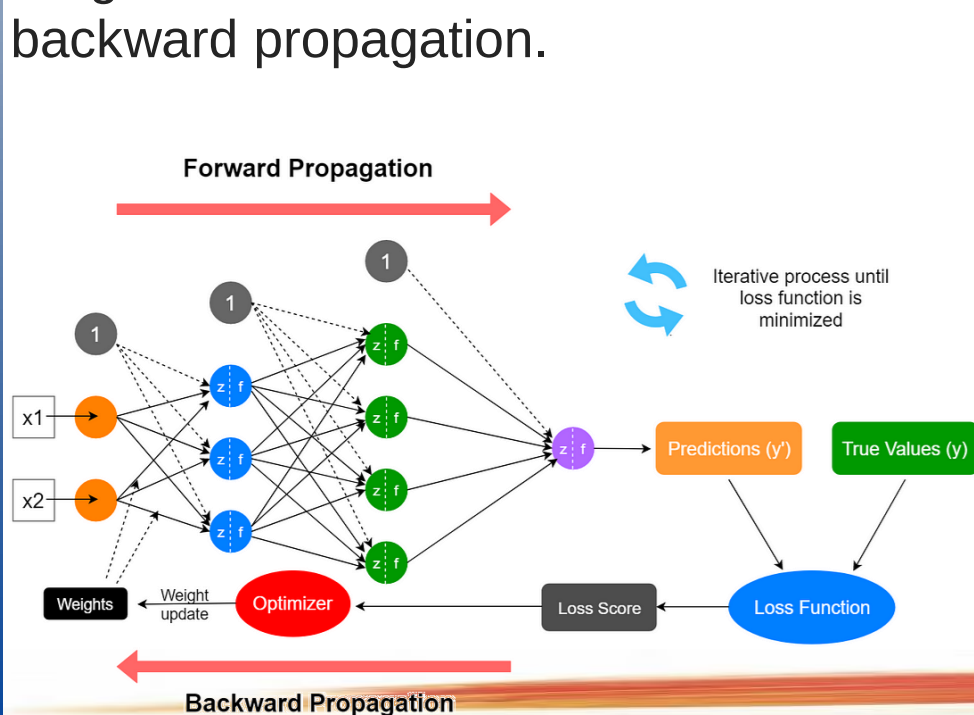
The training of the ANN involves the adjustment of the weights and biases that is done to minimize the error of predictions of the network.

1. Forward Propagation:

It is the simple method of computing the output by passing the input that is passing through the network. it is also very important to note that the model never learns in forward propagation it only learns through backward propagation.

2. Backward Propagation:

It is a common training algorithm that is used in the ANN to calculate the error and update the weights and biases in the form of backward direction. remember that the model always learns in the backward propagation.



1

- Assign Random weights to all the linkages to start the algorithm

2

- Using the inputs and the (Input ->Hidden node) linkages find the activation rate of Hidden Nodes

3

- Using the activation rate of Hidden nodes and linkages to Output, find the activation rate of Output Nodes

4

- Find the error rate at the output node and recalibrate all the linkages between Hidden Nodes and Output Nodes

5

- Using the Weights and error found at Output node, cascade down the error to Hidden Nodes

6

- Recalibrate the weights between hidden node and the input nodes

7

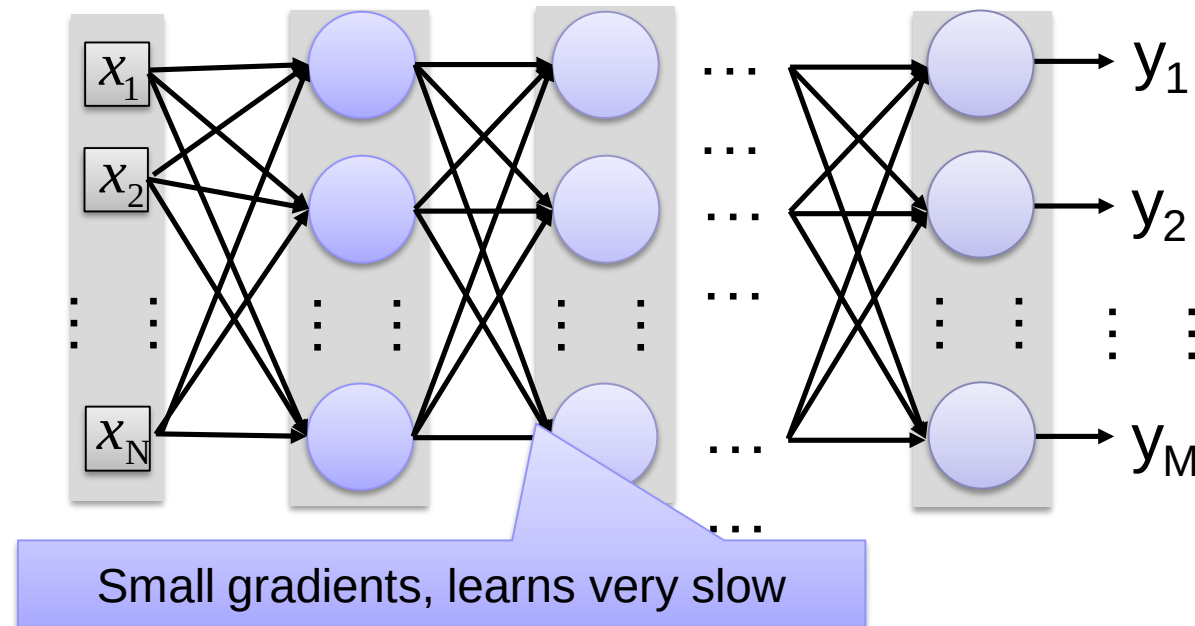
- Repeat the process till the convergence criterion is met

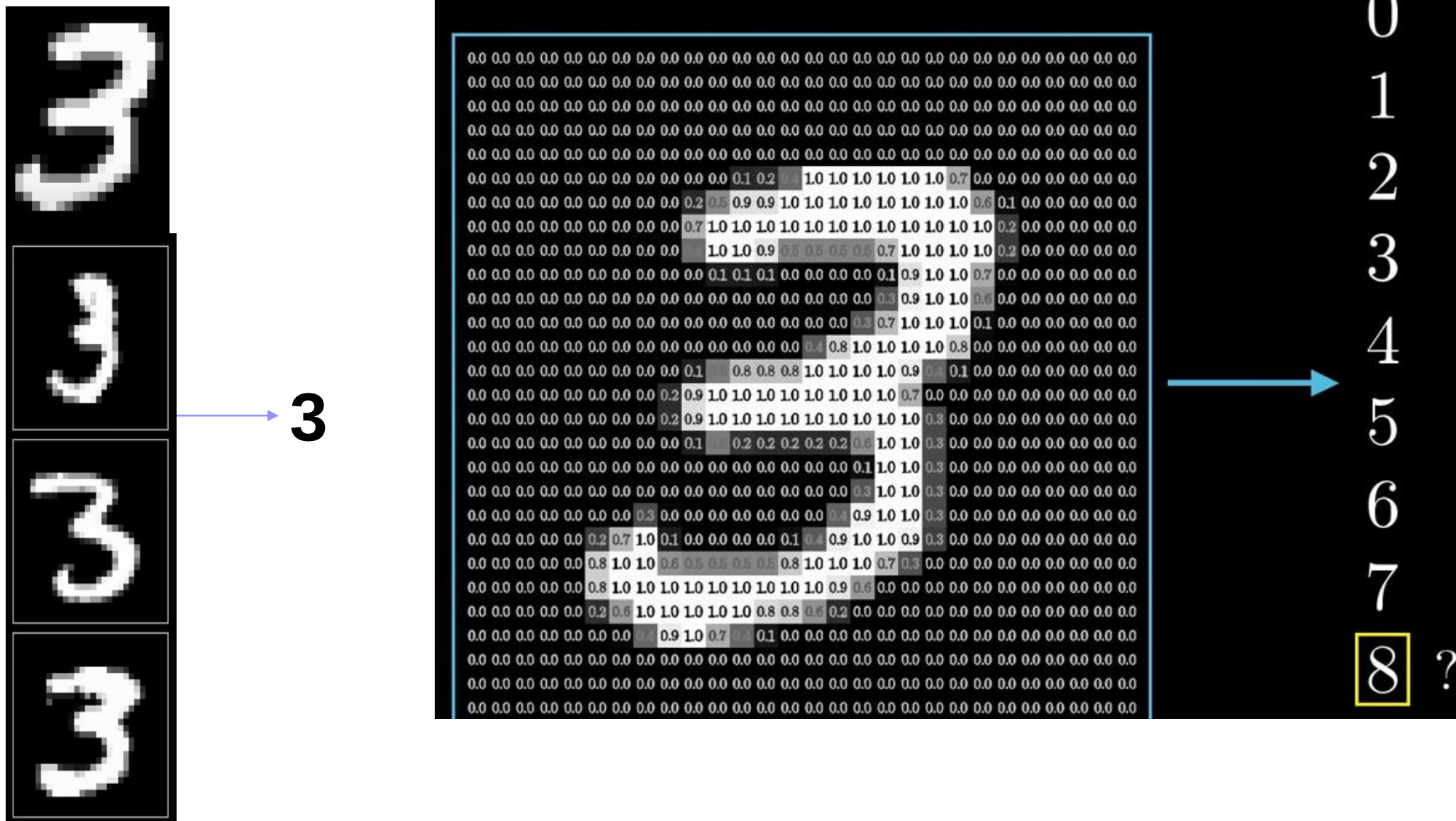
8

- Using the final linkage weights score the activation rate of the output nodes

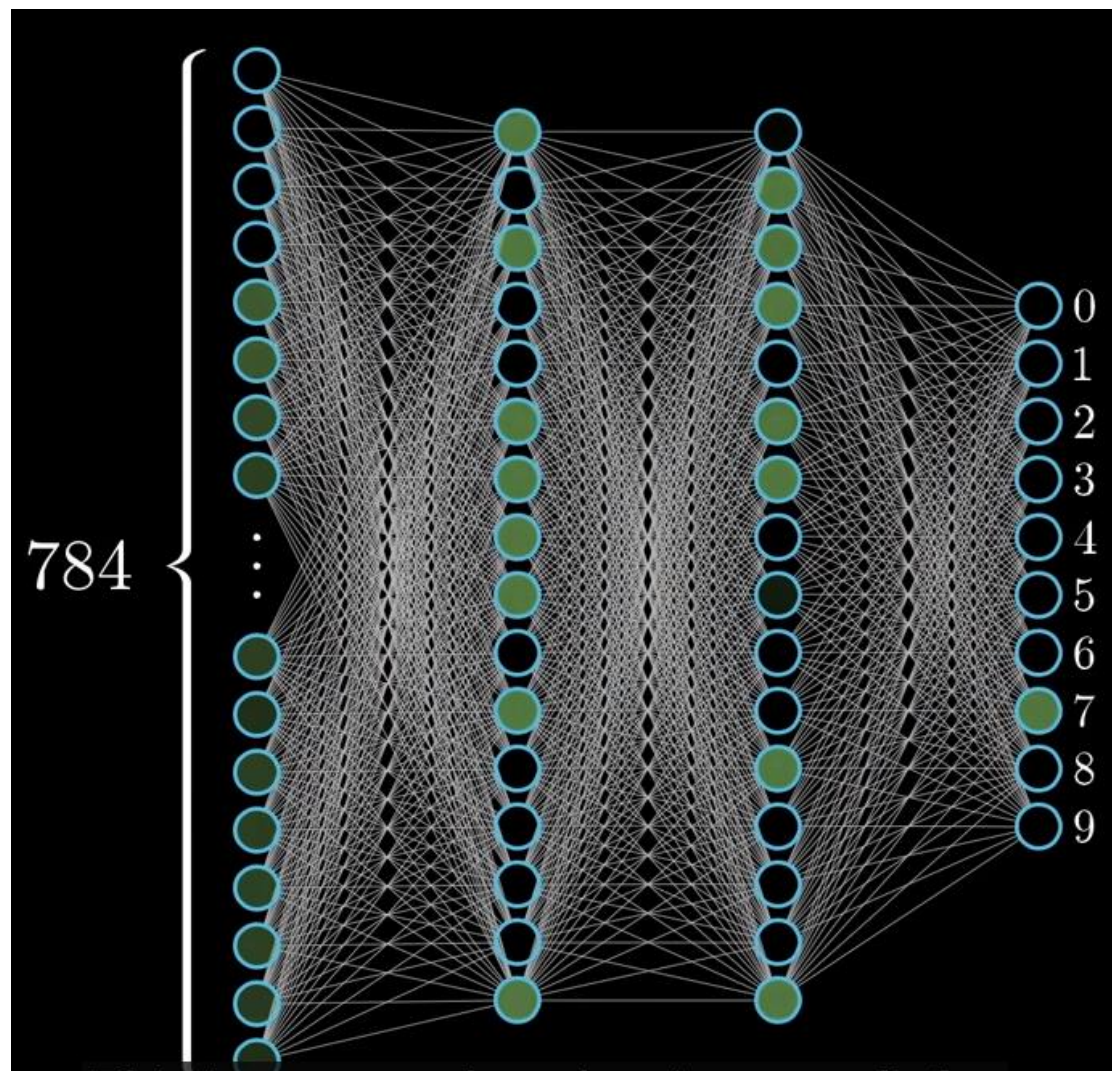
Vanishing Gradient Problem

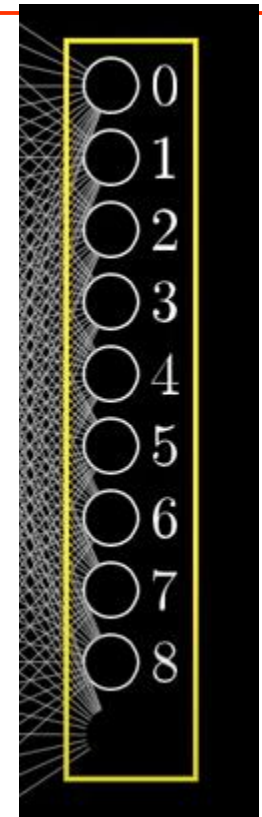
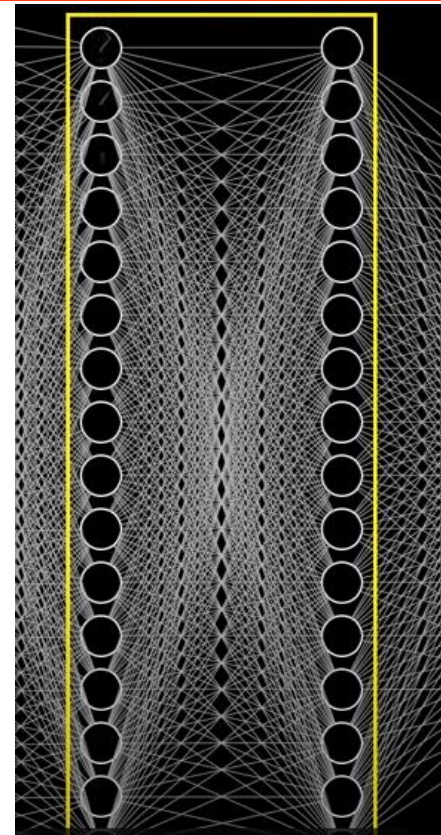
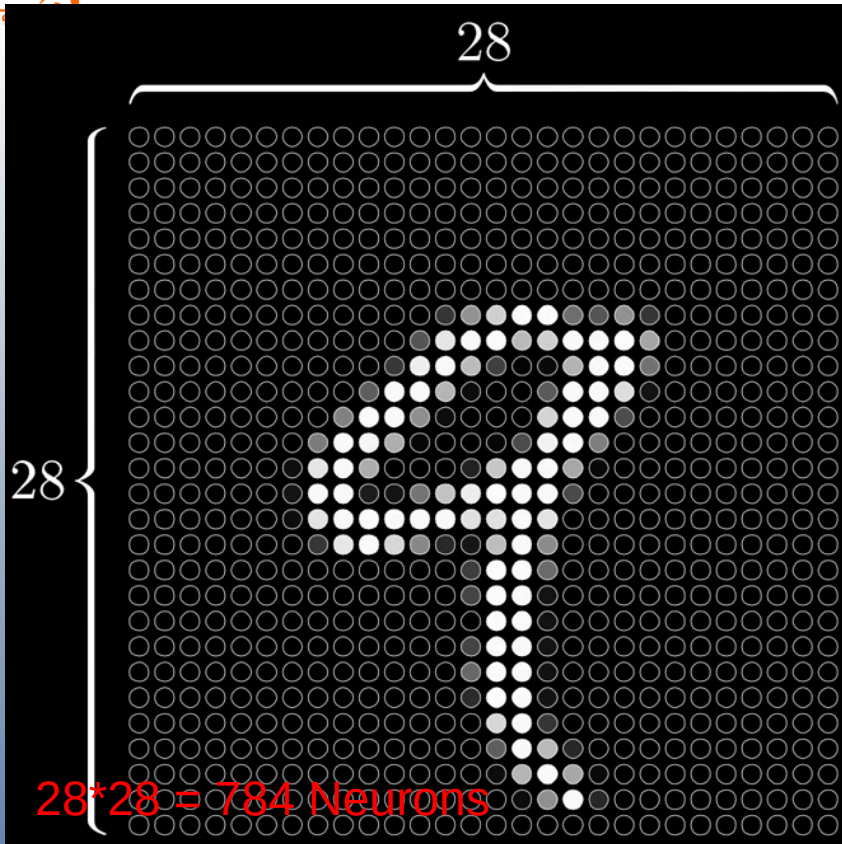
- In some cases, during training, the gradients can become either very small (vanishing gradients) or very large (exploding gradients)
 - They result in very small or very large update of the parameters
 - Solutions: change learning rate, ReLU activations, regularization, LSTM units in RNNs





Structural Component of Neural Network





- Holds number that represent grey scale value of corresponding pixel (0-1)
- This number k/as activation number
- Neuron is lit up if it has high activation number
- 784 neurons make first layer of network

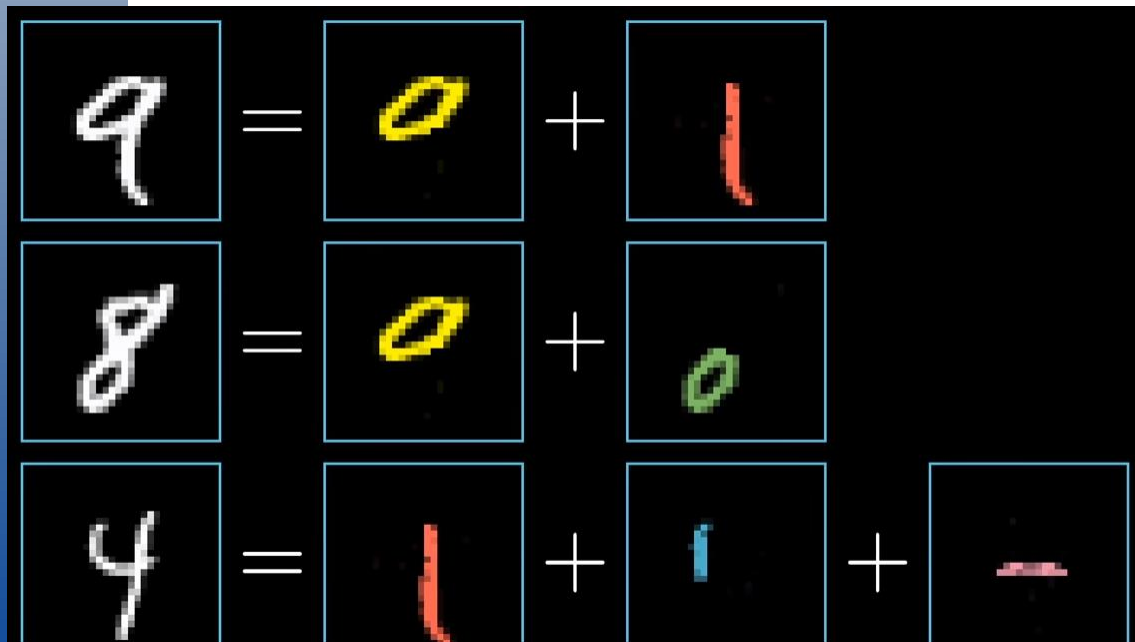
- Number of layers and neurons can be arbitrarily chosen
- Activation in one layer determine activation in next layer

- Has 10 neurons
- Each neuron represents one number
- Activation number represents correspondence between given image with digit

Big Question: How activation in one layer affects the next layer

- If you feed an image lighting up all 784 neurons of input layer according to brightness of each pixel in the image.
- That pattern of activation causes very specific pattern in the next layer.
- This causes some pattern in the one after it. This finally gives some pattern in the output layer.
- The brightest neuron of the output network is the network's choice for digit represented in the image.

Big Question: What do the middle layers do

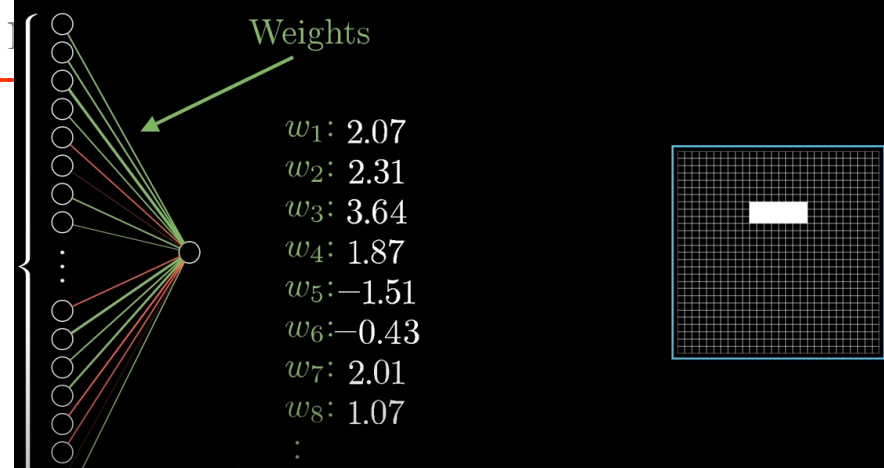
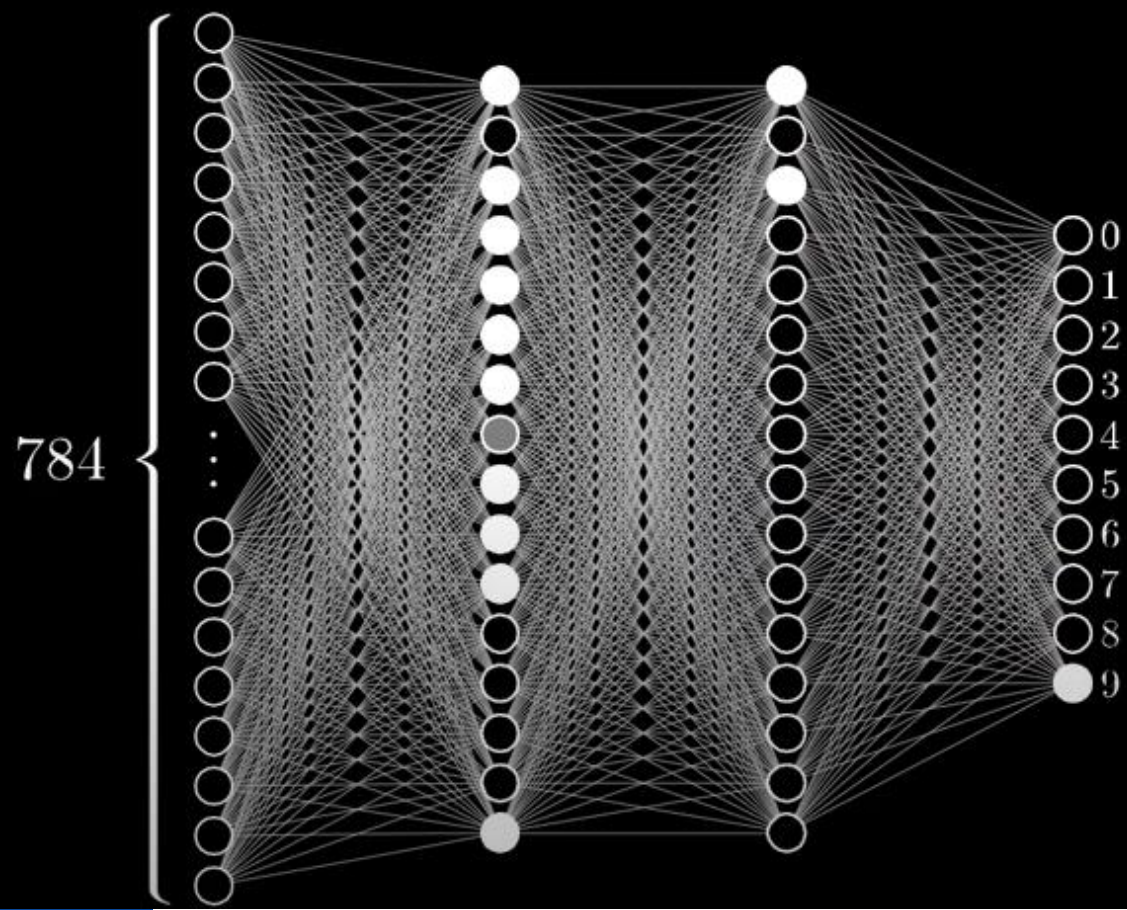


- Each neuron in second to last layer corresponds to one of these sub features
- Which combination of sub features corresponds to which digits
- Sub features identified by identifying little edges
- The second layer corresponds to identifying relevant edges

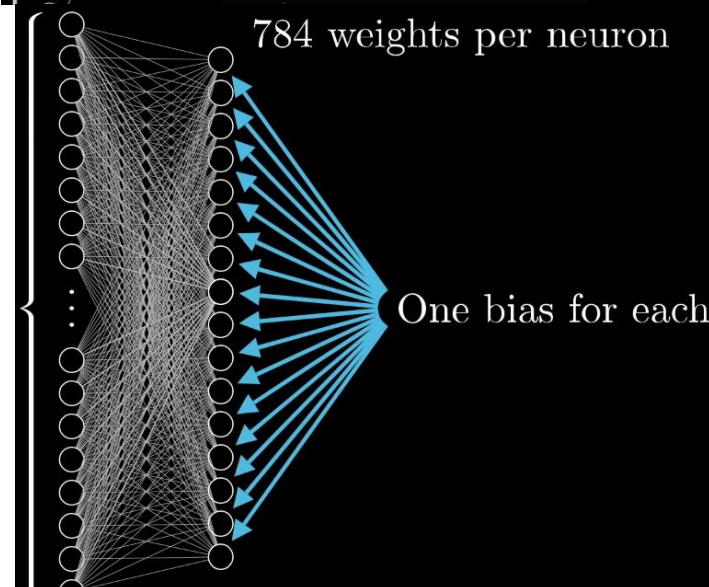
Human brain identifies subfeatures



Activation of Neurons: Feature Identification



Weights and Bias



$$784 \times 16 + 16 \times 16 + 16 \times 10$$

weights

$$16 + 16 + 10$$

biases

13,002

Training the network

- Initialise weights and bias randomly
- Different handwritten digits along with labels
- 13000 weights and bias are adjusted to improve performance
- After training network can classify data
- Compute cost function and minimise it

Advantages of Artificial Neural Networks

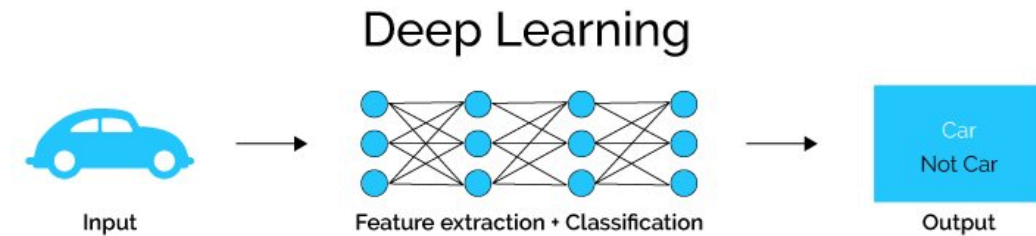
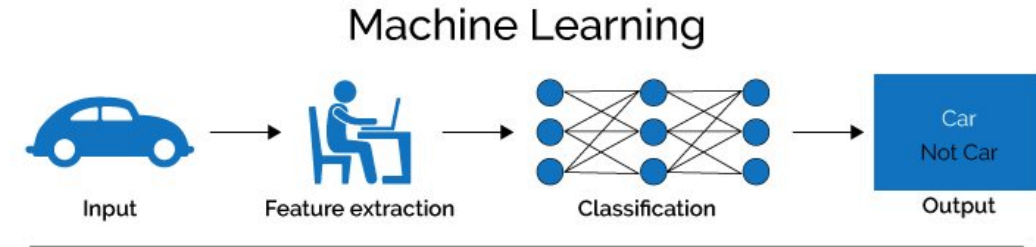
- 1. Pattern Recognition:** ANNs can recognize intricate patterns in complex data, making them effective in tasks like image recognition and speech processing.
- 2. Parallel Processing:** ANNs process information in parallel, allowing for efficient computation, especially on hardware like GPUs.
- 3. Adaptability:** ANNs can learn from data and adjust their parameters to improve performance over time, a process known as training.
- 4. Generalization:** A well-trained ANN can generalize its learning to new, unseen data, enhancing its predictive capabilities.

Disadvantages of Artificial Neural Networks

- 1. Large Data and Computation Requirements:** ANNs require substantial amounts of labeled training data for effective learning. Training deep networks can demand significant computational resources, including powerful GPUs.
- 2. Overfitting:** ANNs can be prone to overfitting, where they learn the training data's noise rather than the underlying patterns. This can lead to poor generalization on new, unseen data.
- 3. Hyperparameter Tuning:** Choosing appropriate hyperparameters (such as the number of layers, neurons, and learning rates) can be challenging and may require experimentation.
- 4. Slow Convergence:** Training ANNs can be time-consuming, especially for complex architectures. Convergence to an optimal solution may require numerous iterations.
- 5. Limited Data Efficiency:** ANNs may not perform well when the available training data is scarce or unbalanced.
- 6. Categorical Data Handling:** ANNs require numerical data, making it necessary to preprocess categorical data before feeding it to the network.
- 7. Dependency on Initial Weights:** The network's performance can depend on the initial weights and biases, which need careful initialisation strategies.

Deep Learning

- Uses multiple layers to progressively extract higher-level features from the raw input.
- Each level learns to transform its input data into a slightly more abstract and composite representation.
- In an image recognition application, the raw input may be a matrix of pixels; the first representational layer may abstract the pixels and encode edges; the second layer may compose and encode arrangements of edges; the third layer may encode a nose and eyes; and the fourth layer may recognize that the image contains a face.
- Can learn which features to optimally place in which level on its own.



Machine learning vs. deep learning



Machine learning

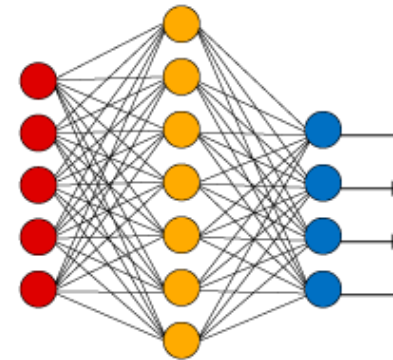
Uses algorithms and learns on its own but may need human intervention to correct errors



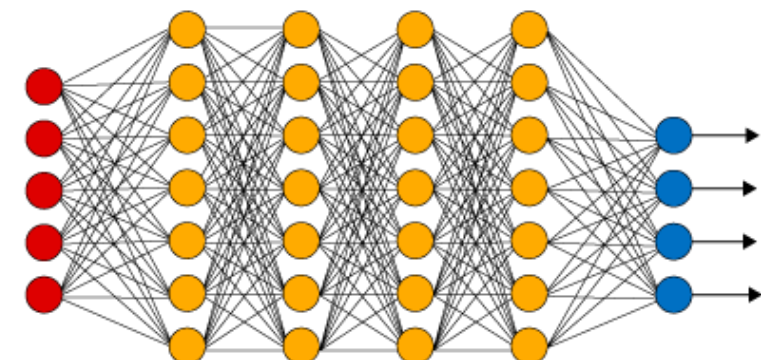
Deep learning

Uses advanced computing, its own neural network, to adapt with little to no human intervention

Simple Neural Network



Deep Learning Neural Network



● Input Layer ● Hidden Layer ● Output Layer



learned
low-level
features

learned
mid-level
features

learned
high-level
features

learned
classifier

cat

Deep, "end-to-end" learning:

- DL applies a multi-layer process for learning rich hierarchical features
Input image pixels → Edges → Textures → Parts → Objects



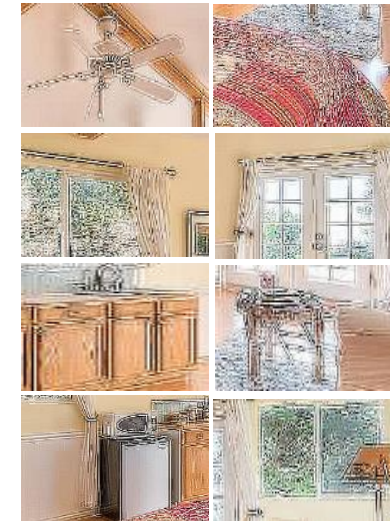
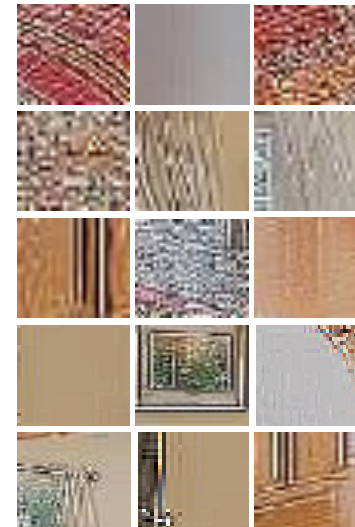
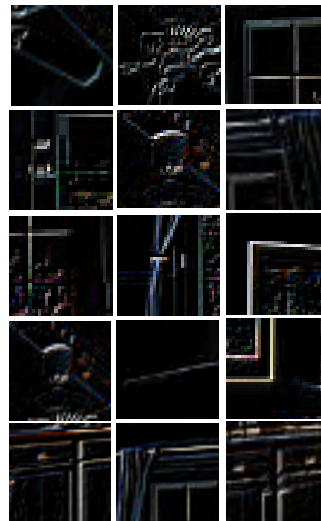
Low-Level
Features

Mid-Level
Features

High-Level
Features

Trainable
Classifier

Output



Why is DL Useful?

- ▶ DL provides a flexible, learnable framework for representing visual, text, linguistic information
 - ▶ Can learn in supervised and unsupervised manner
- ▶ DL represents an effective end-to-end learning system
- ▶ Requires large amounts of training data
- ▶ Since about 2010, DL has outperformed other ML techniques
 - ▶ First in vision and speech, then NLP, and other applications

Types of Deep Learning Algorithms

Convolutional Neural Networks (CNNs)

Recurrent Neural Networks (RNNs)

Long Short Term Memory Networks (LSTMs)

Generative Adversarial Networks (GANs)

Radial Basis Function Networks (RBFNs)

Multilayer Perceptrons (MLPs)

Self Organizing Maps (SOMs)

Restricted Boltzmann Machines (RBMs)

Deep Belief Networks (DBNs)

Curse of dimensionality

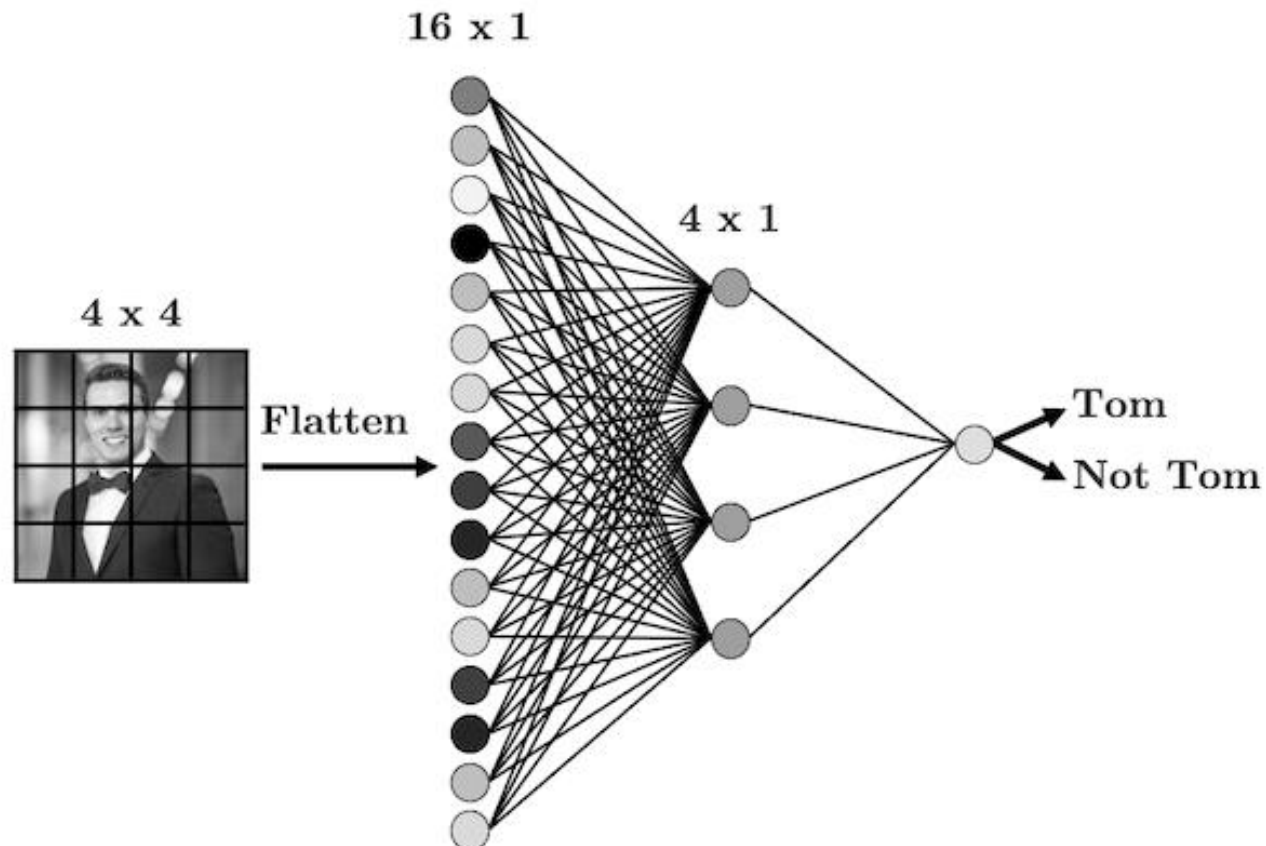
- The core idea in deep learning is that we assume that the data was generated by the composition factors or features, potentially at multiple levels in a hierarchy.
- This assumption allows an exponential gain in the relationship between the number of examples and the number of regions that can be distinguished.
- The exponential advantages conferred by the use of deep, distributed representations counter the exponential challenges posed by the curse of dimensionality.

Convolutional Neural Networks (CNNs)

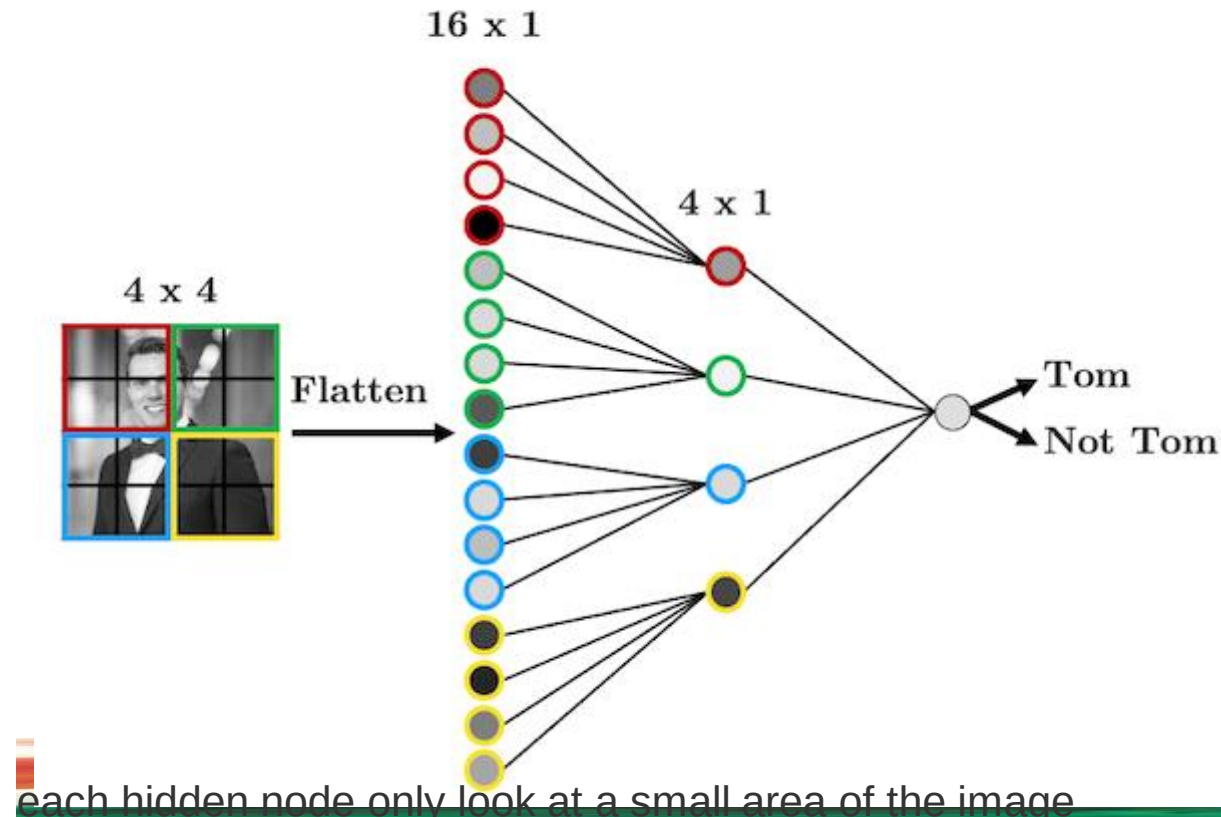
Up until now we've been dealing with "fully connected neural networks" meaning that every neuron in a given layer is connected to every neuron in the next layer. This has two key implications:

- 1.It results in a LOT of parameters.
- 2.The order of our features doesn't matter.

$$\text{Total Parameters} = (16 \times 4 + 4) + (4 \times 1 + 1) = 73$$

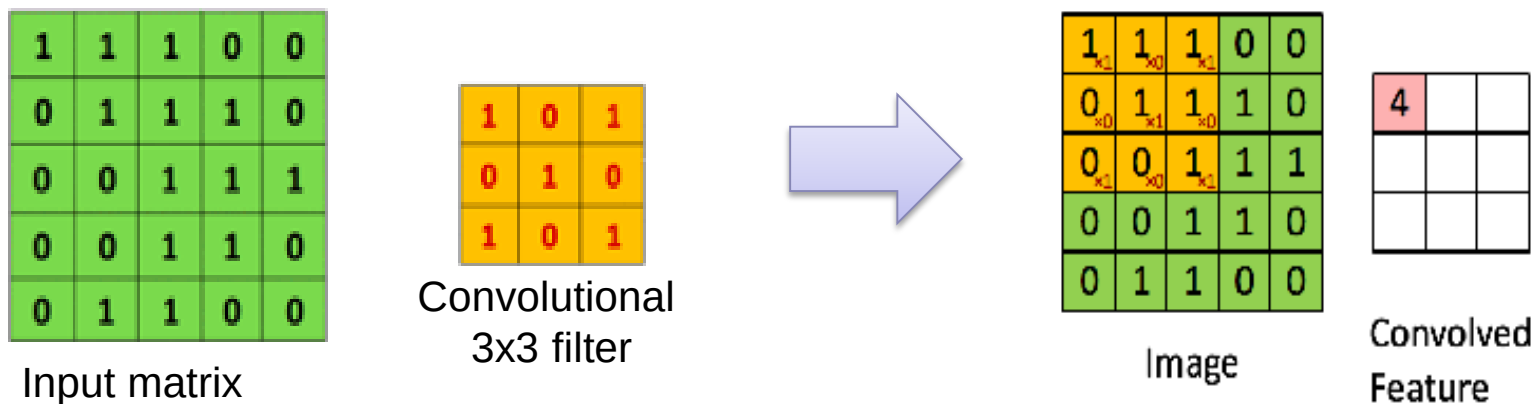


$$\text{Total Parameters} = (4 \times 4 + 4) + (4 \times 1 + 1) = 25$$



Convolutional Neural Networks (CNNs)

- *Convolutional neural networks* (CNNs) were primarily designed for image data
- CNNs use a **convolutional operator** for extracting data features
 - Allows **parameter sharing**
 - Efficient to train
 - Have **less parameters** than NNs with fully-connected layers
- CNNs are **robust to spatial translations** of objects in images
- A convolutional filter slides (i.e., convolves) across the image



Convolutional Neural Networks (CNNs)

- When the convolutional filters are scanned over the image, they capture useful features
 - E.g., edge detection by convolutions

Filter



$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$



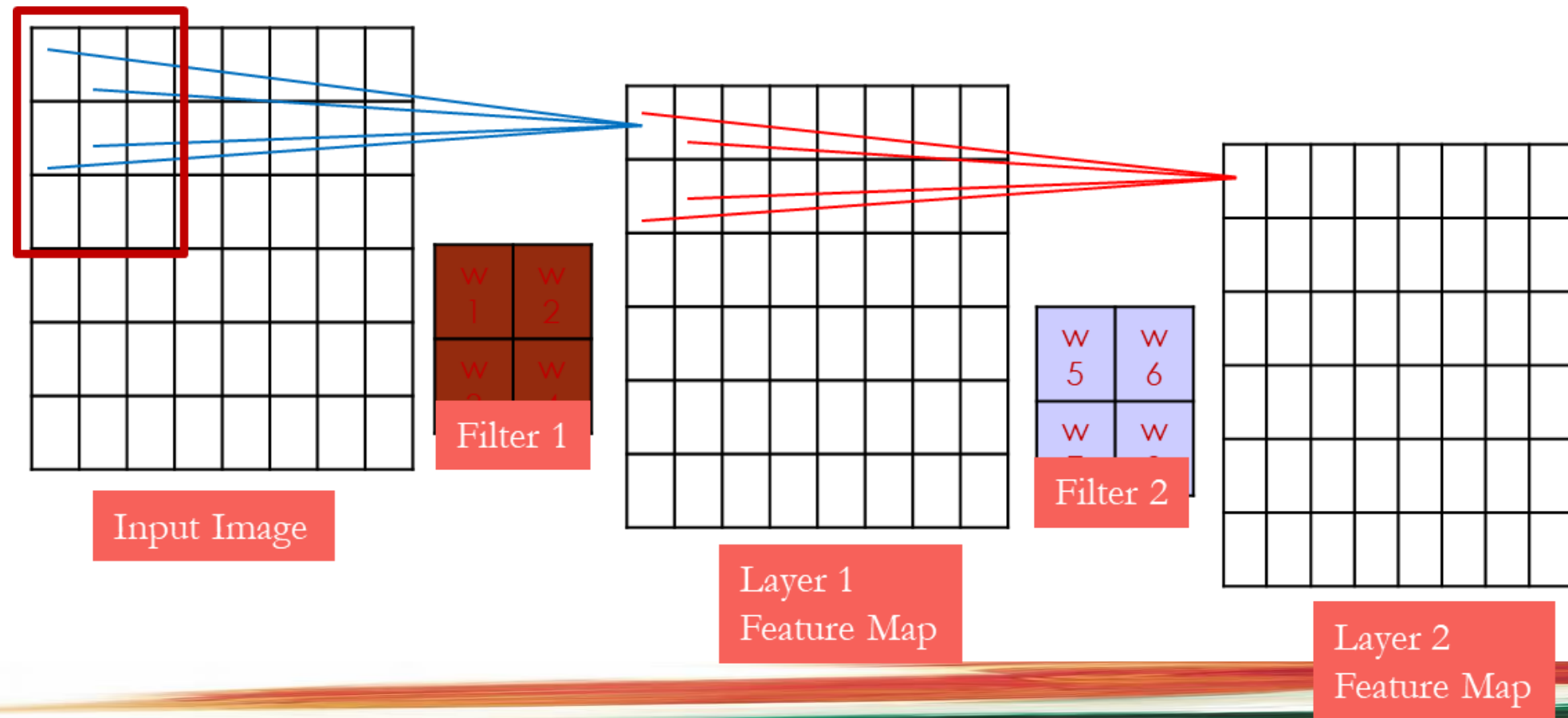
Input Image

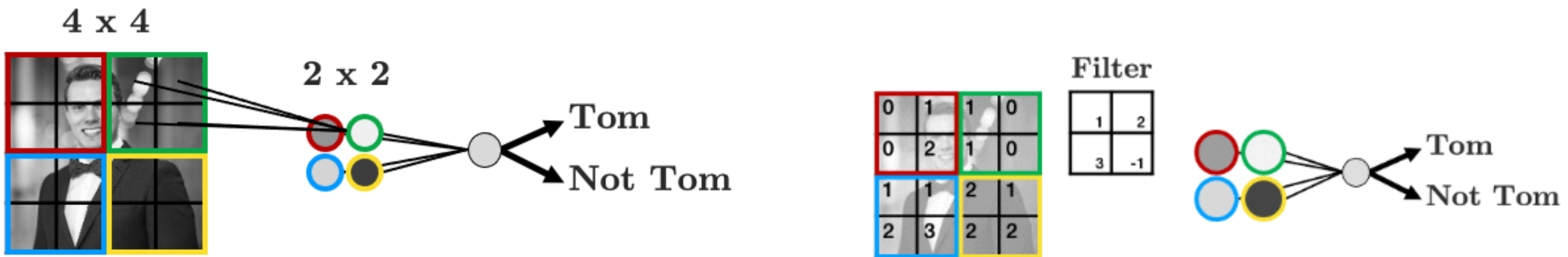


Convolved Image

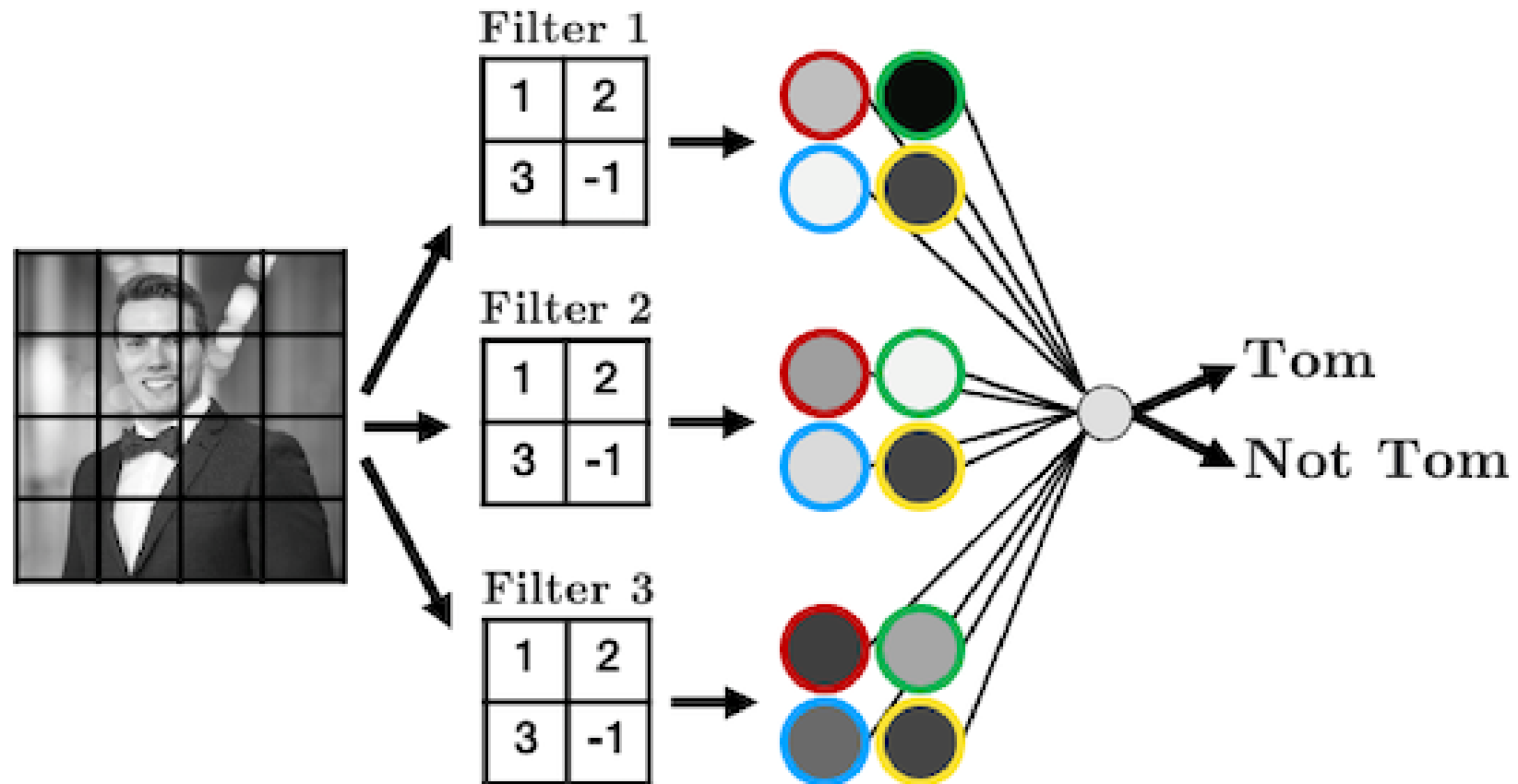
Convolutional Neural Networks (CNNs)

- In CNNs, hidden units in a layer are only connected to a small region of the layer before it (called local **receptive field**)
 - The depth of each **feature map** corresponds to the number of convolutional filters used at each layer



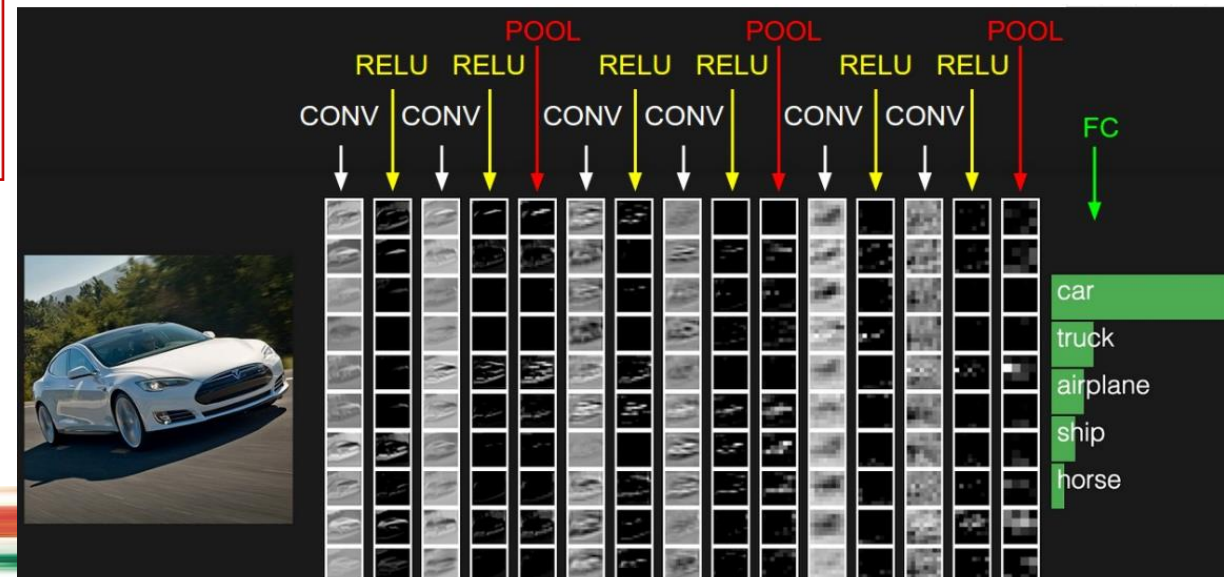
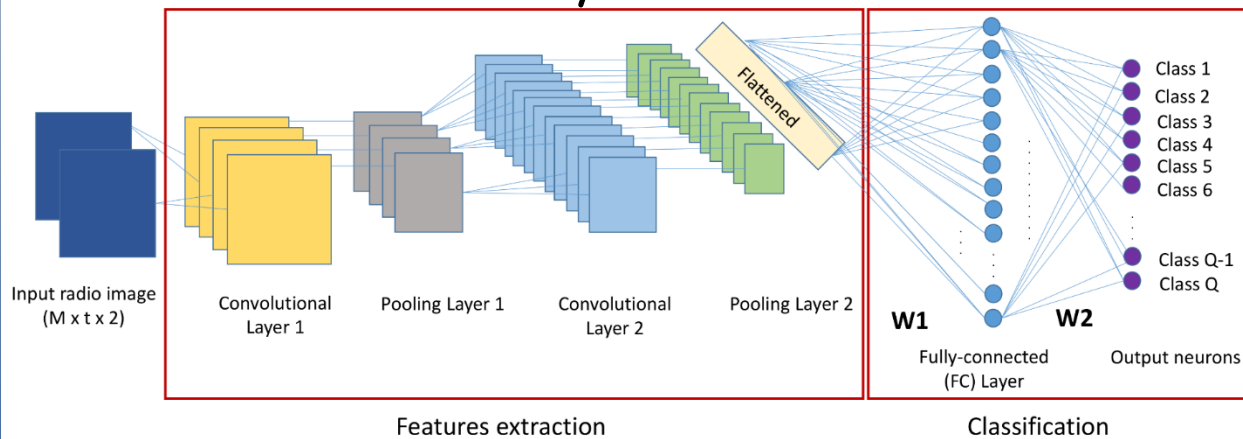


NB: only “green” connections shown for simplicity.



Architecture of CNNs

- A convolutional neural network consists of an input layer, hidden layers and an output layer.
- In a convolutional neural network, the hidden layers include layers that perform convolutions.
- Typically this includes a layer that does multiplication or other dot product, and its activation function is commonly ReLU.
- This is followed by other convolution layers such as **pooling layers**, fully connected layers and normalization layers.



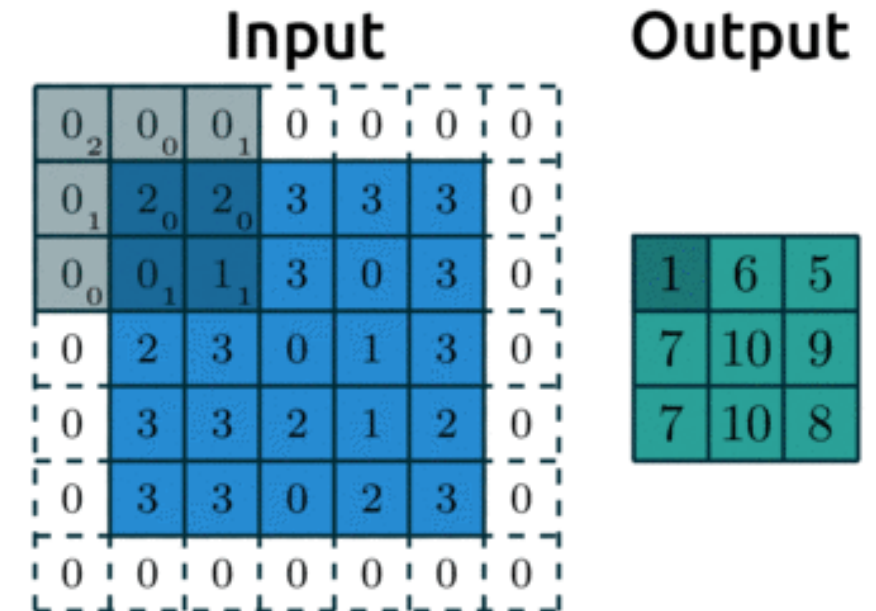
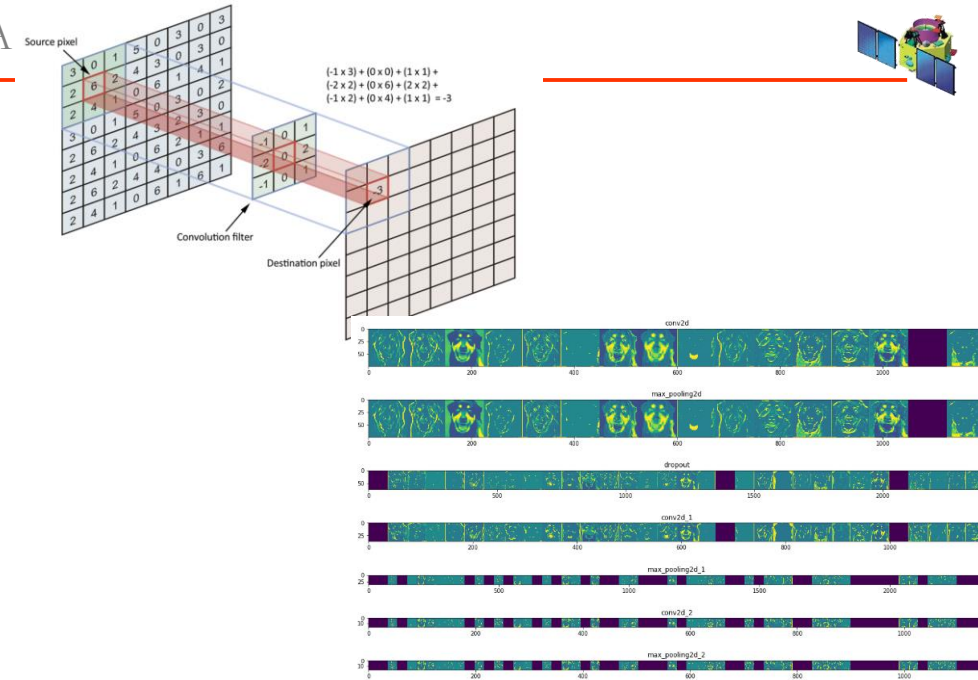
Convolutions layers and Filters

Convolution really just means “to pass over the data”.

- Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image.
- A convolutional layer within a neural network should have the following attributes:
 - Convolutional filters/kernels defined by a width and height (hyper-parameters).
 - The number of input channels and output channels (hyper-parameter).
 - The depth of the convolution kernel/filter (the input channels) must equal the number channels (depth) of the input feature map.
 - The hyperparameters of the convolution operation, like padding size and stride.
- Convolutional layers convolve the input and pass its result to the next layer.

•padding: “pads” the outside of the input 0’s to allow the kernel to reach the boundary pixels

•strides: controls how far the kernel “steps” over pixels.



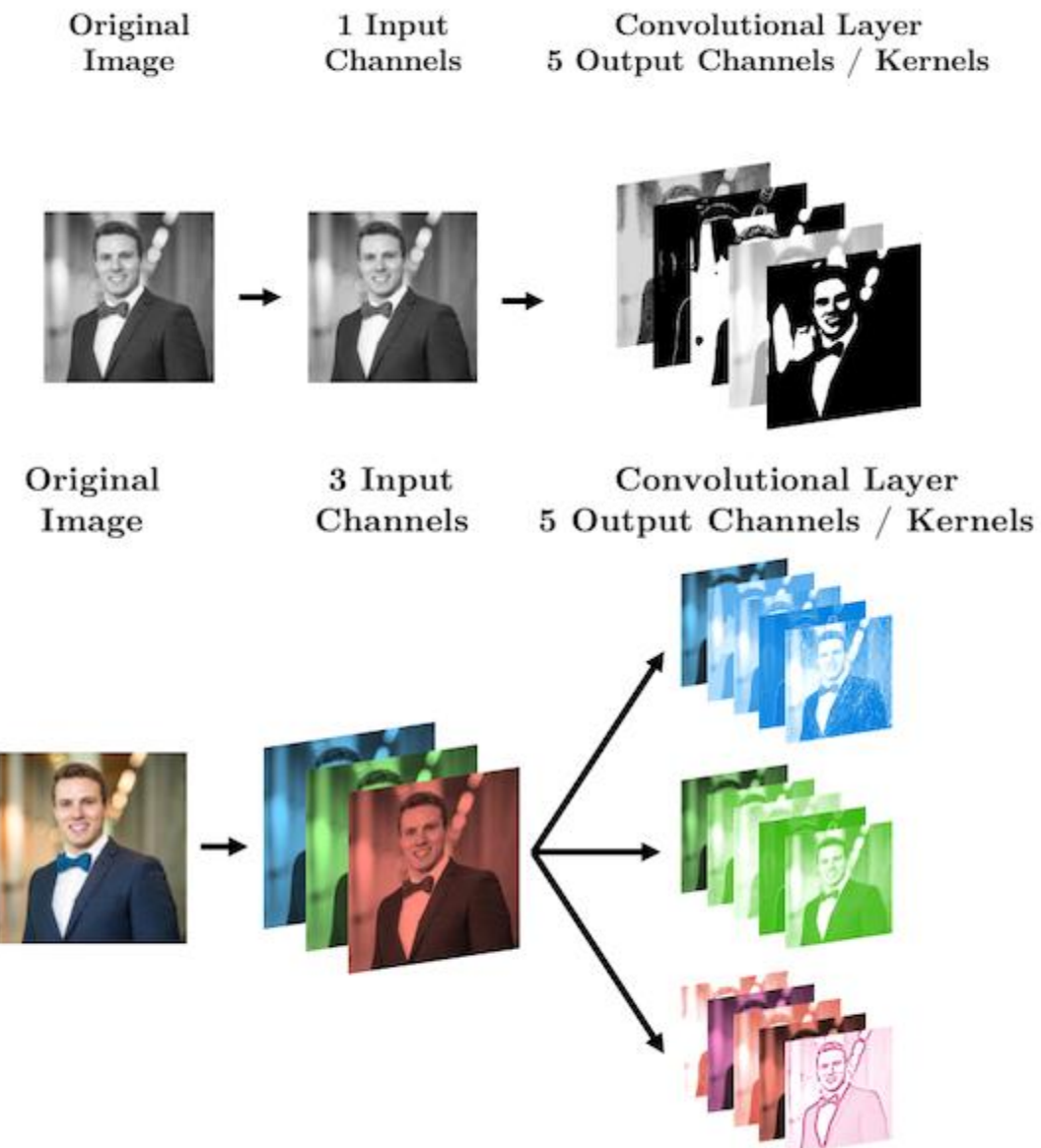
What are “features” in CNN? Features are called “channels” in CNN-lingo

→ they are like the colour channels in an image:

- A grayscale image has 1 feature/channel
- A coloured image has 3 features/channel

What’s important with CNNs is that the **size of our input data does not impact how many parameters we have in our convolutional layers.**, all that matters is:

1. How many features/ Number of channels
2. How many filters/ Number of filters
3. How big the filters are/ Kernel Size



Pooling

Pooling is how to reduce the number of parameters . It aggregates the data, usually using the maximum or average of a window of pixels.

- ▶ **Max pooling**: reports the maximum output within a rectangular neighborhood
- ▶ **Average pooling**: reports the average output of a rectangular neighborhood
- ▶ Pooling layers reduce the spatial size of the feature maps
 - ▶ Reduce the number of parameters, prevent overfitting

Input

1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

Output

7	9
8	

1	3	5	3
4	2	3	1
3	1	1	3
0	1	0	4

MaxPool with a 2×2 filter with stride of 2

4	5
3	4

Output Matrix

Input Matrix

max pooling

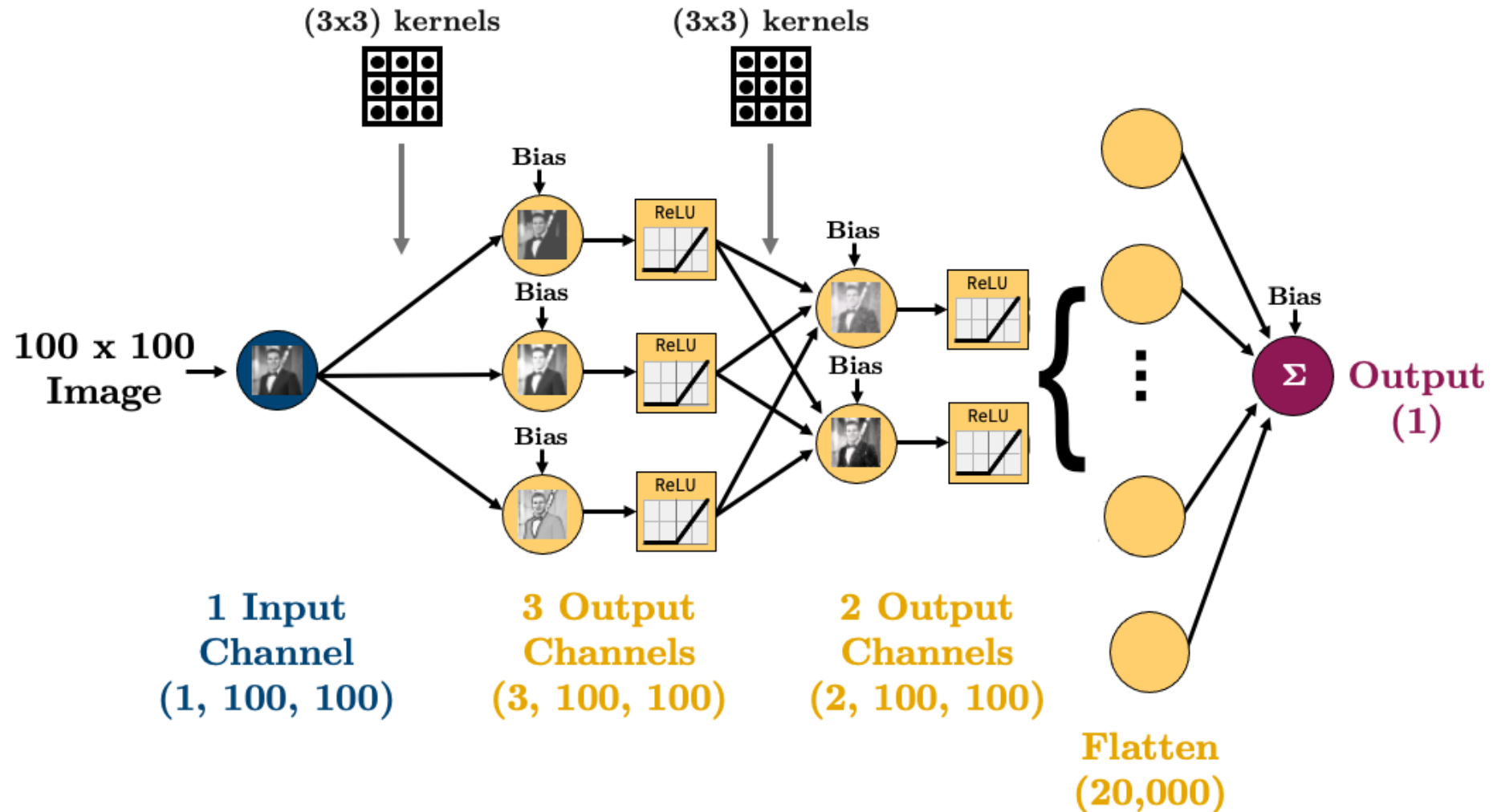
20	30
112	37

average pooling

13	8
79	20

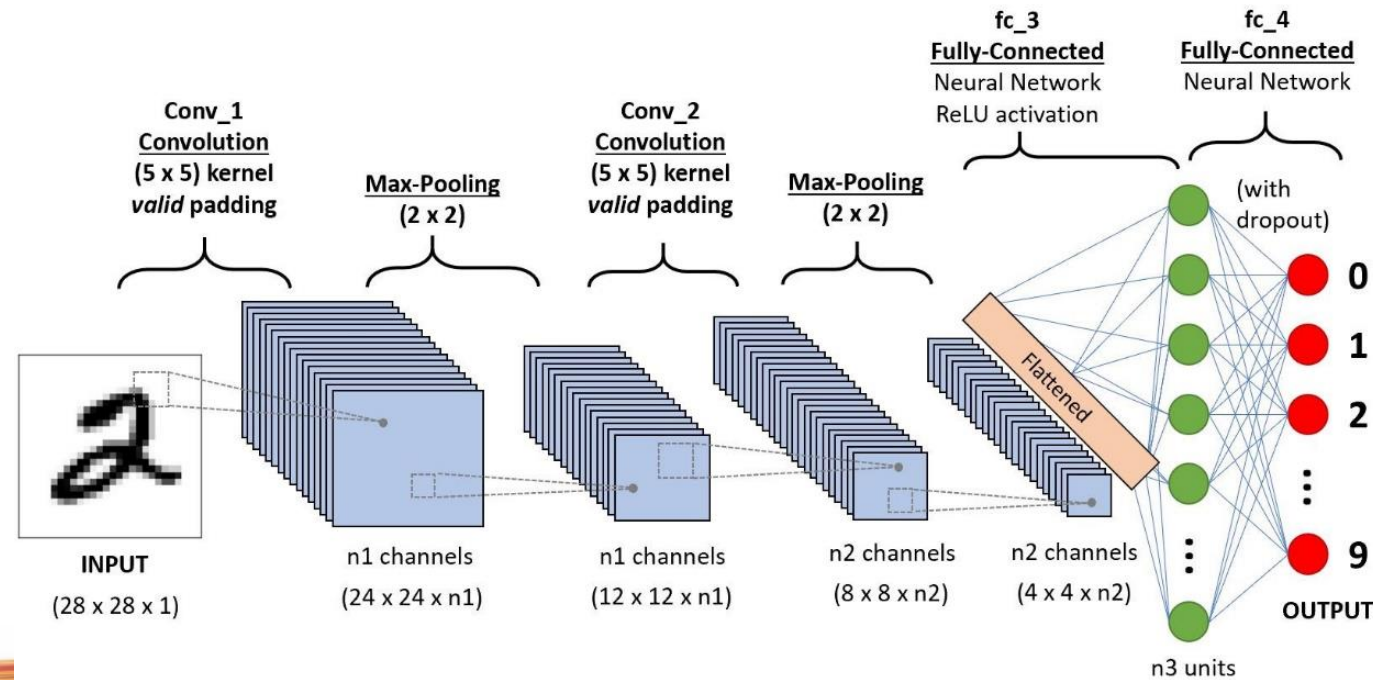
Flattening

But we're going to eventually want to do some regression or classification. That means that by the end of our network, we are going to need to flatten our images:



Classification – Fully Connected Layer (FC Layer)

- Fully connected layers connect every neuron in one layer to every neuron in another layer. It is the same as a traditional multi-layer perceptron neural network (MLP). The flattened matrix goes through a fully connected layer to classify the images.
- The flattened output is fed to a feed-forward neural network and backpropagation applied to every iteration of training. Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them.
- In place of fully connected layers, we can also use a conventional classifier like SVM. But we generally end up adding FC layers to make the model end-to-end trainable.



Two sorts of parameters in machine learning:

- **Machine learnable parameters** – The parameters that the algorithms learn/estimate on their own during training for a particular dataset
- **Hyper-parameters** are variables that machine learning engineers or data scientists provide precise values to regulate how algorithms learn and modify the model's performance.

Learning Rate:

- Used to govern the pace at which an algorithm updates or learns the values of a parameter estimate.
- **Regulates the weights of neural network concerning the loss gradient.** It indicates how often the neural network refreshes the notions it has learned.
- A desirable learning rate is low enough for the network to converge on something useful while yet being high enough to train in a reasonable length of time.
- Smaller learning rates necessitate more training epochs because of the fewer changes. On the other hand, larger learning rates result in faster changes.
- Larger learning rates frequently result in a suboptimal final set of weights.

Epoch : Term used in machine learning to indicate the number of passes of the entire training dataset the machine learning algorithm has completed.

Loss Functions :

- A loss function is a function that compares the target and predicted output values; measures how well the neural network models the training data. When training, we aim to minimize this loss between the predicted and target outputs.
- The hyperparameters are adjusted to minimize the average loss
- A loss function/error function is for a single training example/input.
- A cost function, on the other hand, is the average loss over the entire training dataset.

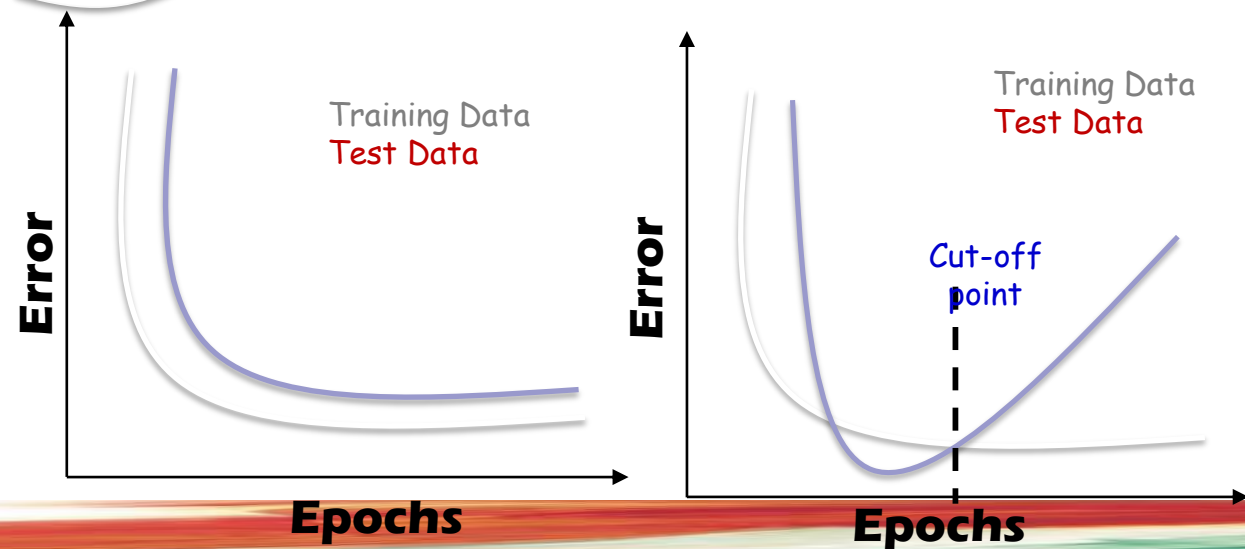
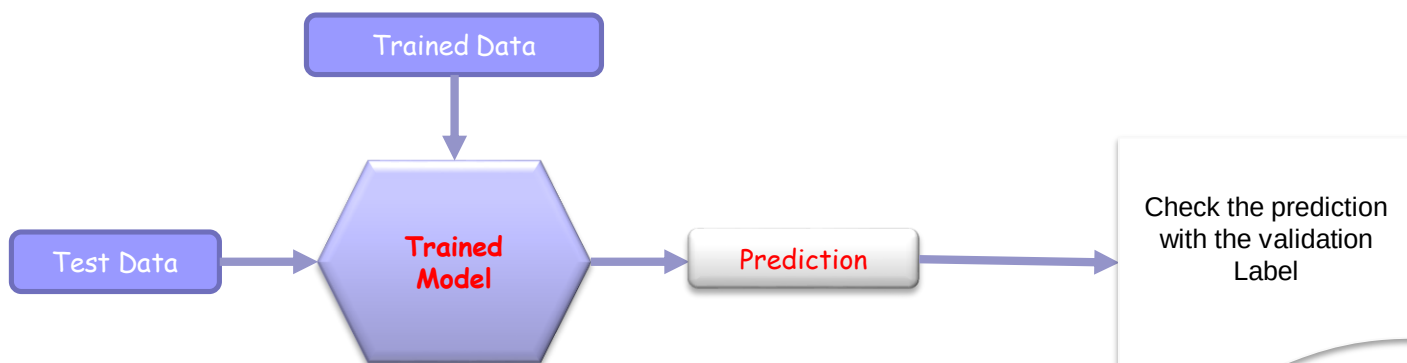
Optimiser

- Optimizer algorithms are optimization method that helps improve a deep learning model's performance.
- An optimizer is a function or an algorithm that adjusts the attributes of the neural network, such as weights and learning rates
- Thus, it helps in reducing the overall loss and improving accuracy.
- To achieve good performance its is necessary to check which optimizer has better performance

Model Evaluation

First, the data is splitted into a **training set** and a **test set**. If the model would perform well on both, with similar behavior, the model is successful.

If we overfit on the training data then that means we would perform poorly on new test data then one should look for the point to cut off training time!

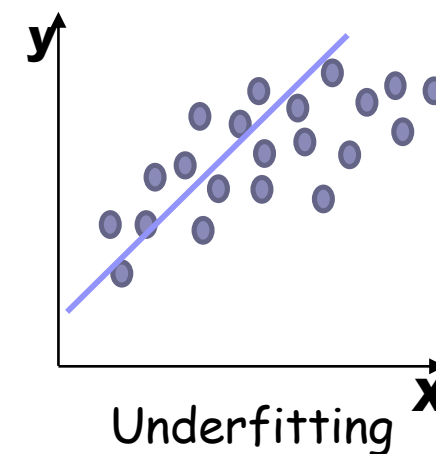
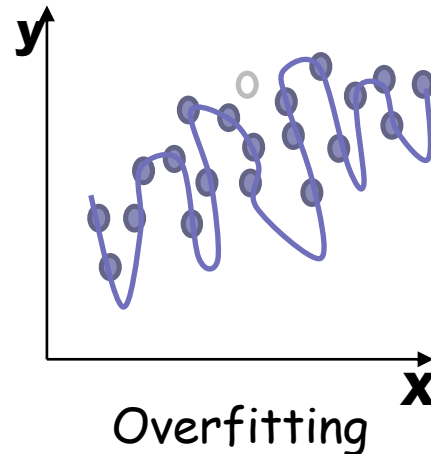
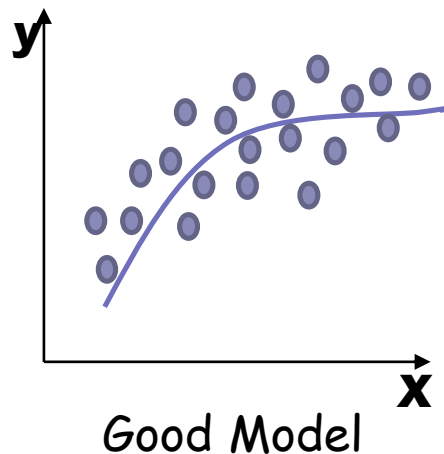


Model Evaluation

Overfitting: Less error in training data, More error in testing data; Less Bias and More Variance

Underfitting: High error in training data, More error in testing data; More Bias and More Variance

Best Fit: Less Bias and Less Variance



Overfitting

The model fits too much to the noise from the data.

This often results in low error on training sets but high error on test/validation sets.

Underfitting

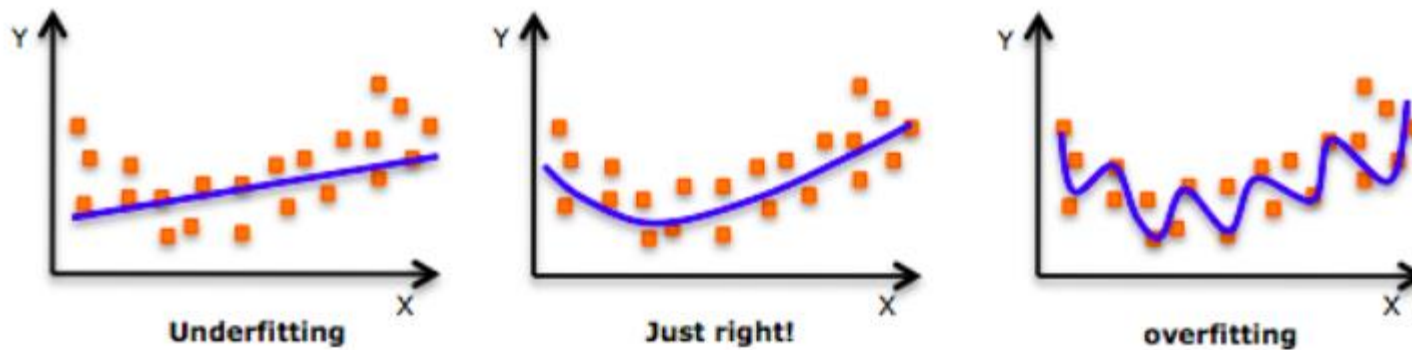
Model does not capture the underlying trend of the data and does not fit the data well enough.

Low variance but high bias.

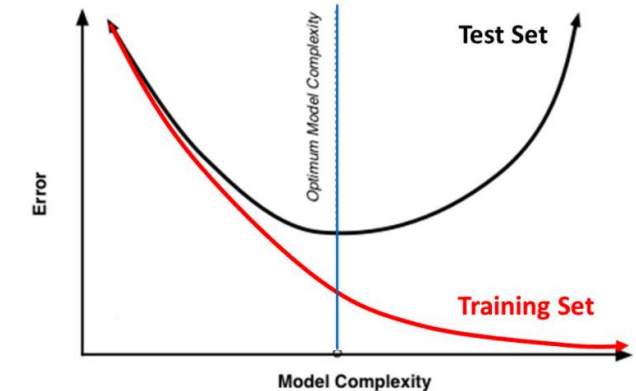
Underfitting is often a result of an excessively simple model

Regularization

- One of the most common problems data science professionals face is to avoid **overfitting**.
- Regularization is a technique used in machine learning and deep learning to prevent overfitting and improve the generalization performance of a model.
- It involves adding a penalty term to the loss function during training.
- This penalty discourages the model from becoming too complex or having large parameter values, which helps in controlling the model's ability to fit noise in the training data. Regularization methods include L1 and L2 regularization, dropout, early stopping, and more. By applying regularization, models become more robust and better at making accurate predictions on unseen data.



Training Vs. Test Set Error



Data Augmentation

The simplest way to reduce overfitting is to increase the size of the training data.

It can be considered as a mandatory trick in order to improve our predictions.

shift



shift



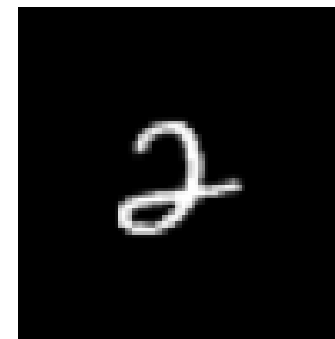
shear



shift & scale

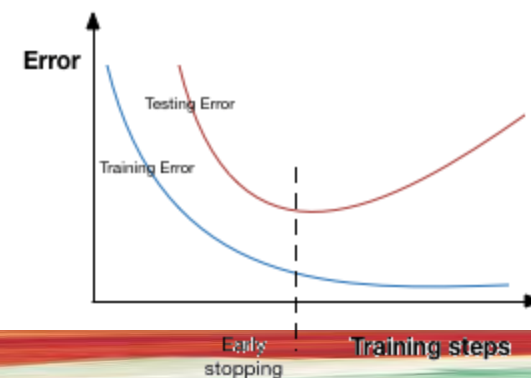


rotate & scale



Early stopping

Early stopping is a kind of cross-validation strategy where we keep one part of the training set as the validation set. When we see that the performance on the validation set is getting worse, we immediately stop the training on the model. This is known as early stopping.



Model Evaluation - Classification type model

- In Classification model, **ideally** either the model is correct in its prediction or it is incorrect.
- In real world, we will have a count of correct matches and incorrect matches. In order to evaluate that there are 4 key classification metrics:-

Confusion Matrix

		Predicted Condition		
		Prediction Positive	Prediction Negative	
True Condition	Condition Positive	True Positive (TP)	False Negative (FN) (type II error)	$\text{Recall} = \frac{\sum TP}{\sum \text{condition positive}}$
	Condition Negative	False Positive (FP) (type I error)	True Negative (TN)	
		$\text{Precision} = \frac{\sum TP}{\sum \text{prediction positive}}$		$\text{F1 score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$

Accuracy number of correct predictions made by the model divided by the total number of predictions.

$$\text{Accuracy} = \frac{\text{Correct Prediction}}{\text{Total prediction}}$$

Precision Precision is to identify only the relevant data points in the classification data (User's accuracy or correctness).

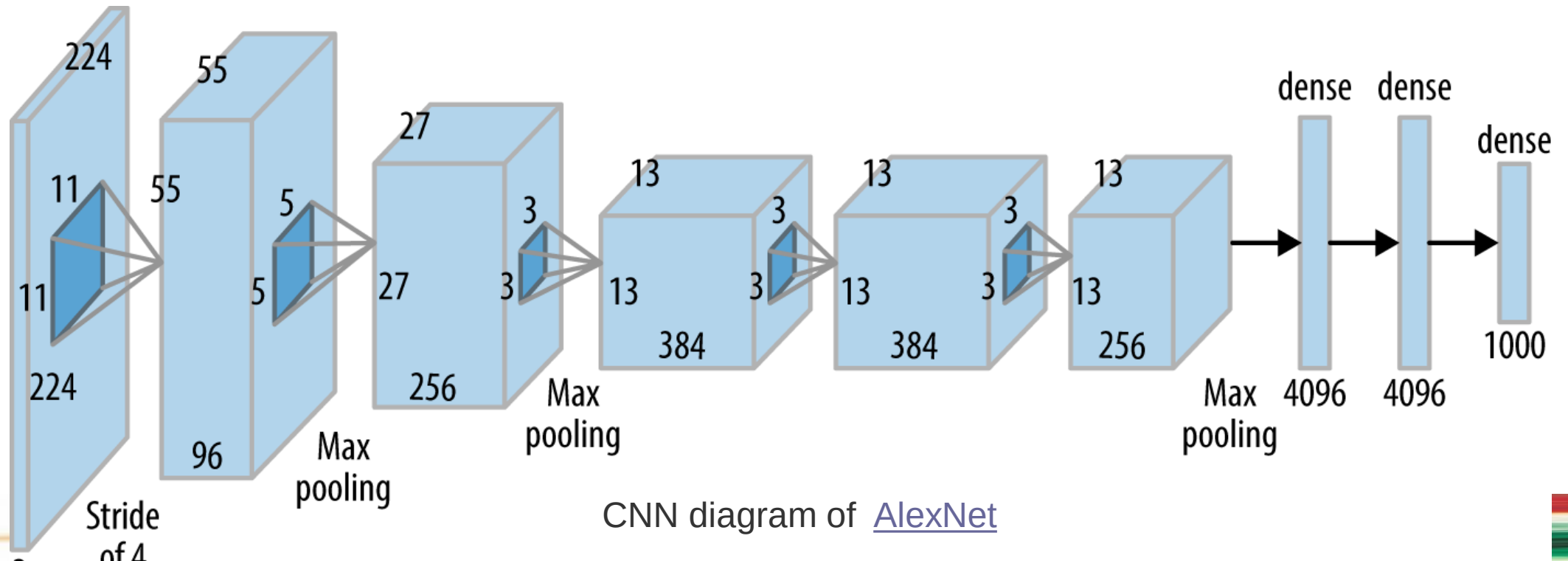
Recall Ability of a model to find all the relevant cases within a dataset (Producer's accuracy or completeness).

F1-Score Harmonic mean of precision and recall taking both metrics into account.

$$\text{F1 score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

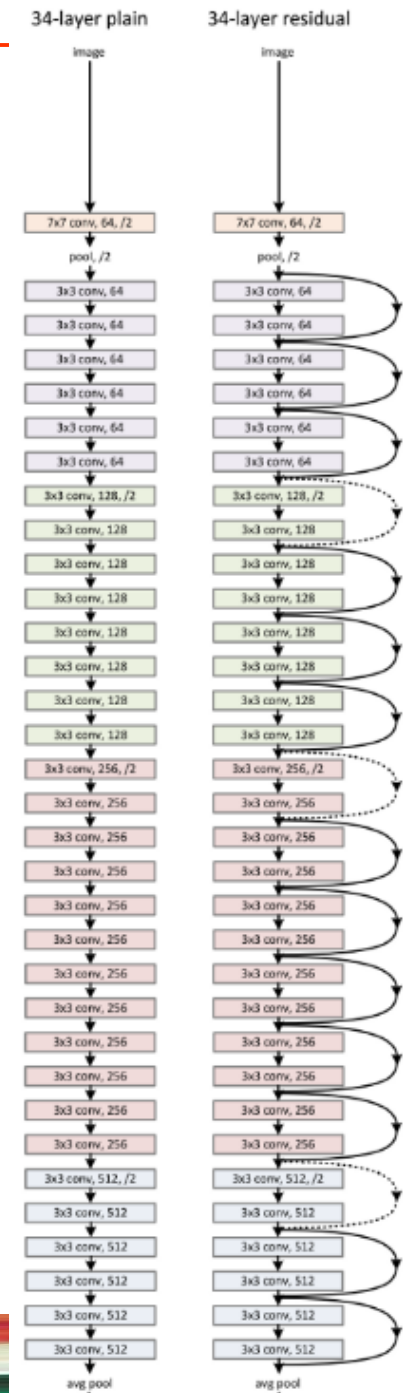
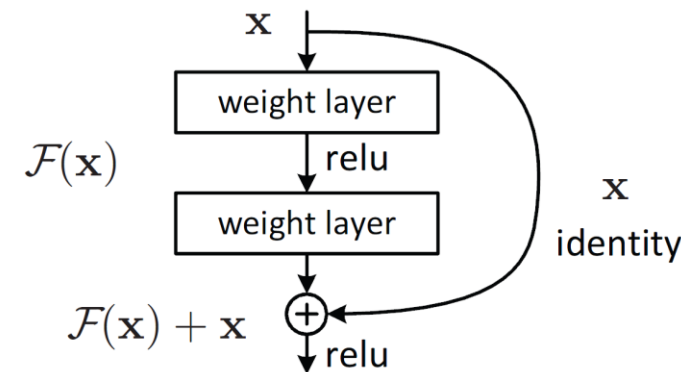
There are various architectures of **CNNs** available which have been key in building algorithms which power and shall power AI as a whole in the foreseeable future. Some of them have been listed below:

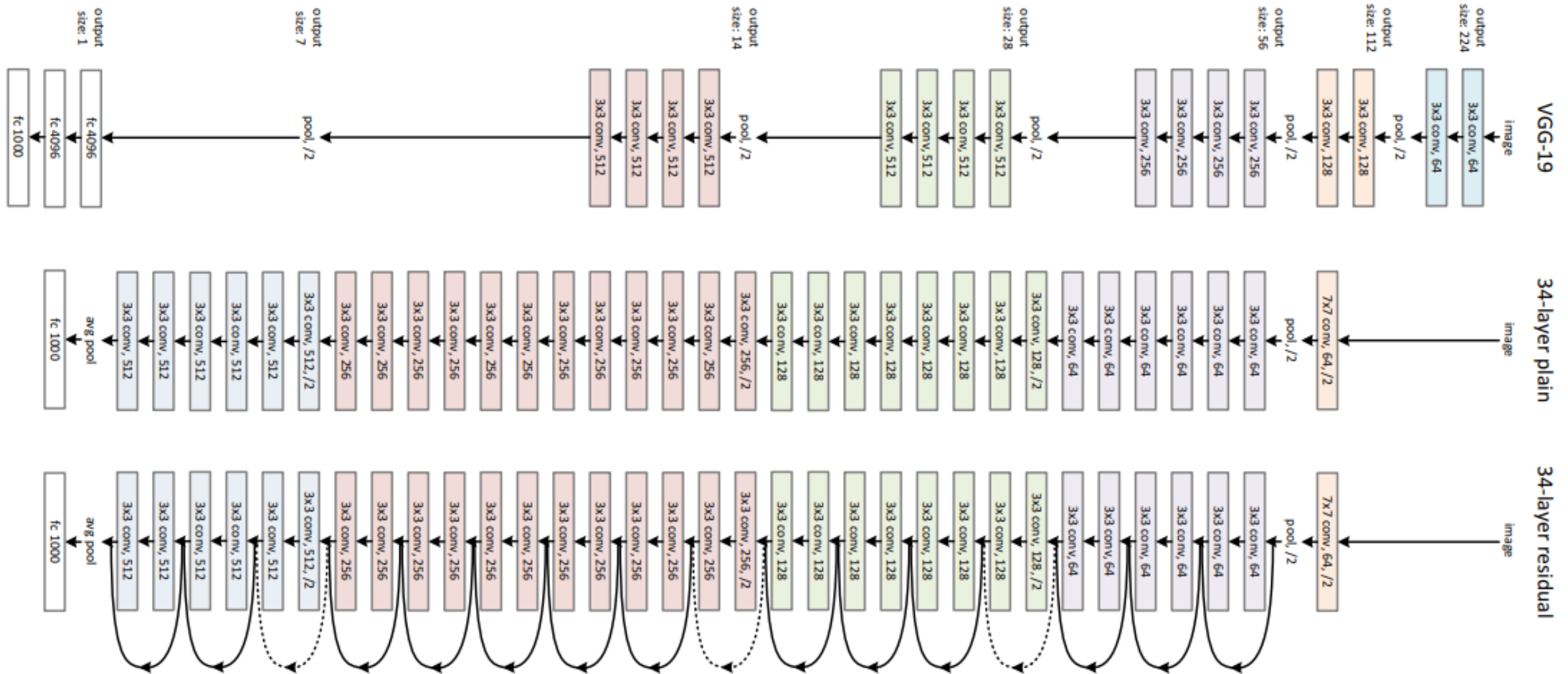
- a) LeNet
- b) AlexNet
- c) VGGNet
- d) GoogLeNet
- e) ResNet
- f) ZFNet



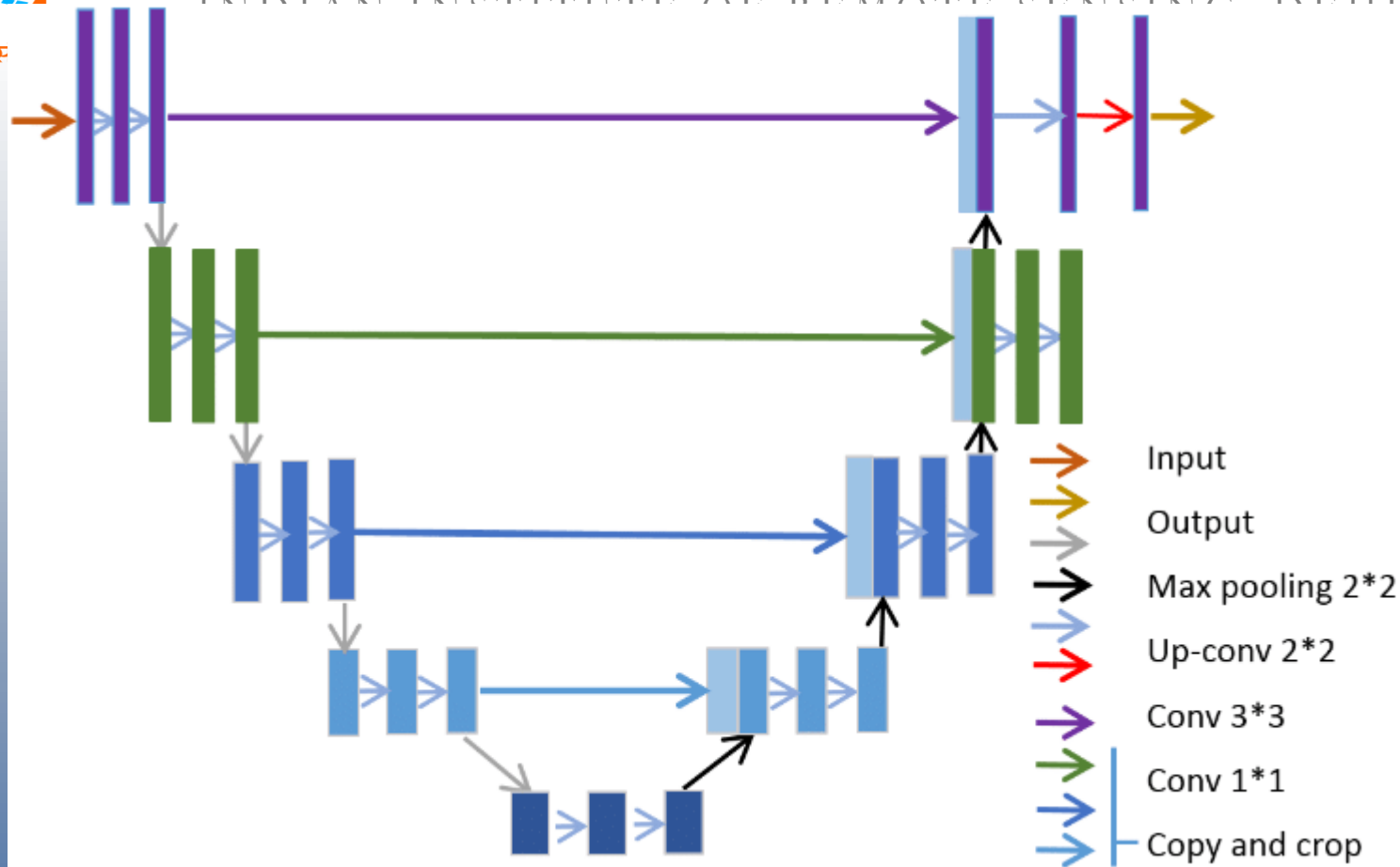
Residual CNNs

- **Residual networks** (ResNets)
 - Introduce “identity” **skip connections**
 - Layer inputs are propagated and added to the layer output
 - Mitigate the problem of vanishing gradients during training
 - Allow training very deep NN (with over 1,000 layers)
 - Several ResNet variants exist: 18, 34, 50, 101, 152, and 200 layers
 - Are used as base models of other state-of-the-art NNs
 - Other similar models: ResNeXT, DenseNet



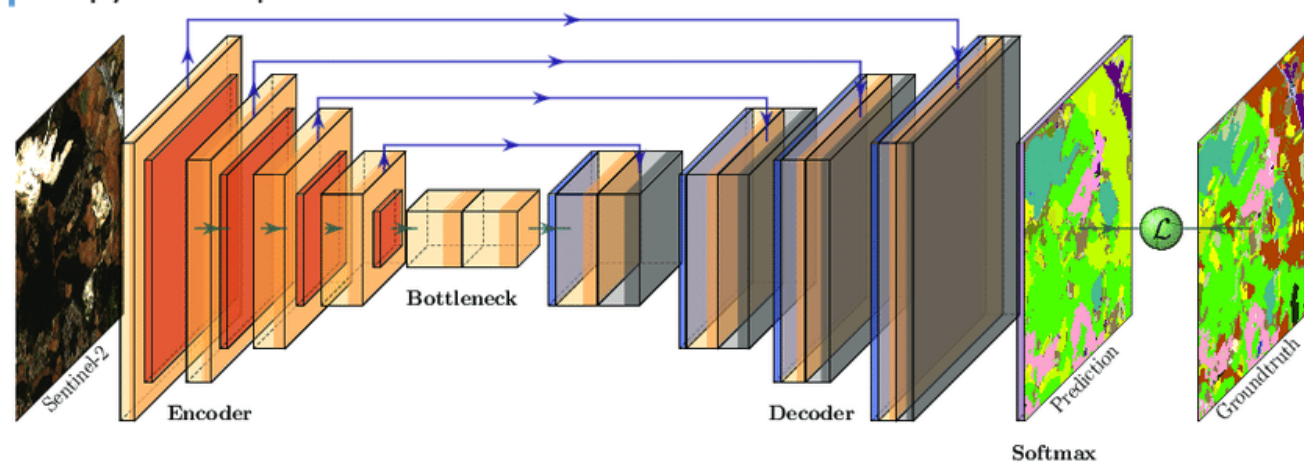


- Made up of residual blocks with skip connections
- Solves problem of vanishing gradients



U-Net – 34 Architecture

- Effective for tasks where output should be of same size as input image.
- Useful for image segmentation



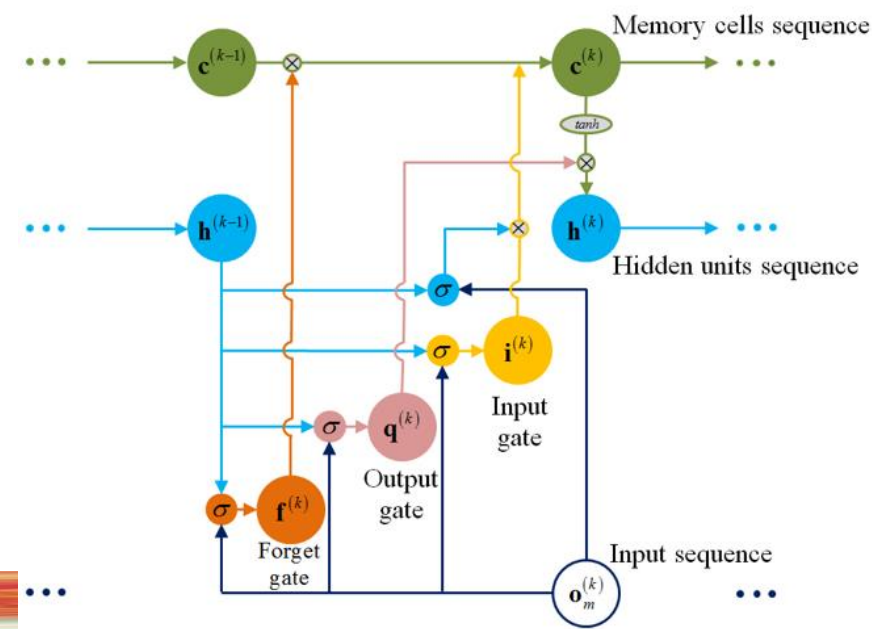
- Involves single neural network trained end to end that takes image as input and predicts bounding boxes and class labels
- Operates on 45 frames per second and upto 155 frames per second

Recurrent Neural Networks (RNNs)

- *Recurrent NNs* are used for modeling **sequential data** and data with varying length of inputs and outputs (Videos, text, speech, DNA sequences, human skeletal data)
- RNNs introduce recurrent connections between the neurons
 - This allows processing sequential data one element at a time by selectively passing information across a sequence
 - Memory of the previous inputs is stored in the model's internal state and affect the model predictions
 - Can capture correlations in sequential data
- RNNs use **backpropagation-through-time** for training
- RNNs are more sensitive to the vanishing gradient problem than CNNs

LSTM Networks

- *Long Short-Term Memory (LSTM)* networks are a variant of RNNs
- LSTM mitigates the vanishing/exploding gradient problem
 - Solution: a **Memory Cell**, updated at each step in the sequence
- Three gates control the flow of information to and from the Memory Cell
 - **Input Gate**: protects the current step from irrelevant inputs
 - **Output Gate**: prevents current step from passing irrelevant information to later steps
 - **Forget Gate**: limits information passed from one cell to the next
- Most modern RNN models use either LSTM units or other more advanced types of recurrent units



Deep learning frameworks

Caffe



DEEPLARNING4J



TensorFlow



Caffe2

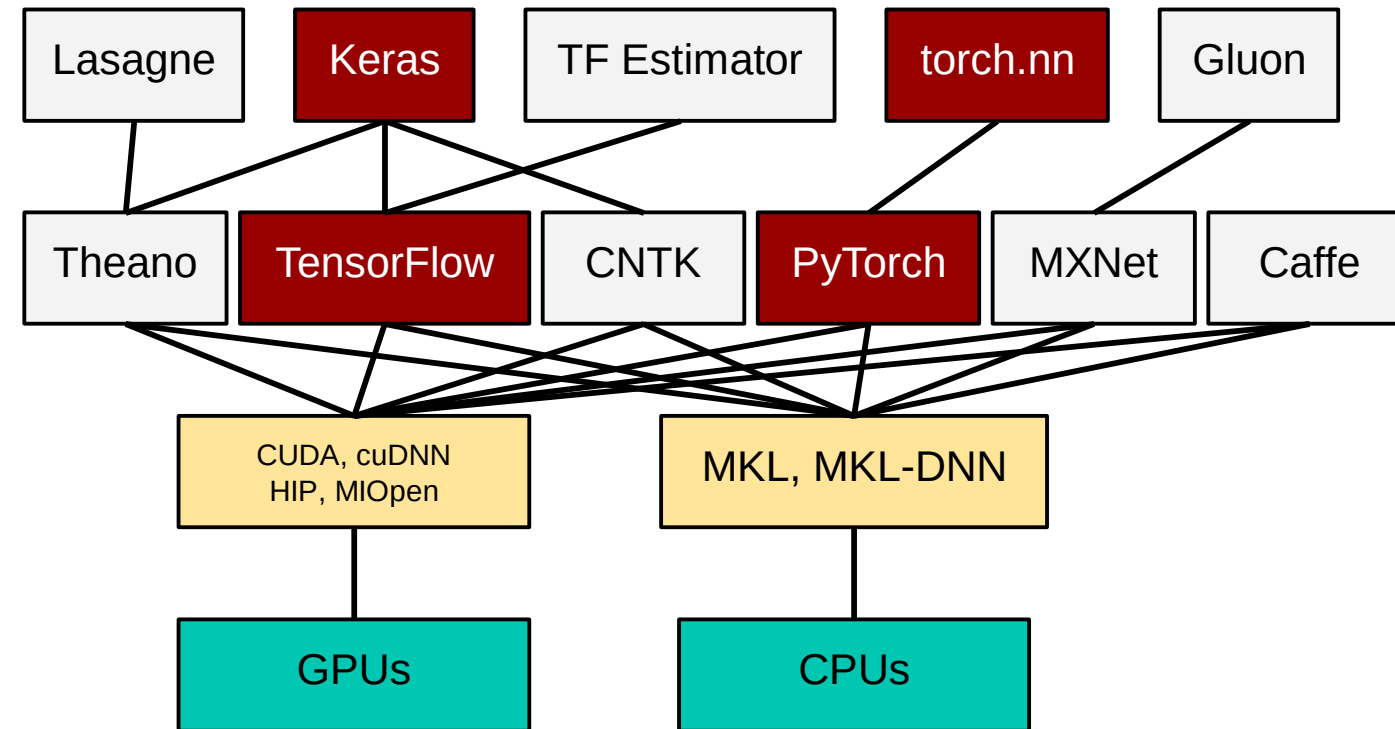


dmlc
mxnet



theano

Deep learning frameworks



⑩ Keras is a high-level neural networks API

⑩ included in TensorFlow 2 as `tf.keras`

⑩ <https://keras.io/> , <https://www.tensorflow.org/guide/keras>

⑩ PyTorch is:

⑩ a GPU-based tensor library

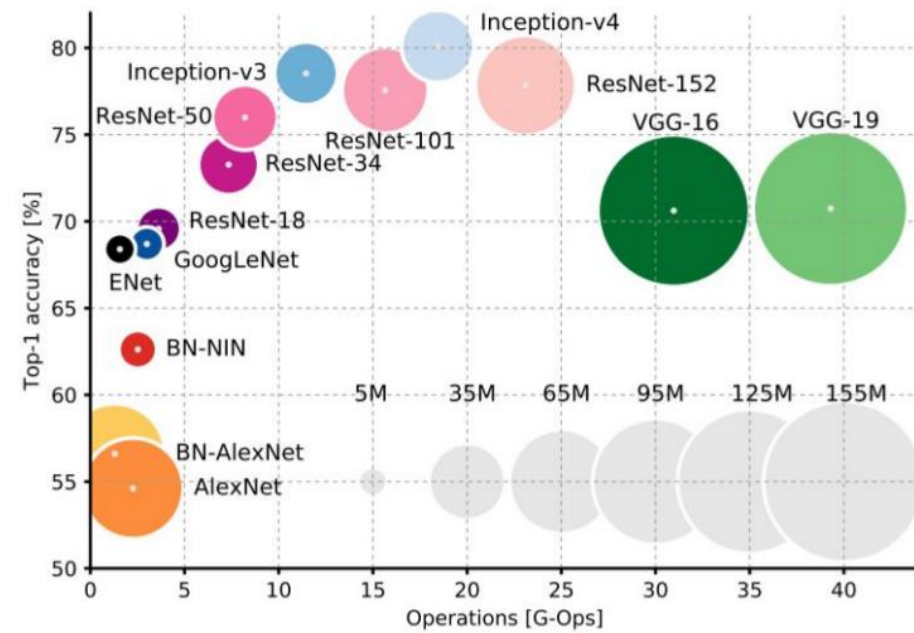
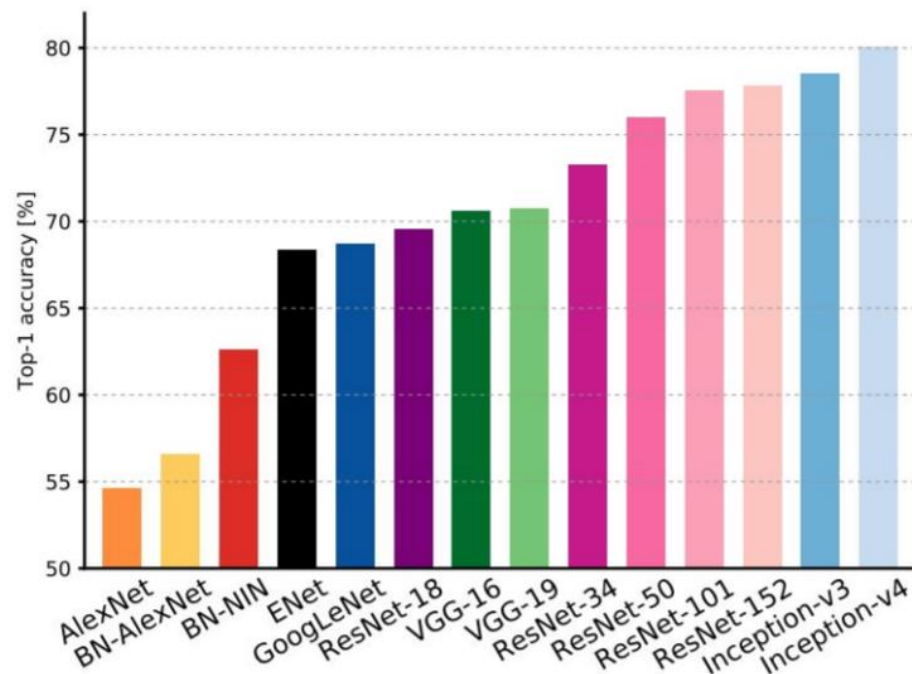
⑩ an efficient library for dynamic neural networks

⑩ <https://pytorch.org/>

Computational complexity

■ The memory bottleneck

Comparing complexity...



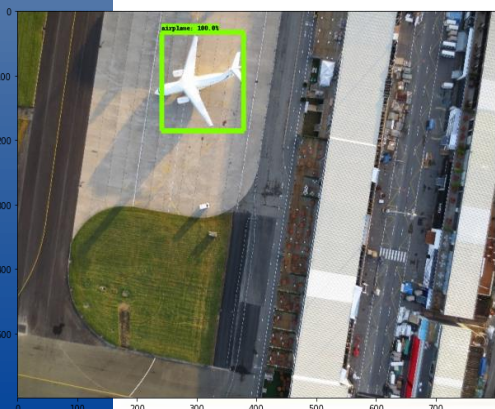
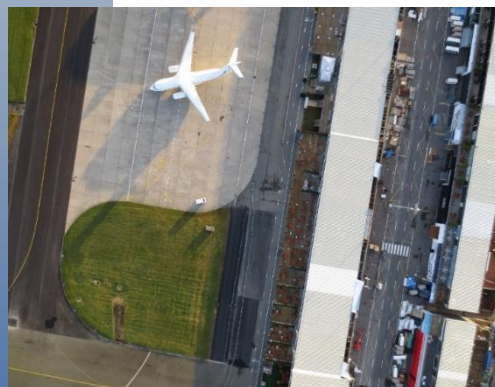
An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Application of Machine learning in Remote Sensing

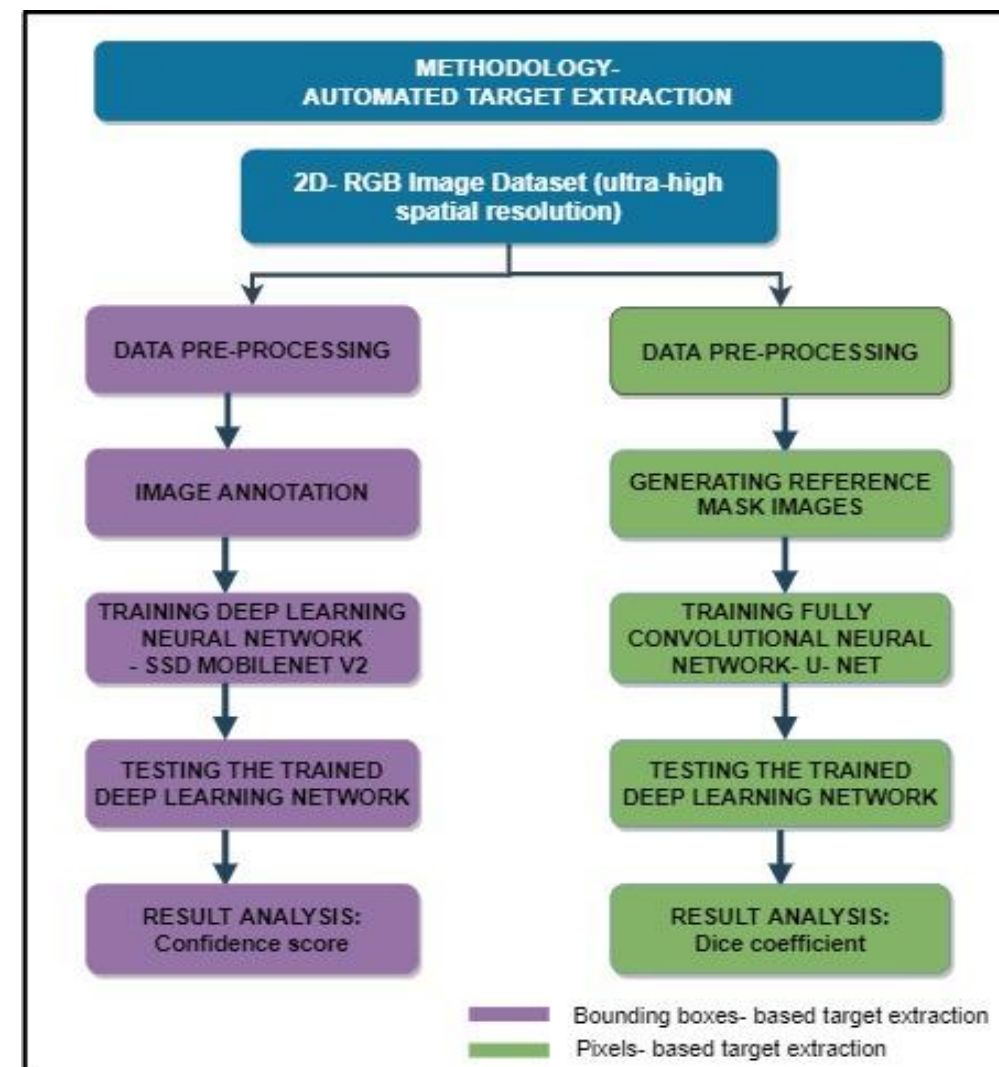
Automated target extraction using ultra-high spatial resolution images

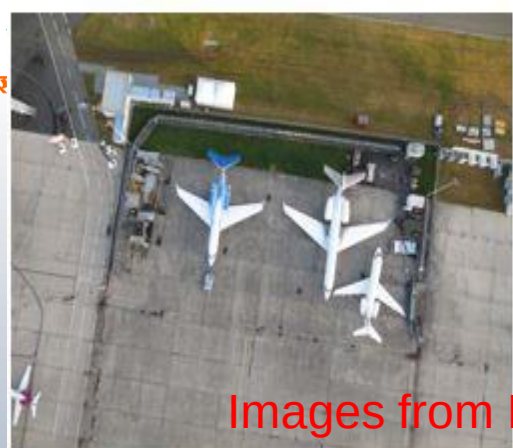


Test Image

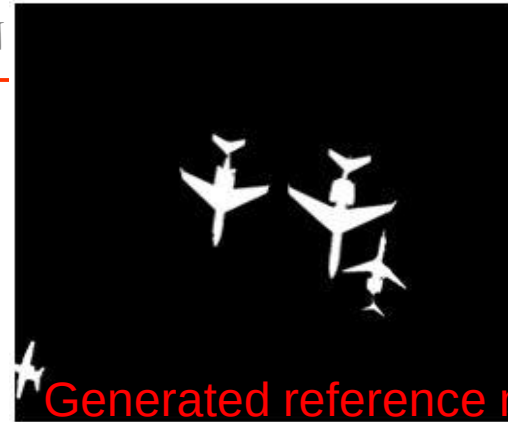


Result





Images from Dataset

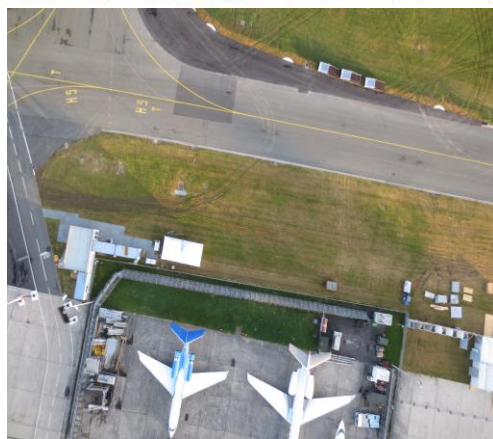


Generated reference mask images

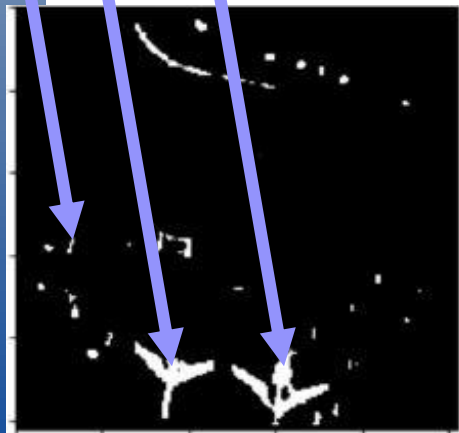


Noise

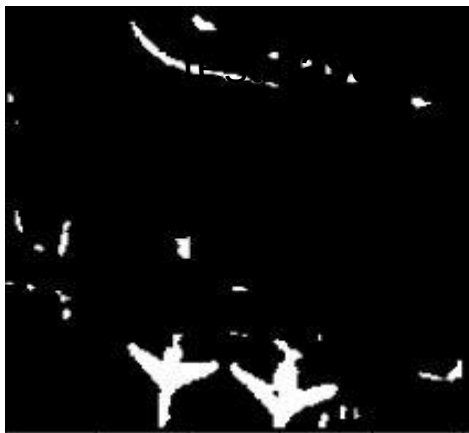
Target



Reference mask image



Epochs:30



Epochs:50



Epochs:70



Epochs:100



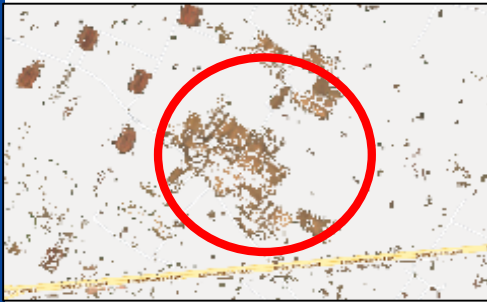
Epochs:150

Automatic detection and monitoring of brick kiln

Objective: To develop an efficient and a scalable solution for automated detection and mapping of brick kilns using satellite



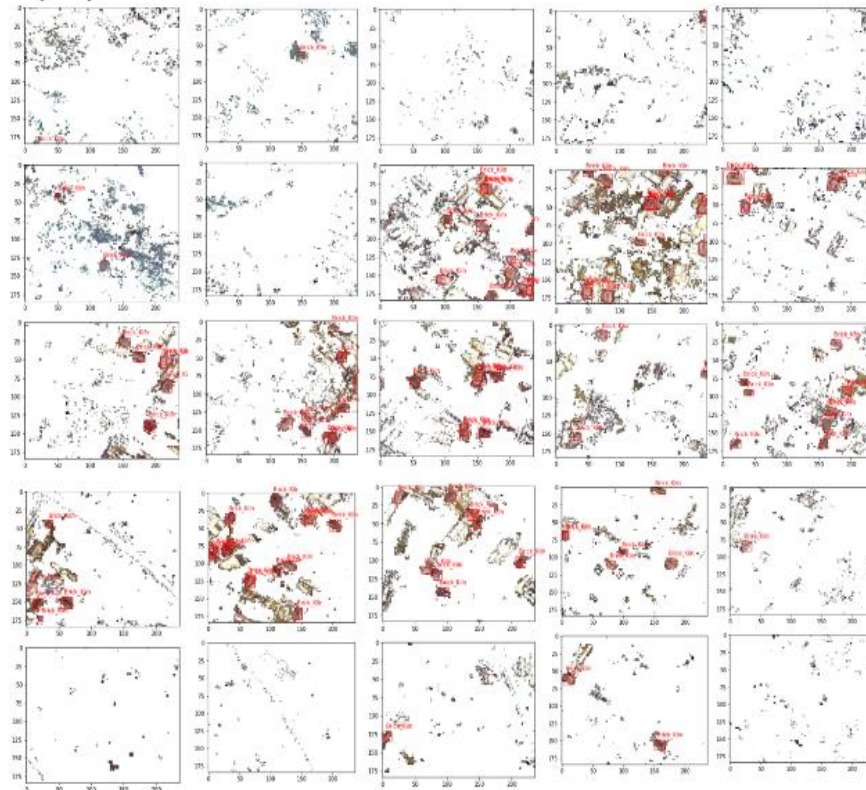
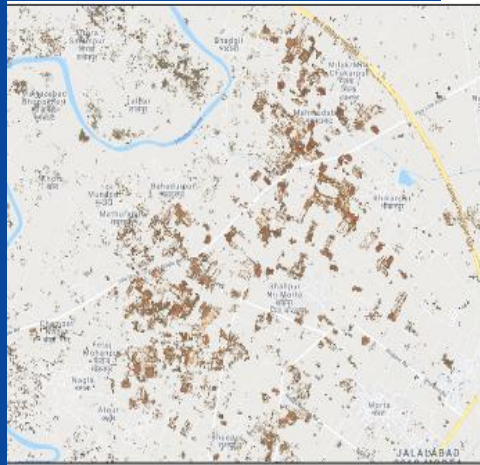
Sentinel 2A



False Positive (RF classifier)



FP improved after

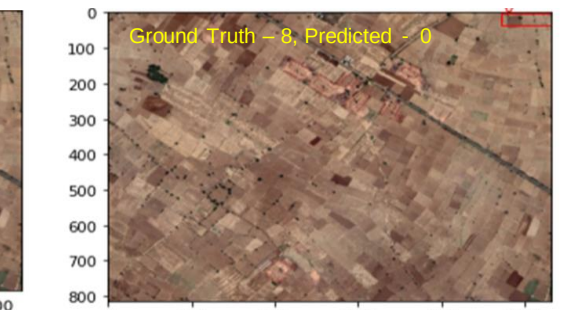
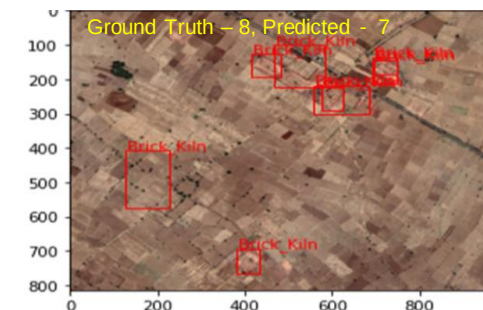
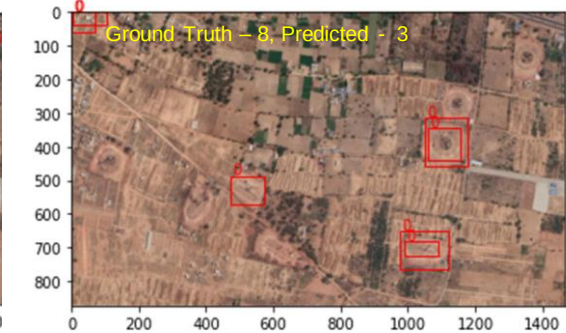
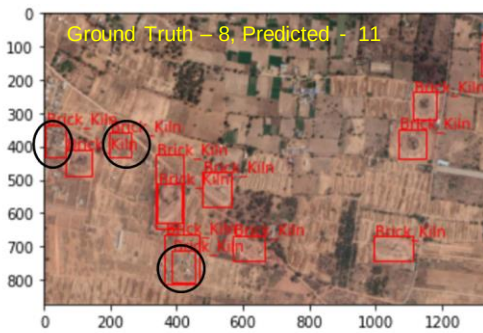
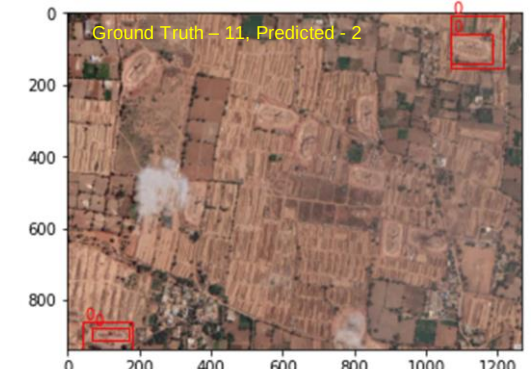
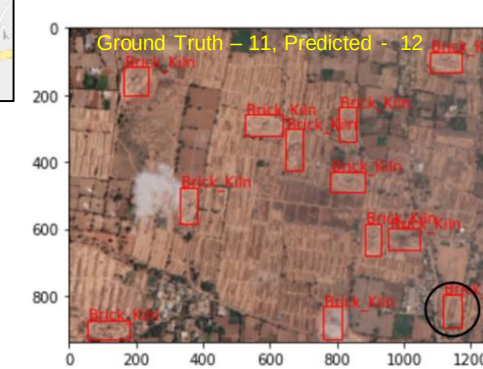


	Predicted True	Predicted False	Recall
Actual True	74	0	1.0
Actual False	16	-	
Precision	0.82		F1 Score =0.90

Comparing two training models created with ResNet-50 using 2 different dataset.

Model 1 – training using **Google Earth Pro images**.

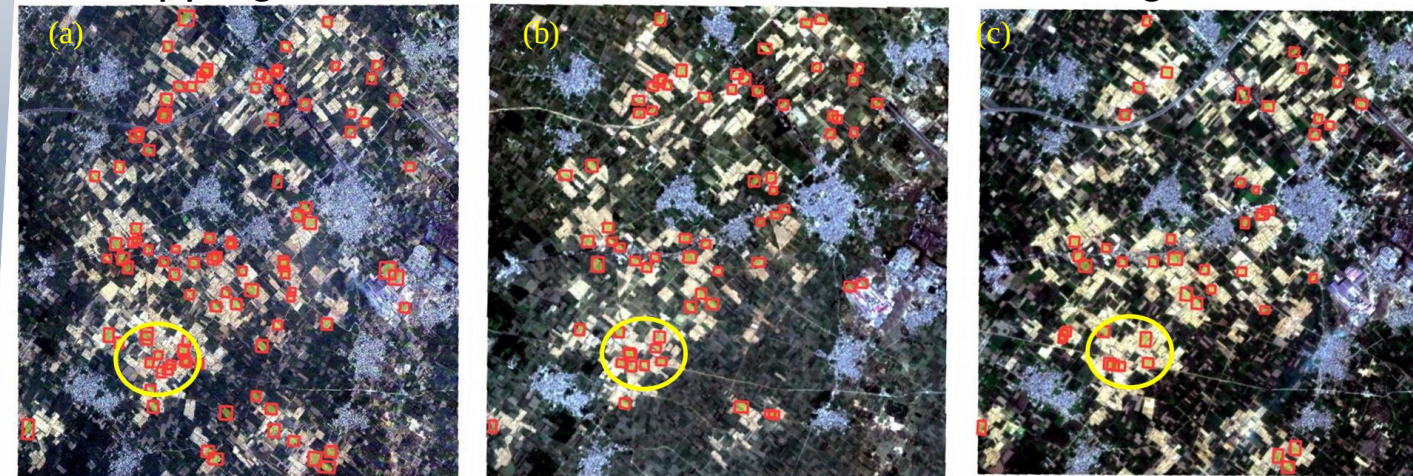
Model 2 – training using the **ESRI dataset**



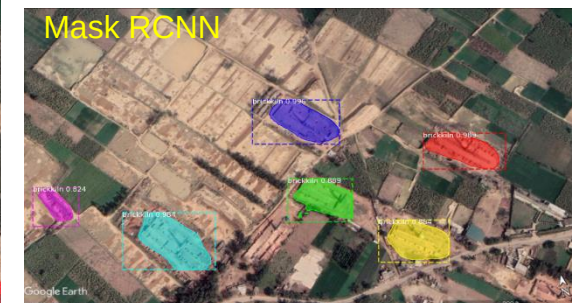
Brick Kiln extraction results (East Delhi/UP)

Scalable Framework For Deep Learning Based Detection And Segmentation of Brick Kilns Using Satellite Images

1. To evaluate the existing models for detection and segmentation and selection of optimal model for brick kilns.
2. To create a scalable framework for extracting brick kilns.
3. Mapping of active and abandoned brick kilns using time series data.

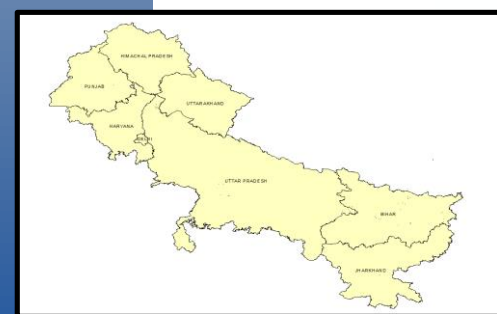


Brick Kiln Segmentation using YOLOv7 on study site in (a) Cartosat-3 MX Imagery (b) PlanetScope –DOVE and (c) Sentinel-2

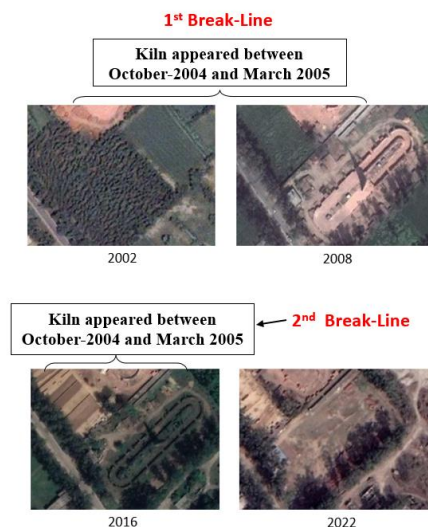
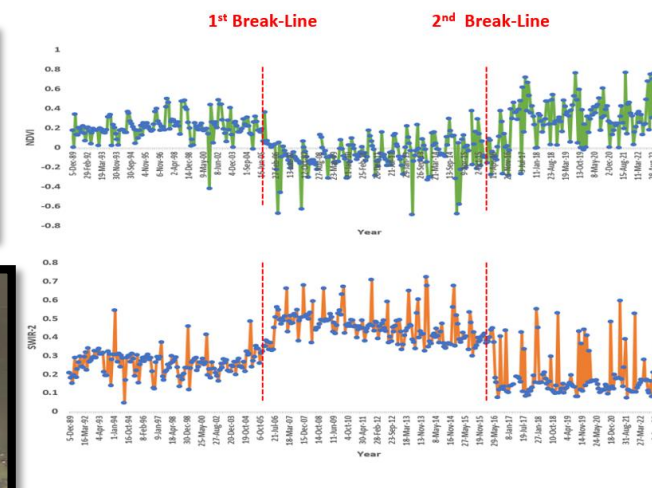
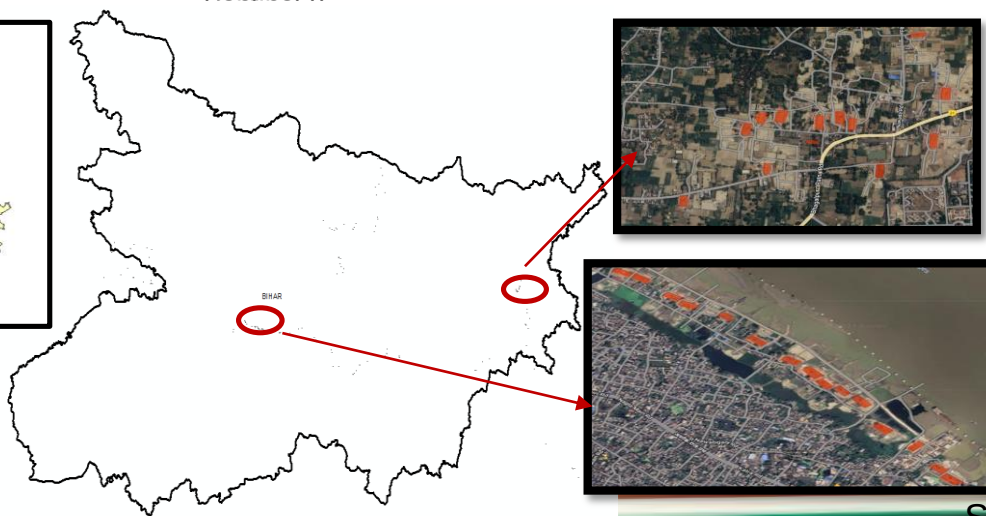


Optimal model selection : a) Yolo V7 (91.3%); b) Mask RCNN (76.8%)

Satellite	Precision	Recall	F1-score
Cartosat-3 MX	0.949	0.878	0.912
PlanetScope-DOVE	0.878	0.607	0.717
Sentinel-2	0.850	0.532	0.654



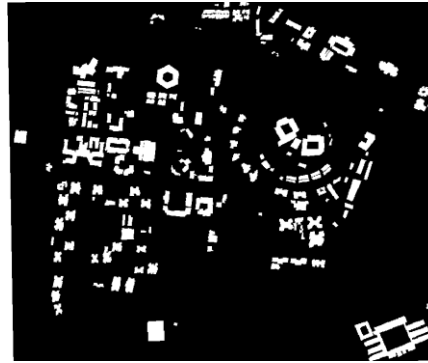
Brick Klin Inventory using AI in brick kiln belt of northern India



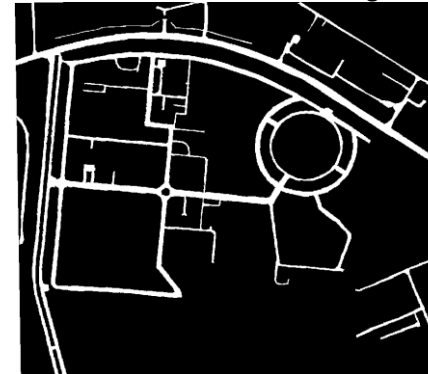
Spectral Time Series Analysis for mapping active and abandoned kilns

High resolution satellite data and ultra high resolution UAV data used for urban feature extraction

Objective: Semantic Segmentation of High Resolution image for urban Feature extraction using Convolutional Neural Network.



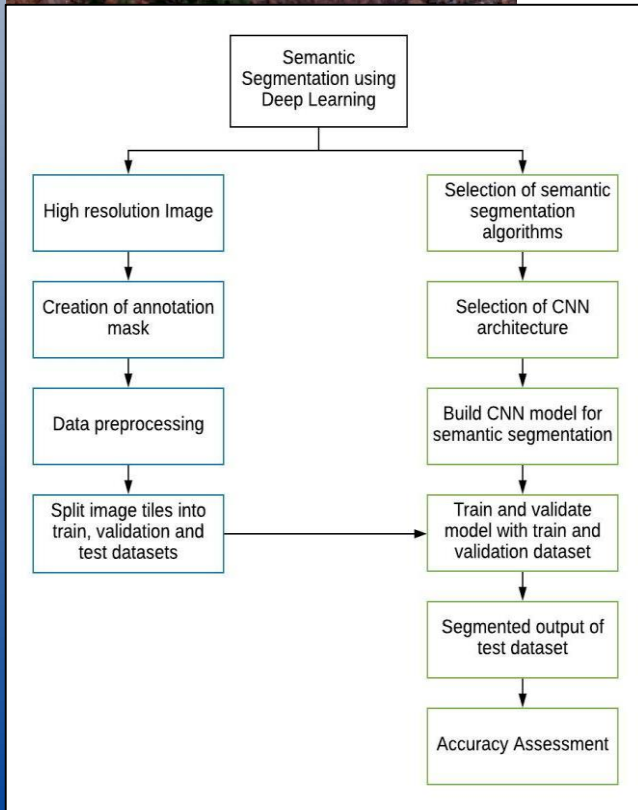
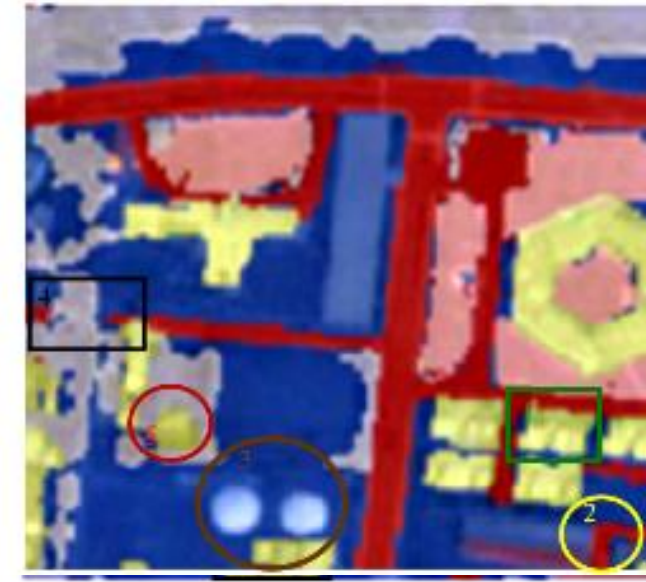
Annotations for buildings



Annotations for roads



Annotations for vegetation



Highlighted area no.	Description
1	The model segmented closely placed building and road features with high accuracy retaining their shapes.
2	The road that is difficult to perceive through human vision is segmented accurately.
3	Structures not included in the training, classified in void class but represented by different color schema.
4	Due to mixed pixels, the road is misclassified.
5	A building is surrounded by mixed pixels is segmented accurately.

FEATURE EXTRACTION IN AN URBAN ENVIRONMENT USING A HYBRID OBIA-CNN APPROACH

Segmentation Algorithm Used: Felzenswalb Segmentation; Parameters used : Scale, Sigma, Size

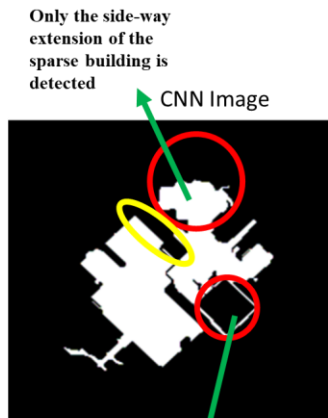
Classification Report:

	precision	recall	f1-score	support
commercial buildings	0.79	0.68	0.73	50
crossroads	0.94	0.92	0.93	50
denseresidential	0.82	0.36	0.50	50
flyovers	0.92	0.96	0.94	50
mediumresidential	0.61	0.82	0.70	50
national highway	1.00	0.88	0.94	50
parking lots	1.00	0.94	0.97	50
river	0.98	0.94	0.96	50
sparseresidential	0.97	0.74	0.84	50
storagetanks	0.53	0.96	0.69	50
accuracy			0.82	500
macro avg	0.86	0.82	0.82	500
weighted avg	0.86	0.82	0.82	500

CNN Architecture Model used:
ResNet152V2



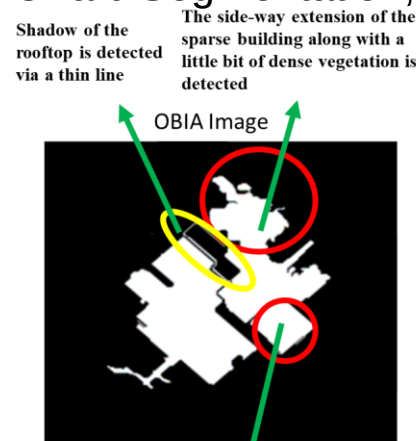
Original Image



Only the side-way extension of the sparse building is detected

CNN Image

The edges of the rooftop are indicated



Shadow of the rooftop is detected via a thin line

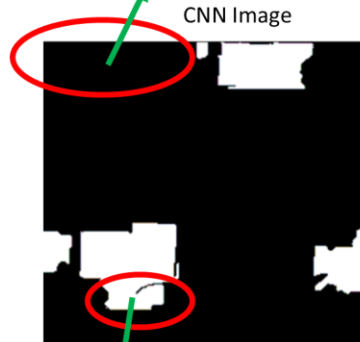
The side-way extension of the sparse building along with a little bit of dense vegetation is detected

OBIA Image

The edges of the rooftop along with the rooftop are indicated



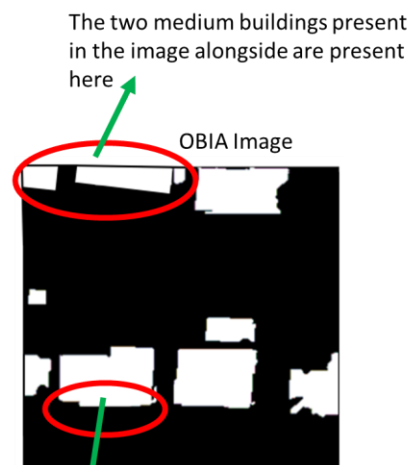
Original Image



The two medium buildings present in the image alongside are absent here

CNN Image

A little built-up area surrounding the medium residential building is also extracted



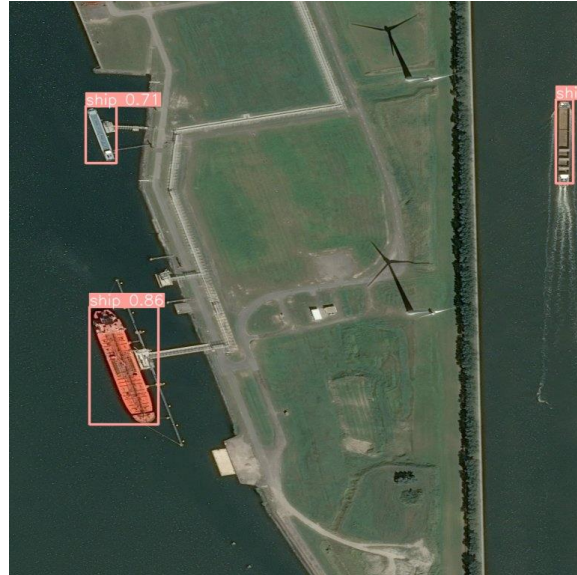
The two medium buildings present in the image alongside are present here

OBIA Image

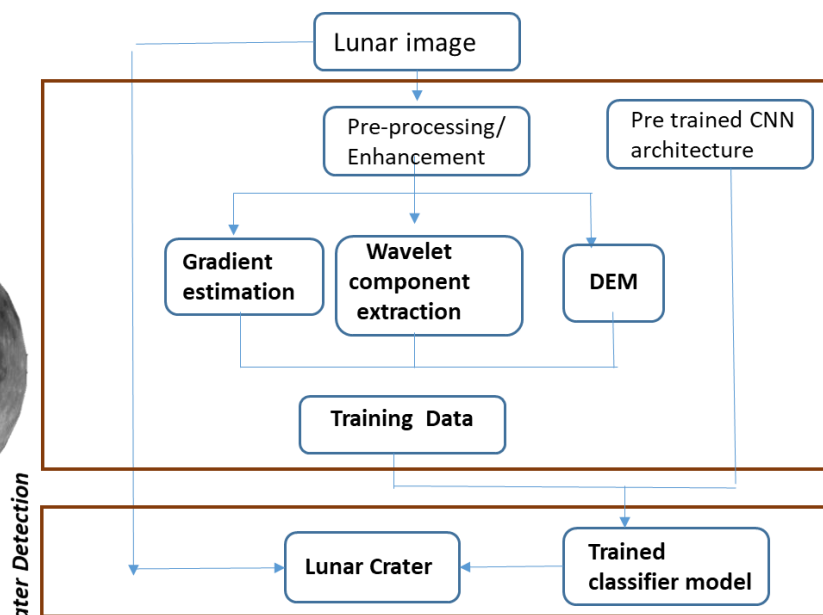
A little built-up area surrounding the medium residential building is absent

Sl. No	Features	User Accuracy (UA)	Producer Accuracy (PA)	Overall Accuracy (OA)
1	Commercial Buildings	73.17%	83.31%	79.27%
2	Crossroads	84.65%	87.25%	85.63%
3	Dense Residential	81.23%	83.07%	84.51%
4	Flyovers	82.52%	84.37%	80.00%
5	Medium Residential	75.00%	73.26%	71.32%
6	National Highway	72.47%	76.21%	74.64%
7	Parking Lots	75.55%	70.00%	74.07%
8	River	83.33%	87.25%	90.91%
9	Sparse Residential	89.04%	84.15%	88.25%
10	Storage Tanks	78.91%	73.67%	77.82%

SHIP DETECTION USING YOLOV8

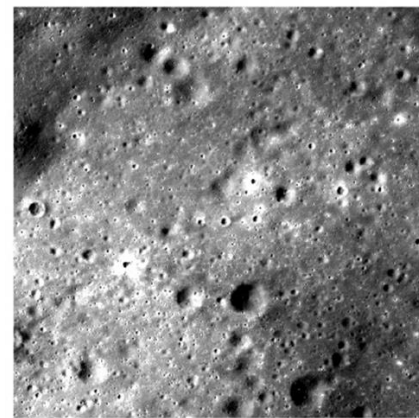
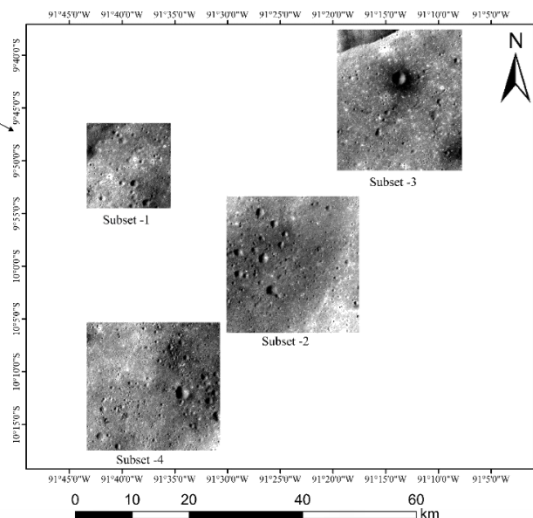
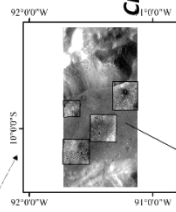
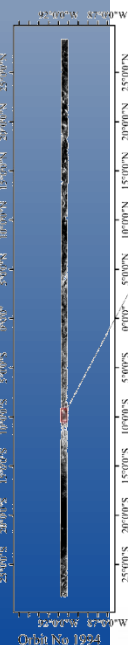
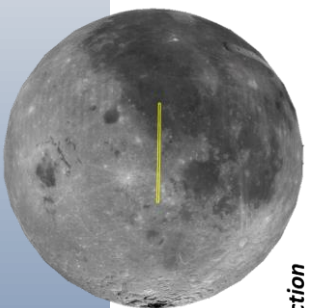


Automatic Lunar Crater Detection with Chandrayaan2 TMC data using Machine Learning

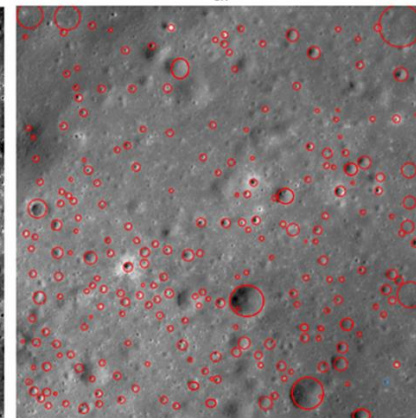


Crater Detection

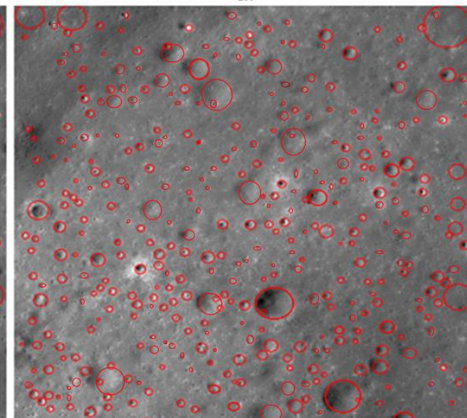
Lunar Crater detection



Original Image subset



Model output without DEM



Model Output with DEM

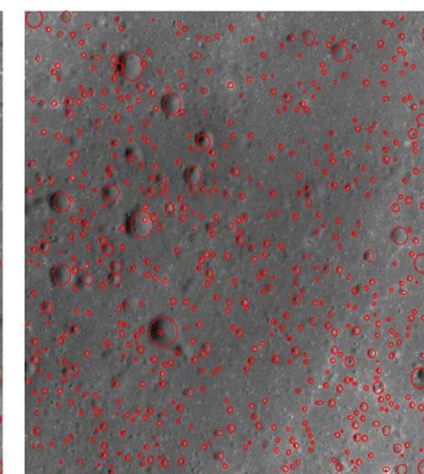
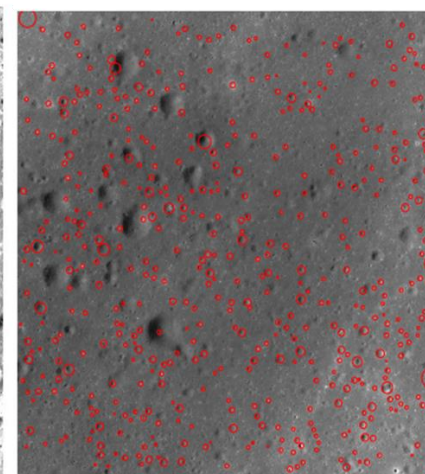
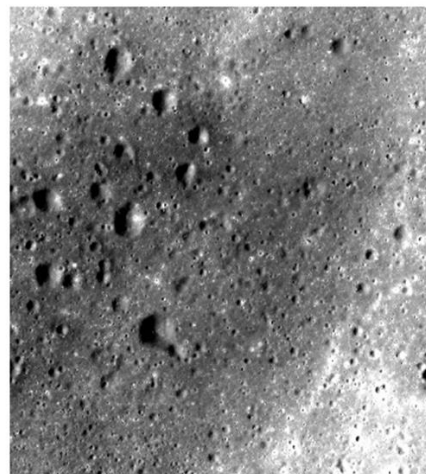


Image Subse t	Traini ng Loss	Validation Loss	Accura cy	IoU	Time/ Epoch	Referenc e Area	No. of craters detected	Craters (Referenc e)	False Positive	True Negative	Accuracy
1	1.12	0.33	0.68	0.69	19	1	78	66	5	17	78%
2	0.24	0.32	0.71	0.72	04	2	97	87	9	19	80%
3	0.68	0.54	0.75	0.83	08	3	114	91	12	35	69%
4	0.53	0.42	0.68	0.74	09	4	86	76	11	21	75%

Model Accuracy

Crater Detection Accuracy

Rooftop Solar Power plant installation through Machine Learning

Image Segmentation

- The Watershed algorithm is especially useful when extracting touching or overlapping objects are in the images.
- The algorithm is very fast and computation inexpensive.



Original image(left). The output from the Watershed model(right)

Edge Detection

- In the traditional Canny edge detection algorithm, there are two fixed global threshold values to filter out the false edges.
- However, as the image gets complex, different local areas will need very different threshold values to accurately find the real edges.



The output from the Canny Edge detection algorithm

Semantic Segmentation

- Here the image is segmented into two parts — rooftop or not a rooftop.
- Semantic segmentation attempts to partition the image into semantically meaningful parts and to classify each part into one of the predetermined classes.

U-NET

Adam optimizer

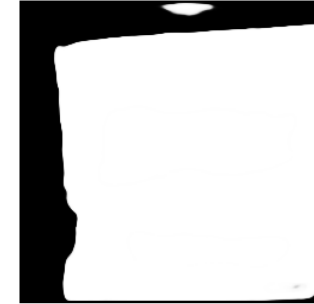
learning rate = $1e-4$

200 epochs

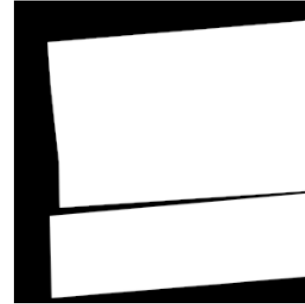


We want to segment the image into two segments — roof and not roof(left) for a given input image(right).

Predicted



Target



Predicted



Target



Predicted



Target



training accuracy was: 0.862 and the average validation accuracy was: 0.793.

Farm Segmentation From Satellite Images Using Holistically-nested Edge Detection

Image Augmentation

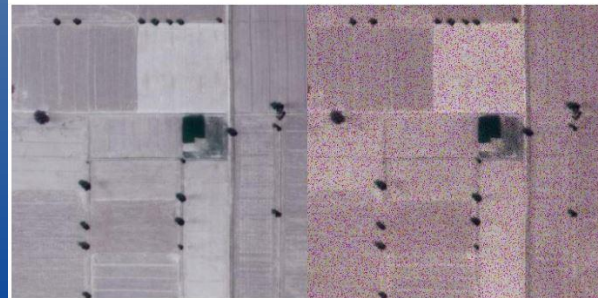
- Technique that is used to artificially expand the data set.
- Helps in reducing the manual ground trothing.
 - a. Scaling
 - b. Flipping
 - c. Rotating
 - d. Gaussian Noise



Original Image → Horizontally flipped → Vertically flipped



Original Image → Rotated 90 degrees



Original Image → With Gaussian Noise

Image Labelling

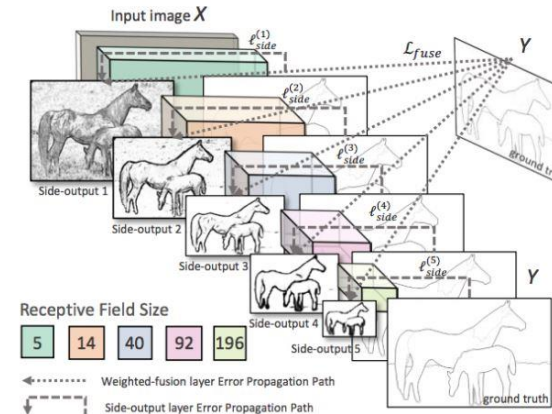
- Manual Labelling of agricultural lands on satellite imagery.
- Extracted the binary image after labelling.



Extracted Binary image after completing labelling

Holistically-nested Edge Detection

- Algorithm is based on CNN architecture consisting of 5 convolutional layers.



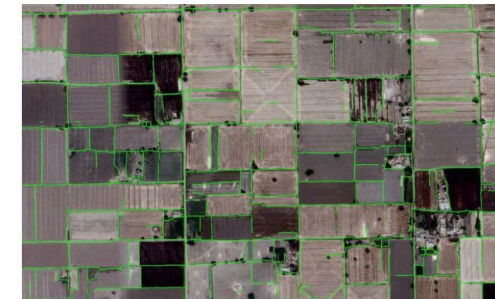
Network Architecture for HED highlighting the error backpropagation paths. Side-output layers are inserted after each convolutional layer. Supervision is imposed at each side-output layer, guiding the side-outputs towards edge predictions with the characteristics we desire. The outputs of HED are multi-scale and multi-level, with the side-output-plane size becoming smaller and the receptive field size becoming larger. One weighted-fusion layer is added to automatically learn how to combine outputs from multiple scales. The entire network is trained with multiple error propagation paths (dashed lines).

$$\mathcal{L}_{\text{side}}(\mathbf{W}, \mathbf{w}) = \sum^M \alpha_m \ell_{\text{side}}^{(m)}(\mathbf{W}, \mathbf{w}^{(m)}), \quad (1)$$

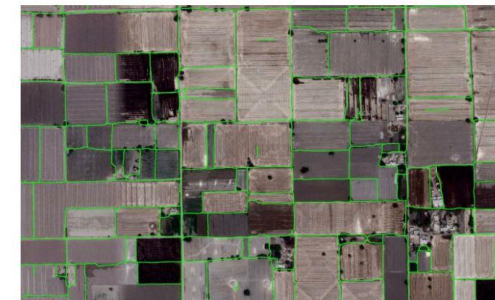
Side-output error for each layer. weights are denoted as $\mathbf{w} = (\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(M)})$, where $\ell(\text{side})$ denotes the image-level loss function for side outputs

Contour Detection, Plotting and Post processing

- Algorithm is based on CNN architecture consisting of 5 convolutional layers.



Satellite Image with prediction after dilation and skeletonization. Hanging edges still remain.

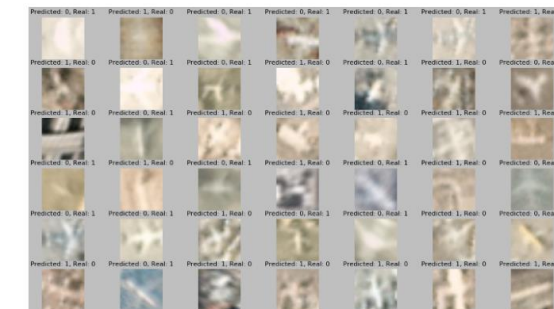
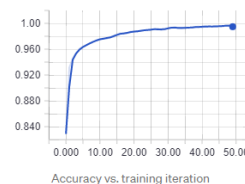


Satellite image with contours after post-processing. Most of the hanging edges are removed.

Automatic target detection in satellite images using deep learning

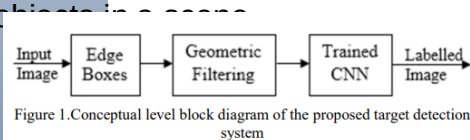
1. Airplane detection

[1] The author proposed a Convolutional Neural Network (CNN) for classifying airplane.



Output showing all of the false positives and false negatives in the test dataset

[2] The author proposed a target detection system for satellite imagery, which uses EdgeBoxes and Convolutional Neural Network (CNN) for classifying target and non-target



(c) Target Detection Results.

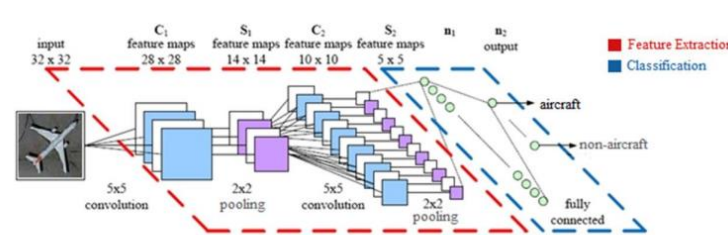


Figure 4. CNN Framework used for feature extraction and classification in the proposed system



Figure 5. Results of the proposed target detection system on some test images from the aircrafts dataset.

[3] The author proposed a saliency and symmetry based object detection that provides stably in obtaining target location and orientation information. The saliency computation and MSER detection algorithm yield initial ROIs. In each ROI, target orientation is estimated by symmetry detection, and then airplane is detected via shape matching.

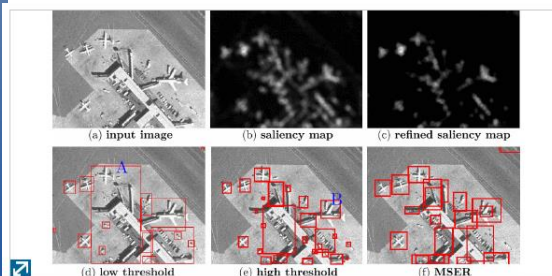


Fig. 1. An example of the Saliency Based ROI Extraction stage. (b) gives the initial saliency map, (c) depicts the saliency map refined by spatial competition, (d) and (e) show ROIs extracted from directly thresholding the saliency map using threshold 0.2, and 0.4, respectively. Comparatively, (f) illustrates the ROIs extracted with the proposed method

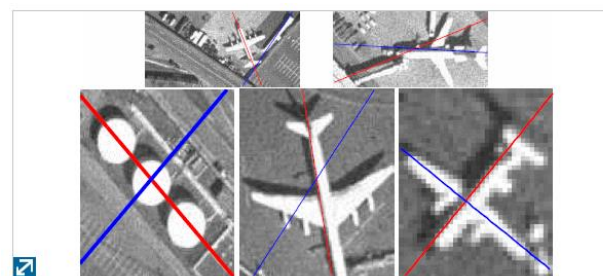


Fig. 2. Symmetry axis detection results. Red line: most likely axis; Blue line: second most likely axis



Fig. 4. Illustrating airplane detection results. Red rectangle: true positive; Green rectangle: false negative



Hybrid deep convolutional neural networks (HDNN), by dividing the maps of the last convolutional layer and the max-pooling layer of DNN into multiple blocks of variable receptive field sizes or max-pooling field sizes, to enable the HDNN to extract variable-scale features. Multiple gradients was computed by applying various threshold limit on images. Then sliding window technique was applied in order to locate the vehicle position.

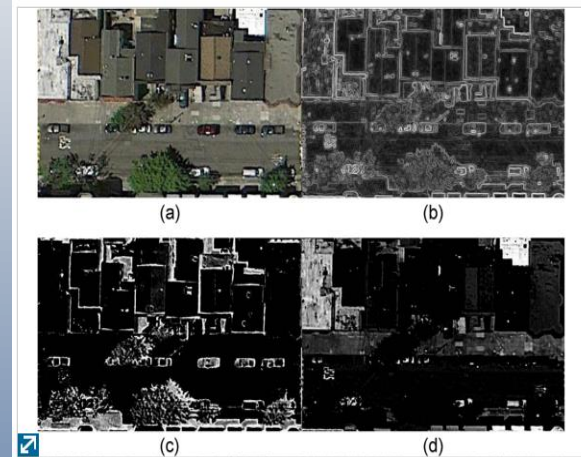


Fig. 3. (a) Original image. (b) Gradient of Dalal and Triggs [15]. (c) Gradient on image thresholding below 60. (d) Gradient on image thresholding from 100.

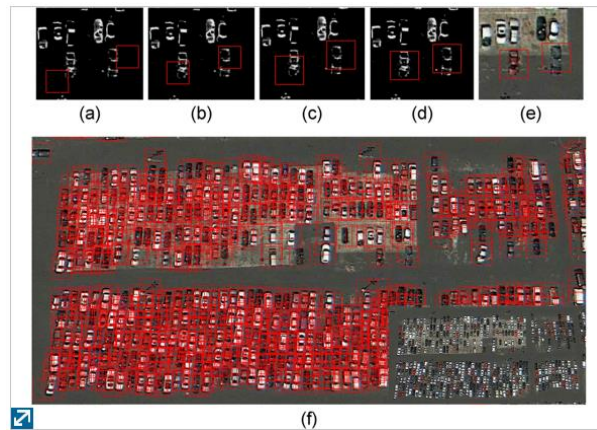


Fig. 4. (a)–(e) Process of locating vehicle by the algorithm 1 on the negative gradient image (threshold = 60). (f) Final locating window set, 99.2% vehicle are exactly located in this complex large park. (a) Initial windows. (b) Move to center. (c) Enlarge twice. (d) Move to center. (e) Original image. (f) The locating window set after filtering repetitive sliding windows.

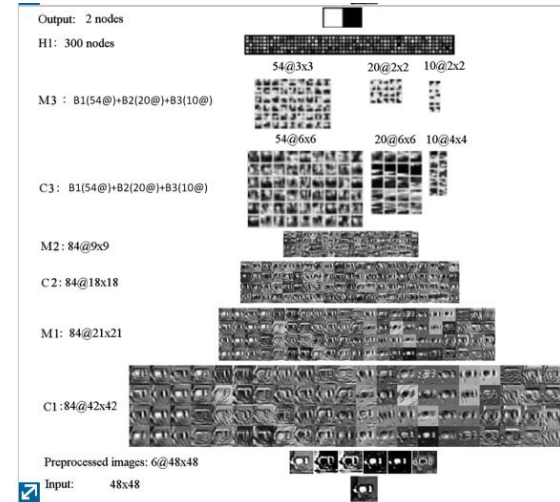


Fig. 5. Structure of HDNN (54-20-10). Where $n_l = 3$, $n_m = 84$, and $n_h = 3$. B_1 has 54 maps, B_2 has 20 maps, and B_3 has 10 maps.



Fig. 7. Some detection results by HDNN (44-20-20) in the City of San Francisco. (Red frames) Cars. (Yellow frames) False alarms.

An automatic approach, consisting of a segmentation step, followed by two stages of object classification. Author applied Linear discriminant analysis (LDA), which is a classical statistical approach for classifying samples of unknown classes, based on training samples with known classes which maximizes the ratio between-class variance/within-class variance. Though the study uses high resolution quick bird images but according to the study 20% of the vehicles are lost due to low contrast and fragmented object.

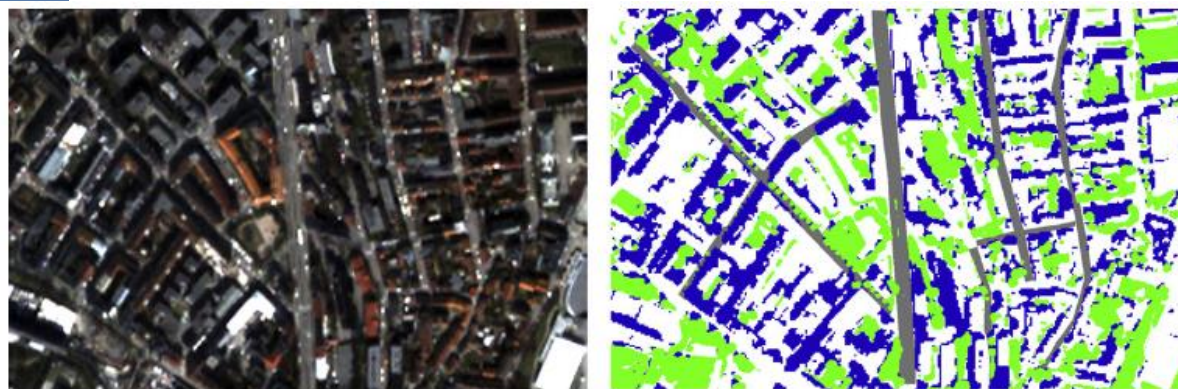


Fig. 2. RGB image (left) and the combined mask (right) with a subset of roads (grey), vegetation (green) and shadows (blue).

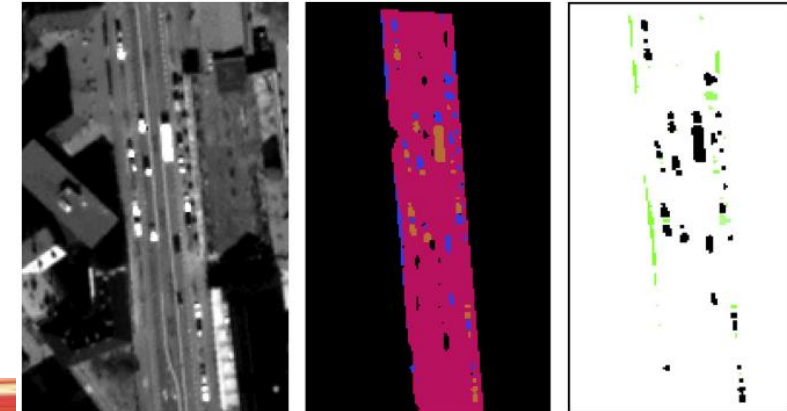


Fig. 3. Result of the segmentation on a part of the road from the GO subimage. Left: Grey level image, Middle: Segmented road surface (pink), dark objects (blue), bright objects (orange) and not within mask (black). Right: Result of pre-classification, where the regions that are black correspond to objects identified as potential vehicles.



Thank You.....

A decorative horizontal bar at the bottom of the slide, featuring a gradient of colors from orange to red to green.